



Goethe
Universität
Frankfurt am Main



The present work was submitted to Lehr- und Forschungsgebiet Educational Technologies at DIPF

Span-based Classifier for Short Answer Scoring and Beyond

Masterarbeit

Vorgelegt von

al Mahdi, Malaz

8053932

Erstgutachter: Prof. Dr. Hendrik Drachsler

Zweitgutachter:

Betreuung: Zhifan Sun (DIPF)

Frankfurt, 14. September 2026

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

**gemäß § 35, Abs. 16 der Ordnung für den Masterstudiengang Informatik
vom 13. Juli 2015**

Hiermit erkläre ich

(Nachname, Vorname)

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Ebenso bestätige ich, dass diese Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde.

Zudem versichere ich, dass die von mir eingereichten schriftlichen gebundenen Versionen meiner Masterarbeit mit der von mir eingereichten elektronischen Version der Masterarbeit übereinstimmen.

Frankfurt am Main, den

Unterschrift der/des Studierenden

Acknowledgments

Note - *This section is optional if you want to thank someone for their support in writing your thesis. The acknowledgements section could look like the following:*

My deepest gratitude goes to my family. I would especially like to thank my father and mother for always supporting and motivating me in the past. Without them this thesis/project would have never been finished.

I would also like to thank Prof. Dr. XYZ and Prof. Dr. ABC for supporting this thesis/project work. I would like to express my gratitude to Mrs/Mr. M.Sc. XZY for being a great supervisor, without his/her help this journey would have been extremely hard.

And finally, a big shout-out to all of my friends and my colleagues at Goethe University.

Abstract

Note - *In most cases it is easier to write the abstract at the end than trying to finish it directly at the beginning.*

An abstract is a brief summary of a research article, thesis, review, conference proceeding, or any in-depth analysis of a particular subject and is often used to help the reader quickly ascertain the paper's purpose [6]. When used, an abstract always appears at the beginning of a manuscript or typescript, acting as the point-of-entry for any given academic paper or patent application. Abstracting and indexing services for various academic disciplines are aimed at compiling a body of literature for that particular subject.

It may also contain brief references, although some publications' standard style omits references from the abstract, reserving them for the article body (which, by definition, treats the same topics but in more depth).

Abstract length varies by discipline and publisher requirements. Typical length ranges from 100 to 500 words, but very rarely more than a page and occasionally just a few words [4]. An abstract may or may not have the section title of "abstract" explicitly listed as an antecedent to content.

According to [53] an abstract should be:

- **Objective** - It should refrain from any personal evaluation,
- **Short** - It should be as short as possible,
- **Comprehensible** - It should have a clear, comprehensible language and structure,
- **Complete** - All essential facts should be included,
- **Accurate** - It should accurately reflect the content and the opinion of the original work

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Problemstellung	1
1.3	Forschungsfragen	1
1.4	Aufbau der Arbeit	1
2	Grundlagen	3
2.1	Automatic Short Answer Grading (ASAG): Aufgabendefinition	3
2.2	Evaluationsmetriken im ASAG	7
2.3	Encoder-basierte Ansätze im ASAG	12
2.4	LLM-basierte Ansätze im ASAG	13
2.5	Rubrik-geleitetes Scoring: Guided-Alignment-Architekturen	15
2.6	Grundlagen der Transformer-Architektur	17
2.6.1	Self-Attention und Multi-Head Attention	18
2.6.2	Kausale vs. bidirektionale Attention-Masken	22
2.7	Parameter-effizientes Finetuning (LoRA)	25
2.8	Verwandte Arbeiten zu Fusionsmechanismen bei Rubrik-Antwort-Alignment . .	27
3	Datensätze	31
3.1	ASAP-SAS: Struktur, Statistik, Label-Skala	31
3.2	Beetle: Domäne und Test-Splits	33
3.3	SciEntsBank: Domäne und Test-Splits	34
3.4	Herkunft der Rubriktexte und Musterlösungen	35
3.5	Vergleich der Benchmarks und Auswahlbegründung	36
4	Methodik: Das RUSPAN-System	39
4.1	Systemarchitektur: Span-Alignment-Modell	39
4.1.1	Eingabeformat	40
4.1.2	Span-Extraktion und Pooling	42
4.2	Fusionsmechanismen	44
4.2.1	CONDIFF: Conditional-Difference-Fusion	44
4.2.2	CONCAT: Einfache Konkatenation als Baseline	44
4.2.3	DIFF: Differenz als alleinige Eingabe	45
4.2.4	GATE: Gated Fusion	45

4.3	Bidirektionale Attention in Decoder-only-Modellen	47
4.3.1	Motivation	47
4.3.2	Umsetzung: Anteilige Aufhebung der kausalen Maske	48
4.4	Verwendete Backbone-Modelle	50
4.4.1	Llama-3.2-1B/3B	50
4.4.2	Mistral-7B	50
4.4.3	BERT/RoBERTa als Encoder-Baseline	50
4.5	Varianten für die Ablationsexperimente	51
4.5.1	Fester Klassifikationskopf als Baseline	51
4.5.2	Ablation der Rubriktexte	51
4.5.3	Variation des Aufgabenkontexts	52
4.5.4	Antwort-Span als Bezugsgröße der Fusion	52
4.6	Trainingskonfiguration	52
5	Experimente und Ergebnisse	57
5.1	Experimentelles Design	57
5.2	Rauschgrenze: Streuung zwischen Läufen	59
5.3	Der Beitrag aufgabenbezogener Information	61
5.4	Fusionsmechanismen	66
5.5	Bezugsgröße der Fusion	69
5.6	Span-Alignment gegenüber einem festen Klassifikationskopf	70
5.7	Attention-Maske	71
5.8	Effekt der Modellgröße	75
5.9	Encoder- gegenüber LLM-basierten Ansätzen	77
5.10	Ergebnisse je Aufgabe auf ASAP-SAS	79
5.11	Instabile Läufe	80
5.12	Robustheit gegenüber der Metrik	81
5.13	Zusammenfassung der Befunde	84
6	Diskussion	87
7	Fazit und Ausblick	89
	Tabellenverzeichnis	91
	Abbildungsverzeichnis	95
	Literatur	97
	Appendices	103

1 Einleitung

1.1 Motivation

1.2 Problemstellung

1.3 Forschungsfragen

1.4 Aufbau der Arbeit

2 Grundlagen

Das vorliegende Kapitel legt die begrifflichen, methodischen und technischen Grundlagen, auf denen das in Kapitel 4 beschriebene RUSPAN-System aufbaut. Zunächst wird die Aufgabe des Automatic Short Answer Grading formal gefasst und von benachbarten Aufgabenstellungen abgegrenzt (Abschnitt 2.1). Anschließend werden die in dieser Arbeit verwendeten Evaluationsmetriken hergeleitet und hinsichtlich ihrer Messvoraussetzungen diskutiert (Abschnitt 2.2). Die Abschnitte 2.3 und 2.4 referieren den Stand der Forschung zu Encoder- und LLM-basierten Verfahren; Abschnitt 2.5 führt in rubrik-geleitete Scoring-Architekturen ein, aus denen sich der Ansatz dieser Arbeit unmittelbar ableitet. Die technischen Voraussetzungen – Transformer-Architektur, Self-Attention, Attention-Masken und parameter-effizientes Finetuning – werden in den Abschnitten 2.6 bis 2.7 entwickelt. Abschnitt 2.8 ordnet die untersuchten Fusionsmechanismen in die breitere Literatur zur Kombination von Vektorrepräsentationen ein.

2.1 Automatic Short Answer Grading (ASAG): Aufgabendefinition

Begriffsbestimmung und Abgrenzung

Unter Automatic Short Answer Grading (ASAG) wird die maschinelle Zuweisung einer Bewertung zu einer frei formulierten, kurzen Schülerantwort verstanden, die auf eine inhaltlich determinierte Frage gegeben wurde. Der maßgebliche Überblicksartikel von Burrows, Gurevych und Stein grenzt die Aufgabe über fünf Merkmale ab: Die Frage verlangt einen erinnerten oder erschlossenen Sachverhalt, die Antwort ist in natürlicher Sprache formuliert, sie umfasst typischerweise eine bis wenige Sätze, die Bewertung bezieht sich auf den Inhalt und nicht auf die sprachliche Form, und der Antwortraum ist offen, aber sachlich eng begrenzt [7]. Damit unterscheidet sich ASAG systematisch vom Automated Essay Scoring (AES), bei dem längere Texte primär hinsichtlich Argumentation, Struktur und sprachlicher Qualität beurteilt werden [44], sowie von Multiple-Choice-Formaten, bei denen kein sprachliches Verstehen erforderlich ist. In der jüngeren Literatur wird zunehmend die Bezeichnung Automatic Short Answer Scoring (ASAS) verwendet, um zu betonen, dass ein Score und nicht eine Note im schulrechtlichen Sinne vergeben wird [2]. Die vorliegende Arbeit verwendet beide Bezeichnungen synonym und folgt dem etablierteren Kürzel ASAG.

Die praktische Relevanz der Aufgabe ergibt sich aus dem Missverhältnis zwischen dem didaktischen Wert konstruierter Antworten und ihren Korrekturkosten: Konstruierte Antworten erlauben im Gegensatz zu geschlossenen Formaten Rückschlüsse auf Fehlvorstellungen und

Begründungsstrukturen, ihre manuelle Bewertung skaliert jedoch linear mit der Zahl der Lernenden [2, 7].

Formalisierung

Ein Datensatz für ASAG wird als Menge annotierter Instanzen

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \quad (2.1)$$

aufgefasst, wobei y_i das von menschlichen Beurteilenden vergebene Referenzlabel ist. Jede Instanz x_i ist ein Tupel textueller Felder

$$x_i = (q_i, s_i, a_i, \mathcal{R}_i), \quad (2.2)$$

bestehend aus der Aufgabenstellung q_i (*question*), einer Musterlösung oder Referenzantwort s_i (*sample solution*), der zu bewertenden Schülerantwort a_i (*answer*) sowie der Menge

$$\mathcal{R}_i = (r_{i,0}, r_{i,1}, \dots, r_{i,K_i-1}) \quad (2.3)$$

der K_i Rubrik-Beschreibungen, die jeweils eine Leistungsstufe sprachlich charakterisieren. Die Felder s_i und $\mathcal{R}_i$ sind benchmarkabhängig verfügbar; ihre Herkunft für die in dieser Arbeit verwendeten Datensätze wird in Kapitel 3 dargestellt.

Die klassische Formulierung der Aufgabe besteht in einer Scoring-Funktion

$$f_\theta : (q_i, s_i, a_i) \mapsto y_i, \quad y_i \in \mathcal{Y}, \quad (2.4)$$

die auf einen *global* festgelegten Labelraum $\mathcal{Y}$ abbildet. $\mathcal{Y}$ ist in aller Regel ordinal skaliert, das heißt zwischen den Stufen besteht eine inhaltlich begründete Rangordnung, ohne dass die Abstände zwischen benachbarten Stufen als äquidistant interpretiert werden dürften. Die Kardinalität und Semantik von $\mathcal{Y}$ variiert erheblich zwischen den Benchmarks und teilweise sogar zwischen den Einzelaufgaben eines Benchmarks (vgl. Kapitel 3).

Genau diese Variabilität motiviert die in dieser Arbeit verfolgte Alternative. Statt eines globalen Labelraums wird die Aufgabe rubrik-konditioniert formuliert: Für jede Instanz existiert eine instanzspezifische Kandidatenmenge $\mathcal{Y}_i = \{0, \dots, K_i - 1\}$, deren Elemente durch die zugehörigen Rubriktexte $r_{i,k}$ semantisch beschrieben sind. Die Scoring-Funktion vergibt dann für jede Stufe einen Score

$$\sigma_\theta : (q_i, s_i, a_i, r_{i,k}) \mapsto s_{i,k} \in \mathbb{R}, \quad \hat{y}_i = \arg \max_{k \in \mathcal{Y}_i} s_{i,k}. \quad (2.5)$$

Formulierung (2.5) ist strikt allgemeiner als (2.4): Sie ist auch dann anwendbar, wenn K_i zwischen Instanzen variiert, und sie erlaubt die Übertragung auf ungesehene Rubriksätze, da das Modell keine Indexzuordnung, sondern eine Passung zwischen Antwort und Kriterienbeschreibung lernt. Die konkrete architektonische Umsetzung von (2.5) sowie die zugehörige Trainingszielfunktion werden in Kapitel 4 entwickelt; die vorliegende Arbeit vergleicht sie durchgängig mit der klassischen Formulierung (2.4) als Baseline. Abbildung 2.1 stellt beide Formulierungen einander gegenüber.

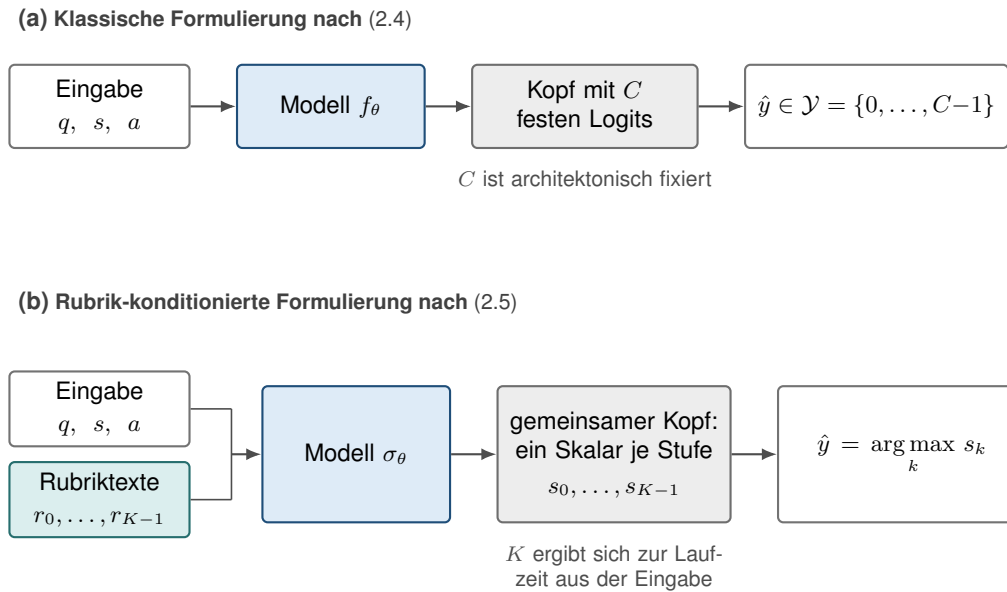


Abbildung 2.1: Die beiden Formulierungen der ASAG-Aufgabe. In (a) bildet ein Kopf fester Breite auf einen global festgelegten Labelraum ab; die Zahl der Stufen ist Teil der Architektur. In (b) erhält das Modell die Rubriktexte als zusätzliche Eingabe und vergibt über einen für alle Stufen *gemeinsamen* Kopf je Kandidatenstufe einen Skalar. Die Zahl der Ausgaben ist damit nicht mehr architektonisch fixiert, sondern folgt aus der Zahl der übergebenen Rubriken.

Generalisierungsdimensionen und Evaluationssplits

Ein einzelner Kennwert auf einer einzelnen Testmenge sagt über ein ASAG-System wenig aus, solange nicht geklärt ist, *welcher Art von Neuheit* das System dabei begegnet. Ein System, das in einer Lehrveranstaltung dieselbe Klausuraufgabe über mehrere Jahrgänge hinweg bewertet, steht vor einer völlig anderen Aufgabe als eines, das auf eine neu formulierte Frage oder gar auf ein anderes Fachgebiet angewandt wird. Die im Rahmen von SemEval-2013 Task 7 eingeführte und seither übliche Aufteilung bildet genau diese Abstufung ab, indem sie drei Testbedingungen danach unterscheidet, welche Bestandteile einer Testinstanz bereits im Training vorkamen [19]:

- *Unseen Answers* (UA): Neu ist ausschließlich die Schülerantwort. Frage, Musterlösung und Rubriktexte hat das Modell im Training bereits vielfach gesehen. Gemessen wird die Fähigkeit, die Bewertungskonvention einer *bekannten* Aufgabe zu reproduzieren.
- *Unseen Questions* (UQ): Neu sind Frage und Antwort, die Fachdomäne bleibt dieselbe. Das Modell muss das Bewertungsprinzip von der einzelnen Aufgabe abstrahiert haben, kann sich aber weiterhin auf vertraute Inhalte stützen.
- *Unseen Domains* (UD): Neu sind Frage, Antwort und Fachgebiet. Diese Bedingung stellt die höchste Anforderung, da weder die Aufgabenformulierung noch der Gegenstandsbe-reich im Finetuning vorkam.

Abbildung 2.2 stellt die drei Bedingungen als geschachtelte Neuheitsgrade dar. Die Aufteilung ist dabei nicht nominell, sondern in den verwendeten Datensätzen strukturell realisiert: Bei

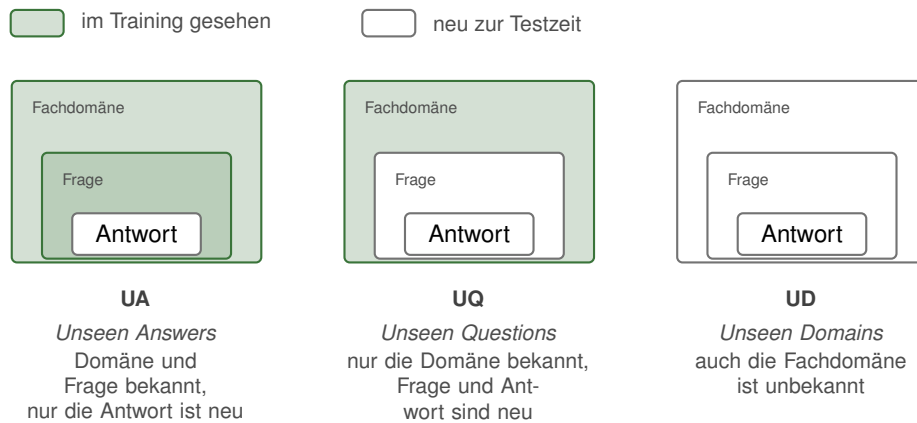


Abbildung 2.2: Die drei Evaluationsbedingungen nach [19] als geschachtelte Neuheitsgrade. Von links nach rechts wächst der Anteil der zur Testzeit unbekannten Struktur: Bei UA ist lediglich die konkrete Schülerantwort neu, bei UQ zusätzlich die Frage, bei UD auch die Fachdomäne. Da lexikalische Überschneidungen mit Trainingsantworten mit zunehmender Neuheit abnehmen, sind UQ- und UD-Werte für die Beurteilung der Übertragbarkeit aussagekräftiger als UA-Werte.

Beetle etwa verwendet der UA-Split exakt dieselben 47 Fragen wie das Training, während die 9 Fragen des UQ-Splits mit den Trainingsfragen eine leere Schnittmenge bilden. Bei SciEntsBank stammen die 46 Fragen des UD-Splits ausschließlich aus drei Themenblöcken, die in keiner einzigen Trainingsfrage auftreten (Kapitel 3).

Warum die Unterscheidung für diese Arbeit konstitutiv ist

Die Aufschlüsselung nach Generalisierungsdimension wird in dieser Arbeit nicht als Konvention übernommen, sondern aus drei inhaltlichen Gründen durchgehend beibehalten.

Erstens ist die UA-Bedingung durch lexikalische Abkürzungen lösbar. Kommt dieselbe Frage im Training vor, überschneiden sich die Testantworten sprachlich erheblich mit Trainingsantworten derselben Bewertungsstufe. Ein Modell kann dort eine hohe Übereinstimmung erreichen, indem es wiederkehrende Formulierungsmuster erkennt, ohne die Bewertungskriterien inhaltlich zu verarbeiten. Eine ausschließlich auf UA gestützte Evaluation überschätzt ASAG-Systeme daher systematisch.

Zweitens lässt sich dieser Kurzschluss in den Ergebnissen dieser Arbeit unmittelbar nachweisen. Entzieht man dem Modell sämtliche aufgabenbezogene Information und belässt ihm allein den Antworttext, so bleibt die UA-Leistung deutlich über dem Zufallsniveau, während sie auf SciEntsBank unter UQ auf die Höhe eines trivialen Raters fällt (Abschnitt 5.3). Auf Beetle, dessen UQ-Fragen aus derselben eng umgrenzten Domäne stammen, tritt dieser Zusammenbruch nicht ein – ein Befund, der die Unterscheidung nicht aufhebt, sondern sie um die Domänenbreite als zweite Einflussgröße ergänzt. UA und UQ messen also nachweislich unterschiedliche Fähigkeiten, und ein aggregierter Wert über beide Bedingungen wäre nicht interpretierbar.

Drittens – und für die Fragestellung entscheidend – ist das Versprechen der rubrik-konditionierten Formulierung (2.5) überhaupt nur unter UQ und UD prüfbar. Ihr Zweck besteht

darin, dass das Modell keine Zuordnung zu Labelindizes lernt, sondern eine Passung zwischen Antwort und Kriterienbeschreibung, und deshalb auf ungesehene Aufgaben und Rubriksätze übertragbar ist. Unter UA sind die Rubriktexte mit denen des Trainings identisch; die behauptete Übertragbarkeit wird dort schlicht nicht auf die Probe gestellt.

Aus diesen Gründen werden in Kapitel 5 sämtliche Ergebnisse nach Split getrennt berichtet, und Aussagen über die Übertragbarkeit stützen sich ausschließlich auf UQ und UD. Zugleich ist eine Einschränkung festzuhalten: Von den drei verwendeten Benchmarks stellt allein SciEnts-Bank einen UD-Split bereit (Kapitel 3). Alle Befunde dieser Arbeit zum Domänentransfer beruhen daher auf einer einzigen Benchmark-Konstellation, wenn auch auf dem mit Abstand umfangreichsten Testsplit.

2.2 Evaluationsmetriken im ASAG

ASAG-Ergebnisse werden in dieser Arbeit über drei Maße berichtet: Quadratic Weighted Kappa (QWK), Macro- F_1 und Accuracy. Alle drei Maße lassen sich aus der Konfusionsmatrix ableiten, unterstellen aber unterschiedliche Skalenniveaus und reagieren unterschiedlich auf Klassenungleichgewichte. Der Abschnitt entwickelt sie zunächst formal und diskutiert anschließend ihre Voraussetzungen.

Notation

Sei $\mathcal{Y} = \{0, 1, \dots, C-1\}$ der betrachtete Labelraum mit C Stufen, seien y_i die Referenz- und $\hat{y}_i$ die Modellvorhersagen über N Testinstanzen. Die Konfusionsmatrix $O \in \mathbb{N}^{C \times C}$ ist definiert durch

$$O_{jk} = |\{i \in \{1, \dots, N\} : y_i = j \wedge \hat{y}_i = k\}|, \quad (2.6)$$

das heißt O_{jk} zählt die Instanzen mit Referenzstufe j und Vorhersage k . Die Randverteilungen werden mit $n_j^{\text{ref}} = \sum_k O_{jk}$ und $n_k^{\text{pred}} = \sum_j O_{jk}$ bezeichnet.

Accuracy

Die Accuracy ist der Anteil exakt korrekt vorhergesagter Instanzen,

$$\text{Acc} = \frac{1}{N} \sum_{j=0}^{C-1} O_{jj}. \quad (2.7)$$

Sie ist unmittelbar interpretierbar, hat aber zwei für ASAG gravierende Nachteile. Erstens gewichtet sie alle Fehler gleich: Die Verwechslung einer maximalen mit einer minimalen Punktzahl geht ebenso ein wie die Verwechslung benachbarter Stufen, obwohl Letztere aus Bewertungssicht deutlich weniger schwer wiegt. Zweitens ist sie gegenüber unbalancierten Labelverteilungen nicht robust: Ein triviales Modell, das stets die häufigste Stufe vorhersagt, erreicht bereits die relative Häufigkeit dieser Stufe. Da die in dieser Arbeit verwendeten Benchmarks teils

deutlich unbalanciert sind (vgl. Kapitel 3), wird Accuracy ausschließlich als ergänzende Größe berichtet.

Macro- F_1

Für eine Stufe k werden Präzision und Recall klassenweise definiert als

$$P_k = \frac{O_{kk}}{n_k^{\text{pred}}}, \quad R_k = \frac{O_{kk}}{n_k^{\text{ref}}}, \quad F_{1,k} = \frac{2 P_k R_k}{P_k + R_k}, \quad (2.8)$$

wobei $F_{1,k} := 0$ gesetzt wird, falls $P_k + R_k = 0$. Das Macro- F_1 ist das ungewichtete arithmetische Mittel über alle Stufen,

$$F_1^{\text{macro}} = \frac{1}{C} \sum_{k=0}^{C-1} F_{1,k}. \quad (2.9)$$

Die ungewichtete Mittelung ist der entscheidende Punkt: Jede Stufe geht unabhängig von ihrer Häufigkeit mit gleichem Gewicht ein, sodass eine schwache Leistung auf seltenen Stufen nicht durch eine starke Leistung auf der Mehrheitsstufe kompensiert werden kann. Macro- F_1 ist damit das strengere Maß unter Klassenungleichgewicht und wird in der jüngeren ASAG-Literatur zunehmend als primäre Kennzahl verwendet. Es ist allerdings gegenüber der Ordinalität des Labelraums blind, da es die Stufen wie nominale Kategorien behandelt.

Quadratic Weighted Kappa

Cohens κ misst die Übereinstimmung zweier Beurteilungsinstanzen unter Korrektur der zufällig zu erwartenden Übereinstimmung [12]. Die gewichtete Variante κ_w verallgemeinert dieses Prinzip, indem Abweichungen unterschiedlicher Schwere unterschiedlich stark gewichtet werden [13]. Sei $E \in \mathbb{R}^{C \times C}$ die unter Unabhängigkeit der Randverteilungen erwartete Häufigkeitsmatrix,

$$E_{jk} = \frac{n_j^{\text{ref}} \cdot n_k^{\text{pred}}}{N}, \quad (2.10)$$

und sei $W \in \mathbb{R}^{C \times C}$ eine Gewichtsmatrix, die jeder Zelle die Schwere der zugehörigen Abweichung zuordnet. Dann gilt

$$\kappa_w = 1 - \frac{\sum_{j,k} W_{jk} O_{jk}}{\sum_{j,k} W_{jk} E_{jk}}. \quad (2.11)$$

Die in dieser Arbeit verwendete quadratische Gewichtung setzt

$$W_{jk} = \frac{(j - k)^2}{(C - 1)^2}, \quad (2.12)$$

sodass $W_{jj} = 0$ für die Diagonale und $W_{jk} = 1$ für die maximal mögliche Abweichung gilt. Einsetzen von (2.12) in (2.11) ergibt das Quadratic Weighted Kappa

$$\text{QWK} = 1 - \frac{\sum_{j,k} (j - k)^2 O_{jk}}{\sum_{j,k} (j - k)^2 E_{jk}}, \quad (2.13)$$

in dem sich der Normierungsfaktor $(C - 1)^{-2}$ herauskürzt. Der Wertebereich ist $\text{QWK} \leq 1$; der Wert 1 entspricht perfekter Übereinstimmung, der Wert 0 einer Übereinstimmung auf Zufallsniveau, negative Werte einer systematisch schlechteren als zufälligen Übereinstimmung. Der quadratische Term bestraft weit entfernte Verwechslungen überproportional und bildet damit die Ordinalstruktur des Bewertungsschemas ab. QWK ist das im Bereich des automatisierten Scorings etablierte Standardmaß und wurde unter anderem als offizielle Metrik des ASAP-Wettbewerbs verwendet [57]. Abbildung 2.3 stellt die quadratische Gewichtung derjenigen 0/1-Gewichtung gegenüber, die der Accuracy zugrunde liegt.

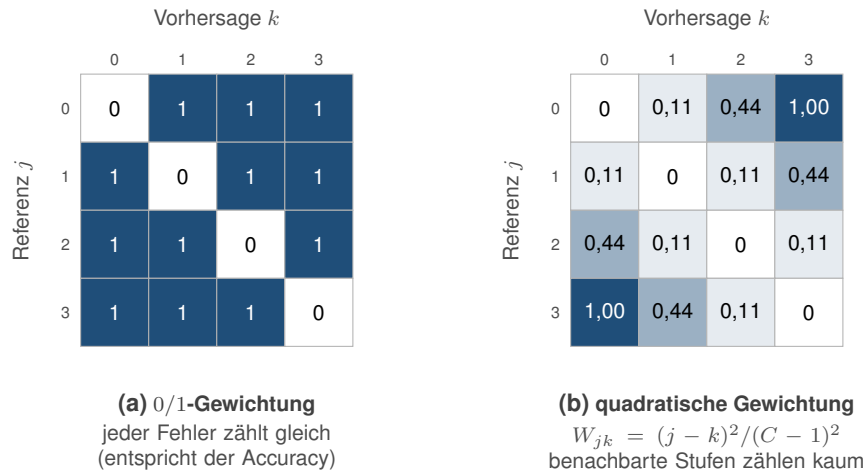


Abbildung 2.3: Gewichtsmatrizen W für einen vierstufigen Labelraum ($C = 4$). Der Eintrag W_{jk} gibt an, wie schwer die Verwechslung der Referenzstufe j mit der Vorhersage k gewertet wird. Die 0/1-Gewichtung in (a) behandelt jede Abweichung gleich und entspricht damit der Accuracy nach (2.7). Die quadratische Gewichtung in (b) nach (2.12) bestraft weit entfernte Verwechslungen überproportional: Eine Abweichung um eine Stufe geht mit dem Gewicht 0,11 ein, eine Abweichung um drei Stufen mit dem neunfachen Wert. Genau diese Abstufung setzt voraus, dass die Labelindizes eine Rangordnung mit interpretierbaren Abständen bilden.

Voraussetzungen und Grenzen von QWK

Die Verwendung von QWK ist an eine inhaltliche Voraussetzung gebunden, die in der ASAG-Literatur nicht immer explizit gemacht wird: Die quadratische Gewichtung (2.12) setzt voraus, dass die Labelindizes j und k auf einer geordneten Skala liegen und dass ihre numerische Differenz als sinnvolles Maß der Abweichungsschwere interpretiert werden kann. Faktisch unterstellt QWK damit ein intervallskaliertes Verhalten der Distanzen, obwohl Bewertungsskalen streng genommen nur ordinal skaliert sind. Bei Punkteskalen wie derjenigen von ASAP-SAS ist diese Idealisierung vertretbar, da die Stufen die Anzahl korrekt genannter inhaltlicher Elemente abbilden und die Abstände daher näherungsweise gleichbedeutend sind.

Problematisch wird die Voraussetzung dagegen bei nominalen Labelsätzen. Das ursprüngliche Fünf-Wege-Schema von SemEval-2013 Task 7 unterscheidet die Kategorien *correct*, *partially_correct_incomplete*, *contradictory*, *irrelevant* und *non_domain* [19]. Diese Kategorien bilden

keine Rangordnung: Eine *contradictory*-Antwort ist nicht in einem wohldefinierten Sinne „besser“ oder „schlechter“ als eine *irrelevant*-Antwort, und eine Verwechslung von *correct* mit *non_domain* ist nicht deshalb viermal so schwerwiegend wie eine Verwechslung von *correct* mit *partially_correct_incomplete*, weil die willkürliche Indizierung dies nahelegt. In diesem Fünf-Wege-Setting ist QWK folglich kein sinnvolles Maß, und die Ergebnisse hängen von der arbiträren Reihenfolge der Kategorien ab.

Die vorliegende Arbeit verwendet für Beetle und SciEntsBank nicht das nominale Fünf-Wege-Schema, sondern die Drei-Wege-Variante mit den Stufen *incorrect*, *partially correct* und *correct*, die eine echte Rangordnung des Leistungsgrades darstellt (vgl. Kapitel 3). Unter dieser Wahl ist die Ordinalitätsannahme von QWK erfüllt und die Metrik über alle drei Benchmarks hinweg vergleichbar interpretierbar. Diese Festlegung wird hier ausdrücklich als methodische Entscheidung ausgewiesen, weil sie die Vergleichbarkeit mit älteren Arbeiten einschränkt, die auf dem Fünf-Wege-Schema berichten.

Unabhängig vom Skalenniveau weist QWK weitere bekannte Schwächen auf. Doewes, Kurdhi und Saxena zeigen, dass der Wert erheblich von der Form der Randverteilungen und von der Länge der Skala abhängt, dass die aus der Kappa-Literatur bekannten Paradoxa auch im Kontext des automatisierten Scorings auftreten und dass QWK deshalb nicht als alleinige Kennzahl verwendet werden sollte [18]. Aus diesem Grund wird Macro- F_1 in dieser Arbeit als zweite Metrik herangezogen: QWK erfasst die ordinale Nähe der Fehlklassifikationen, Macro- F_1 die Leistung auf seltenen Stufen. Die Ergebnistabellen in Kapitel 5 berichten überwiegend QWK; sämtliche Vergleiche mit mehreren Zufallssaaten werden in Abschnitt 5.12 unter Macro- F_1 wiederholt, wo auch die Abweichungen zwischen beiden Maßen diskutiert werden.

Zuverlässigkeit gemessener Unterschiede

Die bisher eingeführten Maße beschreiben, *was* gemessen wird. Für die Auswertung in Kapitel 5 ist zusätzlich zu klären, *wie genau* eine solche Messung ist – also welcher Unterschied zwischen zwei Konfigurationen überhaupt als Ergebnis gelten darf.

Stochastische Anteile im Training

Das Training eines neuronalen Modells ist kein deterministisches Verfahren. Mehrere Komponenten beruhen auf Zufallsentscheidungen:

- die Initialisierung neu hinzugefügter Module – in dieser Arbeit die Fusions- und Scoring-Parameter (Abschnitt 2.7) sowie die LoRA-Matrizen,
- die Reihenfolge, in der die Trainingsinstanzen zu Batches zusammengestellt werden,
- die zufällige Deaktivierung von Einheiten durch Dropout,
- die Aufteilung in Trainings- und Validierungsmenge, sofern kein offizieller Entwicklungssplit vorliegt.

Diese Quellen lassen sich über eine *Zufallssaat* (*random seed*) steuern: Ein fest gewählter Startwert des Zufallszahlengenerators legt die Folge aller Zufallsentscheidungen fest und

macht einen Lauf im Prinzip wiederholbar. Vollständige Reproduzierbarkeit ist damit allerdings nicht garantiert. Nebenläufige Berechnungen auf Grafikprozessoren summieren Gleitkommawerte in nicht festgelegter Reihenfolge; da die Gleitkommaaddition nicht assoziativ ist, können bereits daraus abweichende Ergebnisse entstehen. Auch das parallele Laden der Daten durch mehrere Prozesse kann die Batch-Zusammenstellung beeinflussen.

Streuung zwischen Läufen

Die praktische Folge ist erheblich: Zwei Trainingsläufe mit identischer Konfiguration, aber unterschiedlicher Zufallssaat liefern nicht denselben Wert. Reimers und Gurevych zeigen für Sequenzmarkierungsaufgaben, dass allein die Wahl der Zufallssaat zu statistisch signifikanten Unterschieden führen kann, und plädieren dafür, Verteilungen über mehrere Läufe statt einzelner Werte zu berichten [46]. Dodge u. a. weisen denselben Effekt für das Finetuning vortrainierter Sprachmodelle nach und führen ihn zu vergleichbaren Anteilen auf die Gewichtsinitialisierung und die Datenreihenfolge zurück [17].

Daraus folgt eine Auswertungsregel, die in Kapitel 5 durchgehend angewandt wird. Ein einzelner Messwert ist keine Punktmessung, sondern eine Stichprobe aus einer Verteilung. Der Unterschied zwischen zwei Konfigurationen ist nur dann als Effekt interpretierbar, wenn er *größer* ist als die Streuung, die dieselbe Konfiguration bei bloßem Wechsel der Zufallssaat aufweist. Diese Streuung wird im Folgenden als *Rauschgrenze* bezeichnet; sie wird in Abschnitt 5.2 empirisch bestimmt, statt sie anzunehmen.

Bezeichnet $\{m_1, \dots, m_S\}$ die Messwerte einer Konfiguration über S Zufallssaaten, so wird als Kennwert der Streuung die Spannweite

$$\Delta = \max_s m_s - \min_s m_s \quad (2.14)$$

verwendet. Bei der hier realisierbaren Zahl von $S = 3$ Läufen je Konfiguration ist die Spannweite der robustere Kennwert als eine Standardabweichung, deren Schätzung aus drei Werten sehr unsicher wäre. Aus demselben Grund wird auf inferenzstatistische Tests verzichtet: Deren Voraussetzungen sind bei drei Beobachtungen je Zelle nicht erfüllt, und ein Vergleich der beobachteten Differenz mit der beobachteten Spannweite ist die ehrlichere Darstellung. Abbildung 2.4 veranschaulicht die daraus folgende Entscheidungsregel.

Ablation

Ein wiederkehrendes Auswertungsmuster dieser Arbeit ist die *Ablation*: Aus einer funktionierenden Konfiguration wird gezielt *eine* Komponente entfernt oder ersetzt, während alle übrigen Bestandteile unverändert bleiben. Ein beobachteter Leistungsunterschied ist dann dieser einen Komponente zurechenbar. Die Aussagekraft einer Ablation hängt unmittelbar an der zuvor beschriebenen Rauschgrenze: Sie belegt einen Beitrag der entfernten Komponente nur, wenn der Unterschied die Streuung zwischen Läufen übersteigt.

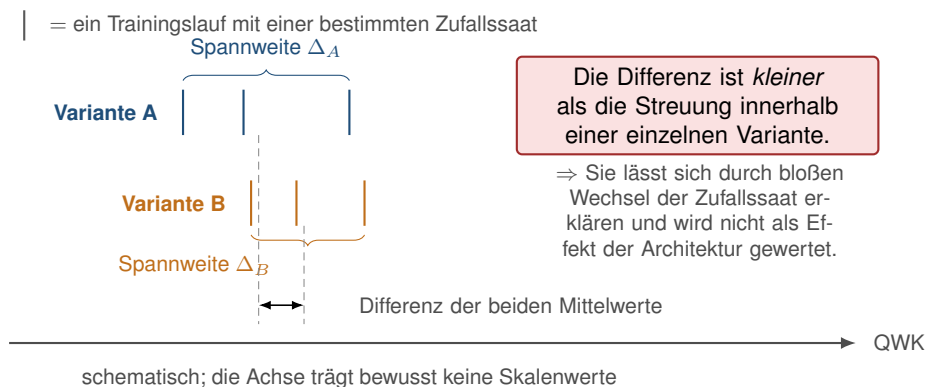


Abbildung 2.4: Auswertungsregel für den Vergleich zweier Konfigurationen. Jeder senkrechte Strich steht für einen Trainingslauf mit einer anderen Zufallssaat; die Klammern markieren die Spannweite (2.14) innerhalb einer Variante, die gestrichelten Linien deren Mittelwerte. Liegt die Differenz der Mittelwerte – wie hier dargestellt – unterhalb dieser Spannweite, so ist sie durch bloßen Wechsel der Zufallssaat erklärbar und wird in Kapitel 5 nicht als Effekt der Architektur interpretiert. Die Abbildung ist schematisch und enthält keine Messwerte; die empirisch bestimmten Spannweiten werden in Abschnitt 5.2 berichtet.

2.3 Encoder-basierte Ansätze im ASAG

Merkmalsbasierte Vorläufer

Die Anfänge des automatisierten Scorings reichen bis zu Page zurück, der Oberflächenmerkmale wie Textlänge und Wortschatzindikatoren mit linearen Modellen verknüpfte [44]. Für kurze, inhaltlich gebundene Antworten trat an die Stelle solcher Stilmerkmale die Messung inhaltlicher Übereinstimmung mit einer Referenzantwort. Mohler und Mihalcea operationalisierten dies über semantische Ähnlichkeitsmaße zwischen Schüler- und Musterantwort [41]; Sultan, Salazar und Sumner zeigten, dass eine kompakte Merkmalsmenge aus Wortalignment, Ausrichtungsgüte und Antwortlänge in Verbindung mit einem einfachen Klassifikator wettbewerbsfähige Ergebnisse liefert [55]. Weitere Arbeiten kombinierten lexikalische Überlappung mit Merkmalen der Antwortstruktur und trainierten Support-Vector-Machines oder lineare Regressionen [40, 39, 7]. Der gemeinsame Nachteil dieser Verfahren liegt in der Merkmalskonstruktion: Sie ist aufgabenspezifisch, aufwendig und erfasst Paraphrasen sowie inhaltlich korrekte, aber lexikalisch abweichende Antworten nur unzureichend.

Neuronale Encoder

Mit der Verfügbarkeit vortrainierter Sprachmodelle verlagerte sich das Feld auf neuronale Encoder. Riordan u. a. untersuchten systematisch rekurrente und konvolutionale Architekturen für das Short Answer Scoring und zeigten, dass gelernte Repräsentationen die manuelle Merkmalskonstruktion weitgehend ersetzen können [48]. Kumar u. a. kombinierten in AutoSAS mehrere Repräsentationsebenen zu einem produktiv einsetzbaren System [31].

Den heute dominierenden Ansatz begründete die Übertragung von BERT [16] und verwandten

Encoder-Modellen wie RoBERTa [37] auf die Aufgabe. Sung u. a. zeigten, dass ein domänenspezifisches Weitertrainieren von BERT auf fachlichen Ressourcen die Scoring-Güte deutlich steigert [56]; Camus und Filighera untersuchten den Transfer verschiedener Transformer-Encoder auf ASAG-Benchmarks [8], und Ormerod demonstrierten Gewinne durch Ensembles vortrainierter Modelle [43].

Architektonisch folgen diese Arbeiten einem einheitlichen Muster. Die textuellen Felder werden zu einer Eingabesequenz konkateniert, typischerweise in der Form

$$[\text{CLS}] \ q_i \ [\text{SEP}] \ s_i \ [\text{SEP}] \ a_i \ [\text{SEP}], \quad (2.15)$$

und durch den Encoder verarbeitet. Aus der resultierenden Zustandsmatrix wird eine einzelne Sequenzrepräsentation gewonnen – üblicherweise der Zustand an der $[\text{CLS}]$ -Position – und über eine lineare Ausgabeschicht auf C Logits abgebildet, aus denen mittels Softmax eine Verteilung über den globalen Labelraum $\mathcal{Y}$ entsteht. Diese Konstruktion entspricht exakt der klassischen Formulierung (2.4).

Ein verwandter, aber architektonisch abweichender Zweig verzichtet auf den festen Klassifikationskopf und bewertet über Ähnlichkeit. Bexte, Horbach und Zesch vergleichen die Schülerantwort mit Ankerantworten verschiedener Bewertungsstufen und leiten die Stufe aus der größten Einbettungsähnlichkeit ab, wobei Sentence-BERT-artige Bi-Encoder [47] zum Einsatz kommen [5]. Li u. a. modellieren die Beziehungen zwischen Antworten, Referenzantworten und Bewertungsstufen als relationales Netz [34]. Diese Verfahren nehmen bereits gedanklich vorweg, was in Abschnitt 2.5 als rubrik-geleitetes Scoring systematisiert wird: Die Bewertungsstufe wird nicht durch einen Index, sondern durch einen Text repräsentiert.

Grenzen

Die Encoder-Linie ist innerhalb eines Benchmarks stark und rechnerisch günstig, weist aber zwei strukturelle Beschränkungen auf. Erstens bindet der feste Klassifikationskopf das Modell an die Kardinalität und Semantik eines konkreten Labelraums; ein Wechsel des Benchmarks oder auch nur eine Aufgabe mit abweichender Punkteskala erfordert eine neue Ausgabeschicht und damit ein erneutes Training [24]. Zweitens erhält das Modell die Bewertungskriterien allenfalls implizit über die Trainingsdaten; die Rubrik als explizite sprachliche Beschreibung dessen, was eine Stufe ausmacht, geht nicht in die Vorhersage ein. Beide Beschränkungen motivieren die in Abschnitt 2.5 dargestellten Architekturen.

2.4 LLM-basierte Ansätze im ASAG

Generative Bewertung

Mit der Verfügbarkeit großer, instruktionsangepasster Sprachmodelle liegt es nahe, die Bewertung als Textgenerierung zu formulieren: Das Modell erhält Frage, Musterlösung, Antwort und gegebenenfalls die Rubriken als Prompt und erzeugt die Bewertungsstufe als Zeichenkette. Kortemeyer evaluiert GPT-4 in dieser Konfiguration auf physikalischen Kurzantworten

[30], Chamieh, Zesch und Gipp untersuchen Zero- und Few-Shot-Prompting systematisch [9], Chang, Taiga und Vilén übertragen den Ansatz auf das Finnische [10], und Ferreira Mello u. a. vergleichen prompt-optimiertes GPT-4 mit klassischen Modellen [22].

Der Ansatz ist maximal flexibel, da er keinen festen Labelraum voraussetzt, weist aber drei Nachteile auf. Erstens ist die autoregressive Dekodierung zur Inferenzzeit deutlich teurer als ein einzelner diskriminativer Vorwärtspass. Zweitens sind die Ergebnisse empfindlich gegenüber Formulierung und Format des Prompts, was die Reproduzierbarkeit beeinträchtigt. Drittens erzwingt die Generierung eine Nachverarbeitung, da das Modell nicht garantiert ein parsebares Label ausgibt.

Datenschutzrechtlicher Kontext

Eine zusätzliche, für den Bildungsbereich zentrale Restriktion betrifft die Verarbeitung von Schülertexten durch kommerzielle Modellanbieter. Schülerantworten sind personenbezogene Daten besonders schutzwürdiger Personengruppen; ihre Übermittlung an externe Programmierschnittstellen wirft datenschutzrechtliche und governancebezogene Fragen auf [21, 58]. Daraus ergibt sich die Anforderung, ASAG-Systeme lokal betreibbar zu halten. Lokal betreibbare generative Modelle in ausreichender Größe sind jedoch rechenintensiv, sodass ein Verfahren wünschenswert ist, das die Repräsentationsstärke großer Sprachmodelle nutzt, ohne zur Inferenzzeit generieren zu müssen.

LLMs als diskriminative Encoder

Genau diese Möglichkeit erschließt eine jüngere Forschungslinie, die autoregressive Sprachmodelle als Textencoder umwidmet. BehnamGhader u. a. zeigen mit LLM2Vec, dass Decoder-only-Modelle durch die Aufhebung der kausalen Maske, ein maskiertes Weitertrainieren und ein kontrastives Nachtraining zu leistungsstarken Encodern werden [3]. Lee u. a. entwickeln mit NV-Embed Trainingsverfahren, die Decoder-Modelle zu generalistischen Einbettungsmodellen machen [32]. Lin u. a. untersuchen die trainingsfreie Umwandlung und identifizieren dabei den Attention-Sink als zentrales Hindernis [35], während Qiao u. a. Decoder-only-Modelle als maskierte Autoencoder reinterpretieren [45]. Weitere Arbeiten belegen die Wirksamkeit des Ansatzes für Klassifikation [49, 36] und dichte Retrieval-Aufgaben [38]. Gekhman u. a. zeigen ergänzend, dass Sprachmodelle faktisches Wissen in ihren verborgenen Zuständen repräsentieren, selbst wenn sie es generativ nicht zuverlässig abrufen – ein Befund, der die encoderbasierte Nutzung für wissensintensive Diskriminationsaufgaben zusätzlich motiviert [23].

Für ASAG ist dieser Ansatz aus zwei Gründen attraktiv. Zum einen kann ein Sprachmodell die heterogenen Felder Frage, Musterlösung, Schülerantwort und Rubriken in einer einzigen langen Sequenz gemeinsam repräsentieren, statt sie paarweise zu vergleichen. Zum anderen bringt es das für die fachliche Beurteilung erforderliche Weltwissen mit. Zugleich besteht eine Forschungslücke: Die bisherigen Arbeiten zur Umwidmung von LLMs als Encoder evaluieren überwiegend auf Einzelsequenz- oder Satzpaaraufgaben, wie sie etwa in Retrieval- und Ähnlichkeitsbenchmarks vorherrschen [3, 32, 36]. Die gemeinsame Verarbeitung mehrerer heterogener Eingabefelder, wie sie ASAG erfordert, ist bislang kaum untersucht.

2.5 Rubrik-geleitetes Scoring: Guided-Alignment-Architekturen

Rubriken als Label-Semantik

Eine Rubrik ist im Kontext der Bildungsmessung eine sprachliche Beschreibung, die festlegt, welche Merkmale eine Antwort auf einer bestimmten Leistungsstufe aufweist. Aus modellierungstechnischer Sicht ist sie damit eine *semantische Repräsentation eines Labels*: Statt die Stufe k als bedeutungsloses Element eines Indexraums zu behandeln, wird sie durch den Text $r_{i,k}$ charakterisiert.

Dieser Gedanke ist nicht auf ASAG beschränkt. In der Verarbeitung natürlicher Sprache existiert eine breite Linie von Arbeiten, die Klassifikationsaufgaben durch Einbeziehung der Label-Semantik verbessern. Dazu zählen die Verbalisierung von Labels im Pattern-Exploiting Training [50, 26] sowie NLI-artige Formulierungen, in denen ein Label als Aussage ausgedrückt und gegen den Eingabetext geprüft wird [61, 20]. Solche Verfahren ersetzen die rein indexbasierte Klassifikation und sind insbesondere in Zero- und Few-Shot-Szenarien wirksam. Rubrik-geleitetes ASAG lässt sich als fachspezifische Ausprägung dieses Prinzips verstehen.

Rubric Retrieval und paarweises Matching

Die derzeit verbreitetste Umsetzung im ASAG behandelt die Aufgabe als Retrieval- oder Matching-Problem: Jede Instanz wird in K_i Paare $(a_i, r_{i,k})$ expandiert, jedes Paar wird einzeln kodiert, und ein Kopf entscheidet, ob die Antwort die im jeweiligen Rubriktext beschriebenen Kriterien erfüllt [52, 24]. Eine verwandte Variante nutzt Ankerantworten verschiedener Stufen als implizite Rubriken und bewertet über Ähnlichkeit [5, 34].

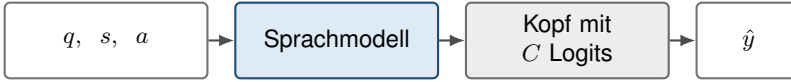
Der Vorteil dieser Formulierung ist ihre Flexibilität: Sie ist unabhängig von einem globalen Labelinventar und damit unmittelbar auf ungesehene Rubriksätze anwendbar. Der Nachteil ist ihr Aufwand. Bezeichnet c die Kosten eines Vorwärtspasses, so erfordert die Bewertung einer Instanz $K_i \cdot c$ statt c , sowohl im Training als auch zur Inferenzzeit. Bei großen Backbones und umfangreichen Datensätzen wird dies schnell prohibitiv. Rubric Retrieval tauscht damit Effizienz gegen Flexibilität ein. Abbildung 2.5 stellt dieses Verfahren der klassischen Klassifikation und dem im Folgenden entwickelten Span-Alignment gegenüber.

Span-Alignment in einem einzigen Vorwärtspass

Einen Ausweg aus diesem Zielkonflikt bietet die gemeinsame Kodierung aller Felder in einer einzigen Sequenz. Gombert u. a. kodieren Antwort und Rubriken zusammen und richten die Repräsentation des Antwort-Spans über ein bilineares Netz an den Repräsentationen der einzelnen Rubrik-Spans aus [24]. Damit entfallen sowohl der feste globale Labelindex als auch die paarweise Expansion.

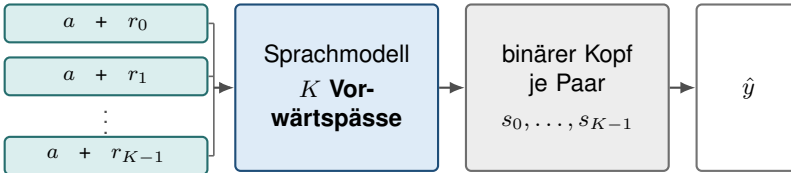
Das allgemeine Schema dieser Architekturfamilie lässt sich wie folgt fassen. Die Felder aus (2.2) werden zu einer einzigen Zeichenkette serialisiert, wobei die Feldgrenzen markiert

(a) Klassifikation mit festem Ausgabekopf



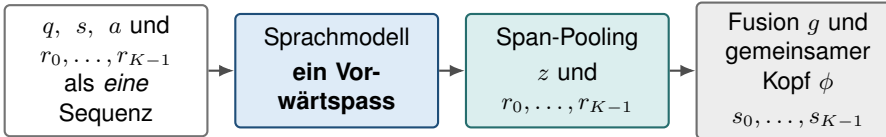
ein Vorwärtspass,
Labelraum global fixiert

(b) Paarweises Matching (Rubric Retrieval)



Aufwand wächst linear
mit der Zahl der Stufen

(c) Span-Alignment in einem einzigen Vorwärtspass (dieser Ansatz)



ein Vorwärtspass,
K zur Laufzeit variabel

Abbildung 2.5: Drei Umsetzungen der Bewertungsaufgabe im Vergleich. Variante (a) ignoriert die Rubriktexte und bildet auf einen architektonisch festgelegten Labelraum ab; sie ist effizient, aber auf einen ungesesehenen Rubriksatz nicht übertragbar. Variante (b) expandiert jede Instanz in K Antwort-Rubrik-Paare und ist damit rubriksatz-unabhängig, benötigt aber K statt eines Vorwärtspasses. Variante (c) kodiert alle Felder gemeinsam in einer Sequenz und gewinnt die Stufenrepräsentationen als Spans aus derselben Zustandsmatrix; sie verbindet die Flexibilität von (b) mit dem Aufwand von (a).

werden und die Zeichenpositionen der Rubrik-Spans mitgeführt werden. Nach der Kodierung durch das Sprachmodell liegt eine Zustandsmatrix

$$H_i \in \mathbb{R}^{T \times d} \quad (2.16)$$

vor, wobei T die Sequenzlänge und d die Modelldimension bezeichnet. Aus H_i werden zwei Arten von Repräsentationen extrahiert:

$$r_{i,k} = H_i[e_{i,k}] \quad \text{Rubrik-Repräsentation der Stufe } k, \quad (2.17)$$

$$z_i = H_i[T] \quad \text{Sequenzrepräsentation.} \quad (2.18)$$

Dabei bezeichnet $e_{i,k}$ die Position des letzten Tokens des k -ten Rubrik-Spans. Die Wahl des letzten Tokens ist nicht beliebig: Unter kausaler Attention aggregiert der Zustand an der Endposition eines Spans die Information des gesamten vorausgehenden Kontexts einschließlich dieses Spans (vgl. Abschnitt 2.6.2). Analog fasst z_i als Zustand des letzten Sequenztokens die gesamte Eingabe zusammen und ist damit das in der Literatur übliche Pooling für LLM-basierte Klassifikatoren [3, 49].

Anschließend wird jede Rubrik-Repräsentation mit einer Kontextrepräsentation über eine Fusionsfunktion g verknüpft,

$$v_{i,k} = g(z_i, r_{i,k}), \quad (2.19)$$

und ein gemeinsamer Scoring-Kopf ϕ projiziert das Ergebnis auf einen Skalar,

$$s_{i,k} = \phi(v_{i,k}) \in \mathbb{R}. \quad (2.20)$$

Die Vorhersage ergibt sich als Argmax über die Kandidatenstufen gemäß (2.5). Da ϕ auf einen Skalar und nicht auf C Logits abbildet, ist die Architektur von der Zahl der Rubrikstufen unabhängig und kann Instanzen mit variierendem K_i ohne Modifikation verarbeiten.

Die vorliegende Arbeit setzt an genau dieser Stelle an. Sie untersucht erstens, welche Kontextrepräsentation in (2.19) einzusetzen ist – die Repräsentation des Antwort-Spans, wie bei Gombert u. a., oder die Sequenzrepräsentation z_i –, zweitens, welche Fusionsfunktion g zu wählen ist (Abschnitt 2.8 und Kapitel 4), und drittens, wie Decoder-only-Modelle über die Steuerung der Attention-Maske für diese diskriminative Aufgabe angepasst werden können (Abschnitt 2.6.2).

2.6 Grundlagen der Transformer-Architektur

Sämtliche in dieser Arbeit verwendeten Modelle beruhen auf der Transformer-Architektur [59]. Dieser Abschnitt entwickelt die für das Verständnis der Methodik erforderlichen Komponenten.

Gesamtaufbau

Eine Eingabesequenz wird zunächst durch einen Tokenizer in eine Folge von T Token zerlegt und über eine Einbettungsmatrix $W_E \in \mathbb{R}^{|V| \times d}$ in einen kontinuierlichen Raum abgebildet, was die Ausgangsmatrix $H^{(0)} \in \mathbb{R}^{T \times d}$ liefert. Diese durchläuft L strukturgleiche Blöcke. Jeder Block ℓ besteht aus einer Multi-Head-Self-Attention-Teilschicht und einer positionsweise angewandten Feed-Forward-Teilschicht, die jeweils in eine Residualverbindung eingebettet und durch eine Normalisierungsschicht stabilisiert werden. In der heute üblichen Pre-Norm-Variante gilt

$$\tilde{H}^{(\ell)} = H^{(\ell-1)} + \text{MHA}(\text{Norm}(H^{(\ell-1)})), \quad (2.21)$$

$$H^{(\ell)} = \tilde{H}^{(\ell)} + \text{FFN}(\text{Norm}(\tilde{H}^{(\ell)})). \quad (2.22)$$

Als Normalisierung verwendet das ursprüngliche Modell Layer Normalization [1], während die in dieser Arbeit eingesetzten Llama- und Mistral-Modelle die rechnerisch günstigere Root-Mean-Square-Normalisierung nutzen [62, 25, 29]. Die Feed-Forward-Teilschicht besteht im ursprünglichen Modell aus zwei linearen Abbildungen mit dazwischenliegender Nichtlinearität; die genannten Modelle verwenden stattdessen eine gated linear unit in der SwiGLU-Variante [51]. Die Ausgabe $H^{(L)}$ der letzten Schicht bildet die in (2.16) verwendete Zustandsmatrix H_i . Abbildung 2.6 fasst Gesamtstapel und Blockaufbau zusammen.

Der zentrale architektonische Unterschied zwischen Encoder-only-Modellen wie BERT [16] und Decoder-only-Modellen wie Llama [25] oder Mistral [29] liegt nicht im Blockaufbau, sondern ausschließlich in der Attention-Maske und im Vortrainingsziel; dies wird in Abschnitt 2.6.2 ausgeführt.

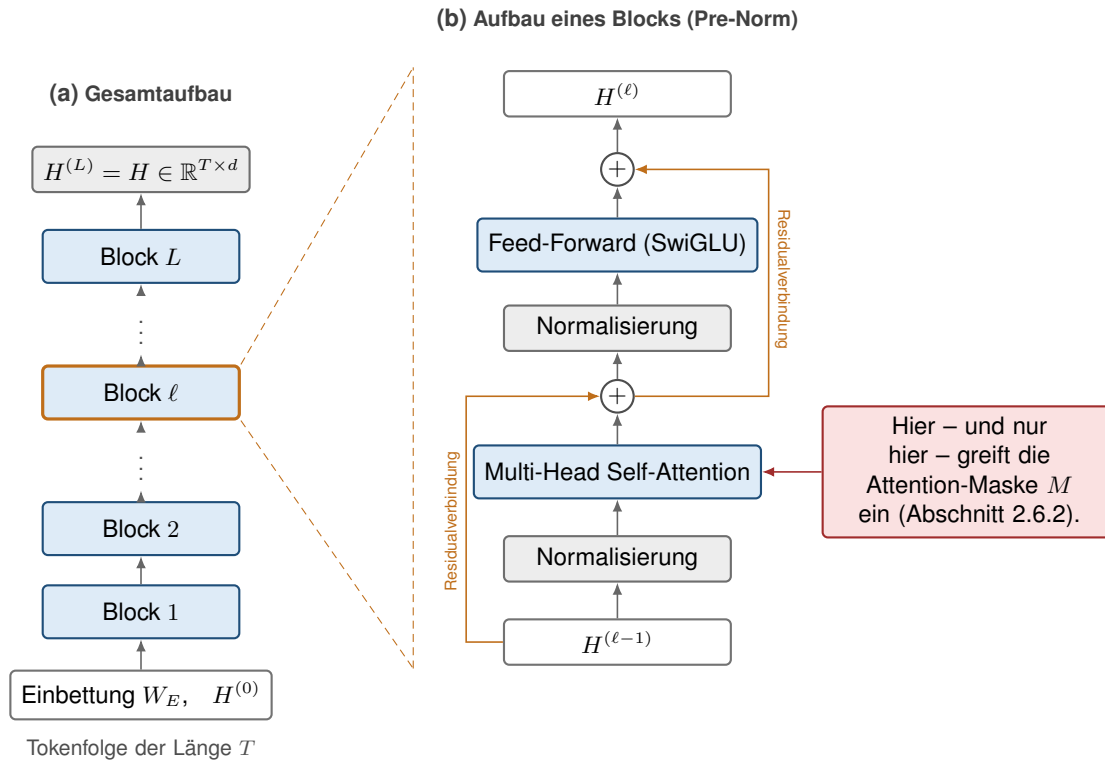


Abbildung 2.6: Aufbau der in dieser Arbeit verwendeten Modelle. Teil (a) zeigt den Gesamtstapel: Die eingebettete Tokenfolge durchläuft L strukturgleiche Blöcke; die Zustandsmatrix der letzten Schicht ist die in (2.16) verwendete Matrix H . Teil (b) zeigt einen einzelnen Block in der Pre-Norm-Variante nach (2.21) und (2.22): Normalisierung, Teilschicht und Residualverbindung wiederholen sich zweimal, einmal für die Self-Attention und einmal für die Feed-Forward-Teilschicht. Encoder- und Decoder-only-Modelle unterscheiden sich in diesem Aufbau nicht, sondern ausschließlich in der Attention-Maske M und im Vor-trainingsziel.

2.6.1 Self-Attention und Multi-Head Attention

Skalierte Punktprodukt-Attention

Self-Attention berechnet für jede Position eine gewichtete Summe über die Repräsentationen aller Positionen, wobei die Gewichte aus der Ähnlichkeit der Positionen zueinander hervorgehen. Ausgehend von $X \in \mathbb{R}^{T \times d}$ werden drei lineare Projektionen gebildet,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2.23)$$

mit $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ und $W_V \in \mathbb{R}^{d \times d_v}$. Die Zeilen von Q , K und V werden als Anfrage-, Schlüssel- und Wertvektoren bezeichnet. Die Attention-Ausgabe ist

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (2.24)$$

Die Matrix $S = QK^\top \in \mathbb{R}^{T \times T}$ enthält in Eintrag S_{ij} das Skalarprodukt zwischen der Anfrage an Position i und dem Schlüssel an Position j , also ein unnormiertes Maß der Relevanz von Position j für Position i . Der Softmax wird zeilenweise angewandt, sodass die Attention-Gewichte α_{ij} für jedes i eine Wahrscheinlichkeitsverteilung über j bilden:

$$\alpha_{ij} = \frac{\exp(S_{ij}/\sqrt{d_k})}{\sum_{j'=1}^T \exp(S_{ij'}/\sqrt{d_k})}, \quad \text{Attention}(Q, K, V)_i = \sum_{j=1}^T \alpha_{ij} V_j. \quad (2.25)$$

Abbildung 2.7 zeigt den vollständigen Berechnungsweg einschließlich des in Abschnitt 2.6.2 eingeführten Maskenterms.

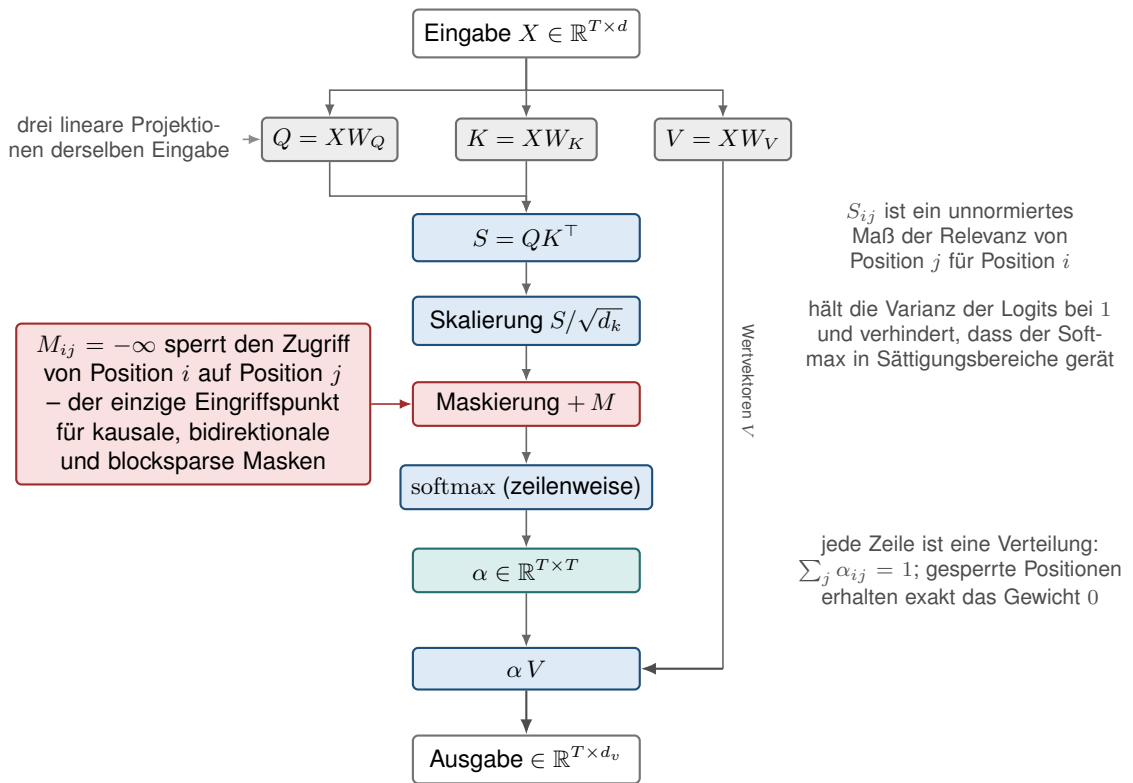


Abbildung 2.7: Berechnungsweg der maskierten skalierten Punktprodukt-Attention nach (2.31). Aus derselben Eingabe X entstehen über drei lineare Projektionen die Anfrage-, Schlüssel- und Wertmatrizen. Das Produkt $QK^\top$ liefert für jedes Positionspaar ein Relevanzmaß, das mit $1/\sqrt{d_k}$ skaliert und um den additiven Maskenterm M ergänzt wird. Erst nach dieser Addition wird der Softmax zeilenweise angewandt, sodass gesperrte Positionen exakt das Gewicht null erhalten. Die Maske ist damit der einzige Eingriffspunkt, über den sich das Sichtbarkeitsverhalten des Modells verändern lässt, ohne Gewichte oder Positionskodierung anzutasten.

Die Skalierung mit $1/\sqrt{d_k}$ ist keine Konvention, sondern numerisch begründet. Sind die Komponenten von Q_i und K_j unabhängig mit Erwartungswert 0 und Varianz 1, so gilt für das Skalarprodukt

$$\mathbb{E}[Q_i^\top K_j] = 0, \quad \text{Var}[Q_i^\top K_j] = d_k. \quad (2.26)$$

Ohne Skalierung wüchse die Streuung der Logits also mit d_k , wodurch der Softmax in Sättigungsbereiche mit verschwindenden Gradienten geriete. Die Division durch $\sqrt{d_k}$ normiert die Varianz auf 1 und hält die Verteilung in einem trainierbaren Bereich [59].

Multi-Head Attention

Eine einzelne Attention-Operation mittelt über alle Relevanzbeziehungen gleichzeitig. Multi-Head Attention führt daher h Attention-Operationen mit eigenen Projektionen parallel aus und konkateniert deren Ausgaben:

$$\text{head}_m = \text{Attention}(XW_Q^{(m)}, XW_K^{(m)}, XW_V^{(m)}), \quad m = 1, \dots, h, \quad (2.27)$$

$$\text{MHA}(X) = [\text{head}_1 \parallel \dots \parallel \text{head}_h] W_O, \quad (2.28)$$

mit $W_O \in \mathbb{R}^{hd_v \times d}$ und $\parallel$ als Konkatenation entlang der Merkmalsdimension. Üblicherweise wird $d_k = d_v = d/h$ gewählt, sodass der Rechenaufwand demjenigen einer einzelnen Attention mit voller Dimension entspricht. Die einzelnen Köpfe können auf diese Weise unterschiedliche Beziehungstypen erfassen. Die in dieser Arbeit verwendeten Modelle nutzen Varianten wie Grouped-Query Attention, bei denen sich mehrere Anfrageköpfe einen Satz von Schlüssel- und Wertköpfen teilen, um den Speicherbedarf des Key-Value-Caches zu senken [29, 25].

Der Rechen- und Speicheraufwand der Self-Attention wächst mit $\mathcal{O}(T^2d)$, da die Matrix S quadratisch in der Sequenzlänge ist. Dies ist für die vorliegende Arbeit unmittelbar relevant, weil die gemeinsame Serialisierung von Frage, Musterlösung, Antwort und sämtlichen Rubriken zu deutlich längeren Sequenzen führt als die paarweise Kodierung – ein Aufwand, der jedoch dadurch mehr als aufgewogen wird, dass nur ein statt K_i Vorwärtspässe erforderlich sind (vgl. Abschnitt 2.5).

Kodierung von Positionsinformation

Die Operation (2.24) ist permutationsäquivalent: Eine Umordnung der Eingabepositionen führt zu einer entsprechend umgeordneten Ausgabe, ohne dass die Reihenfolge selbst in die Berechnung eingeht. Positionsinformation muss daher explizit eingebracht werden. Das ursprüngliche Modell addiert sinusförmige Positionskodierungen auf die Einbettungen [59], BERT verwendet gelernte absolute Positionseinbettungen [16].

Die in dieser Arbeit eingesetzten Decoder-Modelle nutzen dagegen Rotary Position Embedding (RoPE) [54]. RoPE kodiert die Position nicht additiv, sondern multiplikativ, indem die Anfrage- und Schlüsselvektoren positionsabhängig rotiert werden. Der Vektorraum wird dazu in $d_k/2$ zweidimensionale Unterräume zerlegt; für den m -ten Unterraum mit Frequenz $\theta_m = b^{-2m/d_k}$ und Basis b wirkt an Position p die Rotationsmatrix

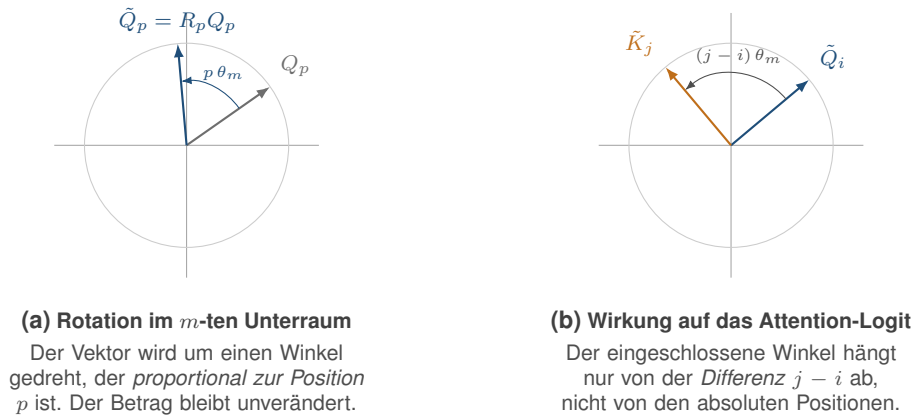
$$R_{p,m} = \begin{pmatrix} \cos(p\theta_m) & -\sin(p\theta_m) \\ \sin(p\theta_m) & \cos(p\theta_m) \end{pmatrix}. \quad (2.29)$$

Bezeichnet R_p die blockdiagonale Zusammensetzung aller $R_{p,m}$, so werden $\tilde{Q}_p = R_p Q_p$ und $\tilde{K}_p = R_p K_p$ gebildet. Für das resultierende Attention-Logit gilt aufgrund der Orthogonalität der

Rotationsmatrizen

$$\tilde{Q}_i^\top \tilde{K}_j = (R_i Q_i)^\top (R_j K_j) = Q_i^\top R_i^\top R_j K_j = Q_i^\top R_{j-i} K_j, \quad (2.30)$$

das heißt das Logit hängt nur von der *relativen* Position $j - i$ ab. Diese Eigenschaft ist für die vorliegende Arbeit in zweifacher Hinsicht bedeutsam. Erstens bleibt RoPE bei einer Änderung der Attention-Maske unverändert anwendbar, sodass die in Abschnitt 2.6.2 beschriebene Aufhebung der Kausalität keine Anpassung der Positionskodierung erfordert. Zweitens eröffnet die Abhängigkeit von relativen Positionen die Möglichkeit, Positionsindizes gezielt umzunummerieren, um Blöcke der Eingabe positional voneinander zu entkoppeln. Abbildung 2.8 veranschaulicht beide Aussagen.



$$\tilde{Q}_i^\top \tilde{K}_j = Q_i^\top R_i^\top R_j K_j = Q_i^\top R_{j-i} K_j$$

Abbildung 2.8: Wirkungsweise von Rotary Position Embedding [54]. Der Anfrage- beziehungsweise Schlüsselvektor wird in $d_k/2$ zweidimensionale Unterräume zerlegt; in jedem Unterraum wird er um einen positionsabhängigen Winkel gedreht (a). Da Rotationsmatrizen orthogonal sind, hebt sich der absolute Anteil im Skalarprodukt auf, und das Attention-Logit hängt nur noch von der relativen Position $j - i$ ab (b), wie in (2.30) hergeleitet. Für diese Arbeit folgt daraus zweierlei: Eine Änderung der Attention-Maske erfordert keine Anpassung der Positionskodierung, und Positionsindizes lassen sich gezielt umnummerieren, um Eingabeböcke positional zu entkoppeln.

Die Wahl der Basis b unterscheidet sich zwischen den in dieser Arbeit verwendeten Backbones erheblich. Mistral-7B-v0.1 verwendet den ursprünglich vorgeschlagenen Wert $b = 10\,000$ [29]. Die Llama-3.2-Modelle setzen dagegen $b = 500\,000$ und kombinieren dies mit einer frequenzabhängigen Streckung der Wellenlängen, um Kontextlängen weit über die im Vortraining gesehenen hinaus zu unterstützen [25]: Niedrigfrequente Komponenten werden um den Faktor 32 gestreckt, hochfrequente unverändert belassen, dazwischen wird interpoliert. Die nominelle Kontextlänge beträgt dadurch 131 072 Token gegenüber einer ursprünglichen Vortrainingslänge von 8 192. Für die vorliegende Arbeit ist dies insofern relevant, als die verwendete maximale Sequenzlänge von 2 048 Token (Abschnitt 4.1.1) deutlich unterhalb beider Grenzen liegt; die Streckung wirkt sich in diesem Bereich also nicht aus.

2.6.2 Kausale vs. bidirektionale Attention-Masken

Maskierung als additiver Term

Die Steuerung, welche Positionen einander sehen dürfen, erfolgt über eine additive Maskenmatrix $M \in (\{0\} \cup \{-\infty\})^{T \times T}$, die vor dem Softmax auf die skalierten Logits addiert wird:

$$\text{Attention}(Q, K, V; M) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V. \quad (2.31)$$

Ein Eintrag $M_{ij} = -\infty$ bewirkt $\exp(\cdot) = 0$ und damit $\alpha_{ij} = 0$: Position i kann Position j nicht beachten. In der Implementierung wird $-\infty$ durch den kleinsten darstellbaren Wert des verwendeten Gleitkommaformats ersetzt.

Zwei Maskentypen sind zu unterscheiden. Die *bidirektionale* Maske schränkt lediglich um Padding-Positionen ein,

$$M_{ij}^{\text{bidir}} = \begin{cases} 0, & \text{falls Position } j \text{ ein gültiges Token ist,} \\ -\infty, & \text{sonst,} \end{cases} \quad (2.32)$$

sodass jede Position jede andere sieht. Dies ist die Maske von Encoder-only-Modellen wie BERT und RoBERTa, deren Vortraining über Masked Language Modeling folgerichtig einen beidseitigen Kontext voraussetzt [16, 37]. Die *kausale* Maske ergänzt die Padding-Bedingung um eine strikte Zukunftssperre,

$$M_{ij}^{\text{kausal}} = \begin{cases} 0, & \text{falls } j \leq i \text{ und } j \text{ gültig ist,} \\ -\infty, & \text{sonst,} \end{cases} \quad (2.33)$$

sodass die Attention-Gewichtsmatrix untere Dreiecksgestalt annimmt. Diese Maske ist konstitutiv für Decoder-only-Modelle: Sie stellt sicher, dass die Vorhersage des Tokens an Position $i + 1$ ausschließlich auf den Positionen $1, \dots, i$ beruht, und macht damit das Training auf allen Positionen einer Sequenz parallel möglich. Abbildung 2.9 stellt beide Maskentypen sowie die weiter unten eingeführte blocksparse Variante gegenüber.

Konsequenzen für diskriminative Nutzung

Aus (2.33) folgt unmittelbar die für diese Arbeit zentrale Asymmetrie: In einem Decoder-only-Modell ist der Zustand $H[i]$ eine Funktion ausschließlich des Präfixes $1, \dots, i$. Zwei Konsequenzen ergeben sich daraus.

Erstens ist die Wahl des Poolings festgelegt. Da nur die letzte Position die gesamte Sequenz gesehen hat, ist das Last-Token-Pooling $z = H[T]$ die naheliegende Sequenzrepräsentation, während in Encoder-Modellen die [CLS]-Position oder ein Mittelwert über alle Positionen verwendet wird. Ebenso folgt daraus die Wahl des *letzten* Tokens eines Rubrik-Spans als Pooling-Anker in (2.17): Nur an dieser Position ist der gesamte Span in die Repräsentation eingegangen.

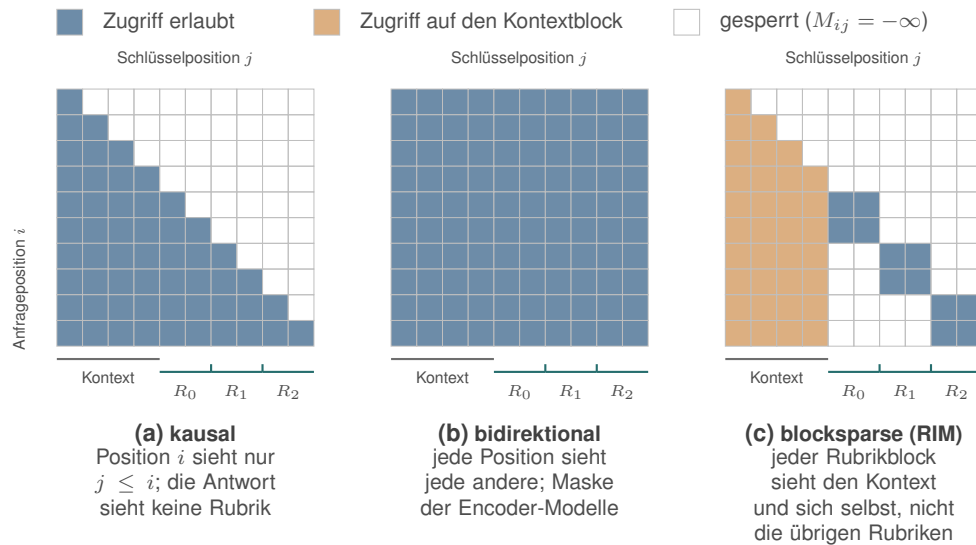


Abbildung 2.9: Drei Maskentypen für eine Beispielsequenz aus einem Kontextblock (Frage, Musterlösung, Antwort) und drei Rubrikblöcken. Gefüllte Zellen bezeichnen zugelassene, leere Zellen gesperrte Zugriffe. Die kausale Maske **(a)** nach (2.33) erzeugt die untere Dreiecksgestalt: Die Antwort-Tokens können keine Rubrik berücksichtigen, und die Rubrik-Repräsentationen sind untereinander asymmetrisch konditioniert, da r_k die vorangehenden Stufen sieht, die nachfolgenden aber nicht. Die bidirektionale Maske **(b)** nach (2.32) hebt beide Asymmetrien auf. Die blocksparse Maske **(c)** nach (2.35) hebt nur die zweite auf: Jede Stufe wird unabhängig von den übrigen, aber unter vollständiger Kenntnis des Aufgabenkontexts kodiert.

Zweitens ist die kausale Maske für eine reine Verstehensaufgabe eine unnötige Einschränkung. Für die Bewertung einer Schülerantwort ist die Kausalitätsbedingung nicht sachlich motiviert; das Modell soll die Eingabe verstehen, nicht fortsetzen. Tokens der Antwort können unter kausaler Maske die nachfolgenden Rubriktexte nicht berücksichtigen, obwohl gerade diese Information für die Beurteilung relevant wäre. Genau diese Beobachtung motiviert die Arbeiten zur Umwidmung von LLMs als Encoder [3, 35, 45].

Aufhebung der Kausalität

Der einfachste Eingriff besteht darin, M^{kausal} durch M^{bidir} zu ersetzen und damit die Zukunftssperre aufzuheben. Da die Maske ein rein additiver Term in (2.31) ist, erfordert dieser Eingriff keine Änderung der Gewichte, der Projektionen oder der Positionskodierung: Die skalierte Punktprodukt-Attention und RoPE bleiben unverändert, lediglich die Menge der zugelassenen Schlüsselpositionen wird erweitert.

Der Eingriff ist allerdings nicht folgenlos. Das Modell wurde ausschließlich unter kausaler Maske vortrainiert; die abrupte Erweiterung des sichtbaren Kontexts stellt eine Verschiebung gegenüber der Vortrainingsverteilung dar. BehnamGhader u. a. begegnen dem mit einem zusätzlichen maskierten Weitertrainieren, das dem Modell die Nutzung des beidseitigen Kontexts erst beibringt [3]. Lin u. a. untersuchen den trainingsfreien Fall und identifizieren dabei den

Attention-Sink als Kernproblem: Autoregressive Modelle konzentrieren einen erheblichen Teil ihrer Attention-Masse auf die ersten Positionen der Sequenz, die als Ablage für überschüssige Aufmerksamkeit dienen [60]. Wird die Maske aufgehoben, verändert sich die Normierung des Softmax in (2.25) für alle Positionen, was diesen Mechanismus stört [35].

Vor diesem Hintergrund ist eine *anteilige* Aufhebung der Kausalität naheliegend, bei der nur die obersten n_{bidir} der L Schichten bidirektional rechnen, während die unteren Schichten kausal bleiben. Mit Schichtindizes $\ell = 1, \dots, L$ gilt

$$M^{(\ell)} = \begin{cases} M^{\text{bidir}}, & \text{falls } \ell > L - n_{\text{bidir}}, \\ M^{\text{kausal}}, & \text{sonst,} \end{cases} \quad n_{\text{bidir}} \in \{0, \dots, L\}. \quad (2.34)$$

Die Zahl n_{bidir} interpoliert zwischen dem unveränderten Decoder ($n_{\text{bidir}} = 0$) und einem vollständig bidirektionalen Modell ($n_{\text{bidir}} = L$). Zweckmäßig wird sie über einen Anteil ρ der Schichtzahl angegeben; die in dieser Arbeit verwendete Umrechnung, die abrundet und Werte ab 1 als absolute Schichtzahl liest, ist in Abschnitt 4.3.2 angegeben. Die Intuition hinter dieser Konstruktion ist, dass untere Schichten überwiegend lokale, lexikalische und syntaktische Merkmale extrahieren und damit näher am Vortrainingsverhalten bleiben sollen, während obere Schichten aufgabenspezifische Aggregation leisten und dort von einem vollständigen Kontext profitieren können. Die systematische Untersuchung von ρ ist ein Kernbeitrag dieser Arbeit; die konkrete Umsetzung wird in Kapitel 4 beschrieben, die Ergebnisse in Kapitel 5 berichtet. Abbildung 2.10 zeigt die resultierende Schichtaufteilung für die untersuchten Anteile.

Blocksparse Masken

Die Aufhebung der Kausalität ist nicht die einzige sinnvolle Modifikation von M . Da die Maske jede Sichtbarkeitsbeziehung einzeln kontrolliert, lässt sie sich auch *blockweise* strukturieren. Zerfällt die Eingabesequenz in einen Kontextblock C (Frage, Musterlösung, Antwort) und K Rubrikblöcke $R_0, \dots, R_{K-1}$, so ist etwa die Vorschrift

$$M_{ij}^{\text{block}} = \begin{cases} 0, & \text{falls } j \in C \text{ oder } i, j \text{ im selben Rubrikblock liegen,} \\ -\infty, & \text{falls } i \text{ und } j \text{ in verschiedenen Rubrikblöcken liegen,} \end{cases} \quad (2.35)$$

realisierbar, ergänzt um die übliche Kausalitätsbedingung innerhalb des Kontextblocks. Unter (2.35) sieht jeder Rubrikblock nur den Aufgabenkontext und sich selbst, nicht aber die übrigen Rubriken.

Die Motivation dieser Konstruktion knüpft unmittelbar an die zuvor beschriebene Asymmetrie an. Unter kausaler Maske ist die Repräsentation r_k von den vorangehenden Rubriken $r_0, \dots, r_{k-1}$ mitbestimmt (Abschnitt 4.3.1); die Stufen werden also nicht unabhängig voneinander kodiert, sondern in der zufälligen Reihenfolge ihrer Serialisierung. Für die Anpassung an einen festen Rubriksatz kann diese Kopplung nützlich sein, weil das Modell die Stufen gegeneinander abgrenzen lernt. Bei Übertragung auf einen Benchmark mit anderer Stufenzahl oder anderer Rubrikformulierung wird sie dagegen zur Fehlerquelle, da die gelernten Abhängigkeiten dort nicht mehr zutreffen. Eine blockspare Maske entkoppelt die Stufen und stellt so die Rubriksatz-Invarianz her, die paarweise Verfahren strukturell besitzen, ohne deren K -fachen Rechenaufwand zu übernehmen [24].

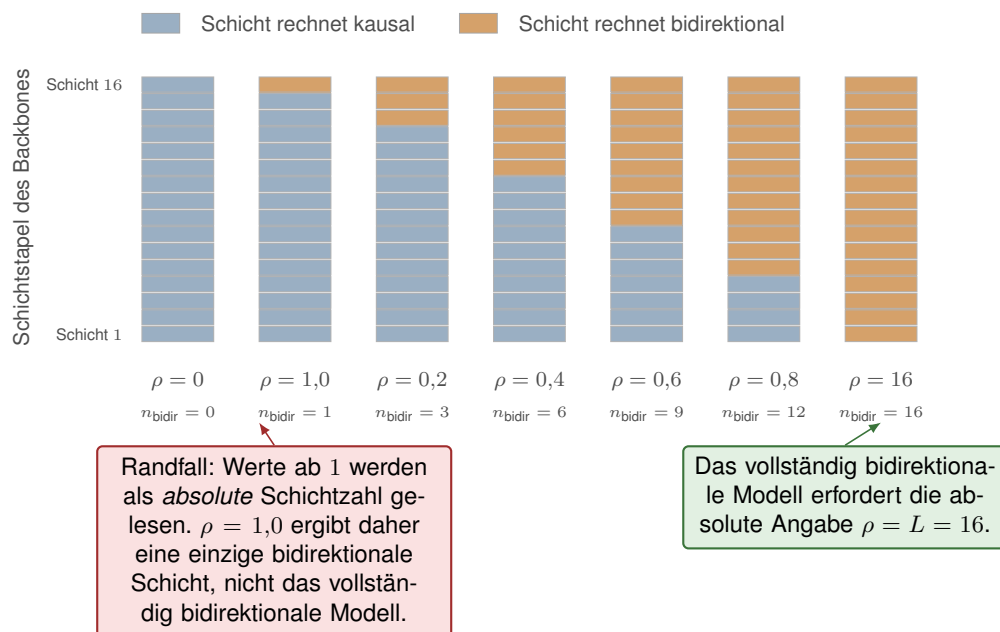


Abbildung 2.10: Anteilige Aufhebung der Kausalität am Beispiel von Llama-3.2-1B ($L = 16$ Schichten). Die bidirektional rechnenden Schichten bilden nach (4.13) stets die oberste zusammenhängende Gruppe; die unteren Schichten bleiben kausal und damit näher am Vortrainingsverhalten. Die Umrechnung des Anteils ρ in eine Schichtzahl folgt (4.15). Zu beachten ist der Randfall am linken Rand: Da Werte ab 1 als absolute Schichtzahl interpretiert werden, bezeichnet die Konfiguration $\rho = 1,0$ genau eine bidirektionale Schicht. Sie liegt damit in der Rangfolge der tatsächlichen Bidirektionalität zwischen dem rein kausalen Modell und $\rho = 0,2$ – ein Umstand, der bei der Interpretation der Ergebnisse in Kapitel 5 berücksichtigt wird.

Für die vorliegende Arbeit ist dieser Mechanismus aus zwei Gründen von Belang, obwohl er hier nicht experimentell untersucht wird. Erstens gehört er derselben Klasse von Eingriffen an wie die in Abschnitt 4.3.2 entwickelte anteilige Aufhebung: In beiden Fällen wird ausschließlich der additive Maskenterm verändert, während Gewichte und Positionskodierung unangetastet bleiben. Zweitens adressiert er dasselbe Spannungsverhältnis zwischen Anpassung und Übertragbarkeit, das sich in den Ergebnissen dieser Arbeit empirisch zeigt (Kapitel 5). Er wird daher in Kapitel 7 als naheliegende Fortsetzung aufgegriffen.

2.7 Parameter-effizientes Finetuning (LoRA)

Motivation

Ein vollständiges Finetuning aktualisiert sämtliche Parameter des Backbones. Für ein Modell mit $7 \cdot 10^9$ Parametern erfordert dies neben den Modellgewichten selbst auch Gradienten und Optimiererzustände; bei Adam mit zwei Momenten in 32-Bit-Genauigkeit entstehen so mehrere hundert Gigabyte Speicherbedarf. Hinzu kommt, dass für jede Kombination aus Backbone, Datensatz und Architekturvariante eine vollständige Kopie der Gewichte gespeichert werden

müsste – bei der in Kapitel 5 durchgeführten Zahl von Konfigurationen ist dies weder speicher- noch rechenökonomisch vertretbar.

Parameter-effiziente Finetuning-Verfahren begegnen dem, indem sie den Großteil der Parameter einfrieren und nur eine kleine Zahl zusätzlicher oder ausgewählter Parameter trainieren. Zu den etablierten Vertretern zählen Adapter-Module, die zusätzliche Engpassschichten in jeden Transformer-Block einfügen [27], sowie Prefix-Tuning, das lernbare Präfixvektoren in den Schlüssel- und Wertraum jeder Attention-Schicht einspeist [33].

Low-Rank Adaptation

LoRA [28] beruht auf der Hypothese, dass die beim Finetuning erforderliche Gewichtsänderung eine niedrige intrinsische Dimension besitzt. Sei $W_0 \in \mathbb{R}^{d \times k}$ eine eingefrorene vortrainierte Gewichtsmatrix. Statt eine unrestringierte Änderung $\Delta W \in \mathbb{R}^{d \times k}$ zu lernen, wird diese als Produkt zweier Matrizen niedrigen Rangs parametrisiert,

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k). \quad (2.36)$$

Der Vorwärtspass der adaptierten Schicht lautet damit

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x, \quad (2.37)$$

wobei α ein Skalierungsfaktor ist. Die Division durch r entkoppelt die effektive Lernrate vom gewählten Rang: Ohne sie müsste die Lernrate bei jeder Änderung von r nachjustiert werden [28]. Abbildung 2.11 zeigt den Aufbau einer so adaptierten Schicht.

Zur Initialisierung wird A zufällig, etwa aus einer Normalverteilung, und B mit Nullen belegt. Dadurch gilt zu Trainingsbeginn $\Delta W = BA = 0$, sodass das adaptierte Modell exakt dem vortrainierten Modell entspricht und das Training von einem definierten Ausgangspunkt startet.

Die Zahl der trainierbaren Parameter je adaptierter Matrix beträgt

$$|\theta_{\text{LoRA}}| = r(d + k) \quad (2.38)$$

gegenüber $d \cdot k$ beim vollständigen Finetuning. Für eine quadratische Projektion mit $d = k = 4096$ und $r = 64$ ergibt dies $64 \cdot 8192 \approx 5,2 \cdot 10^5$ statt $1,7 \cdot 10^7$ Parametern, also etwa drei Prozent. Da W_0 eingefroren bleibt, entfallen zudem Gradienten und Optimiererzustände für den weitaus größten Teil des Modells.

Ein weiterer praktischer Vorteil betrifft die Inferenz: Nach dem Training kann $W' = W_0 + \frac{\alpha}{r} BA$ einmalig berechnet und anstelle von W_0 verwendet werden. Anders als bei Adapter-Modulen entsteht dadurch keinerlei zusätzliche Latenz zur Inferenzzeit [28]. Umgekehrt lassen sich mehrere aufgabenspezifische LoRA-Gewichtssätze auf demselben Backbone vorhalten und zur Laufzeit austauschen, was für die vergleichende Evaluation zahlreicher Konfigurationen unmittelbar von Vorteil ist.

Welche Matrizen adaptiert werden, ist eine Entwurfsentscheidung. Die ursprüngliche Arbeit beschränkt sich auf die Attention-Projektionen; verbreitet ist inzwischen die Anwendung auf

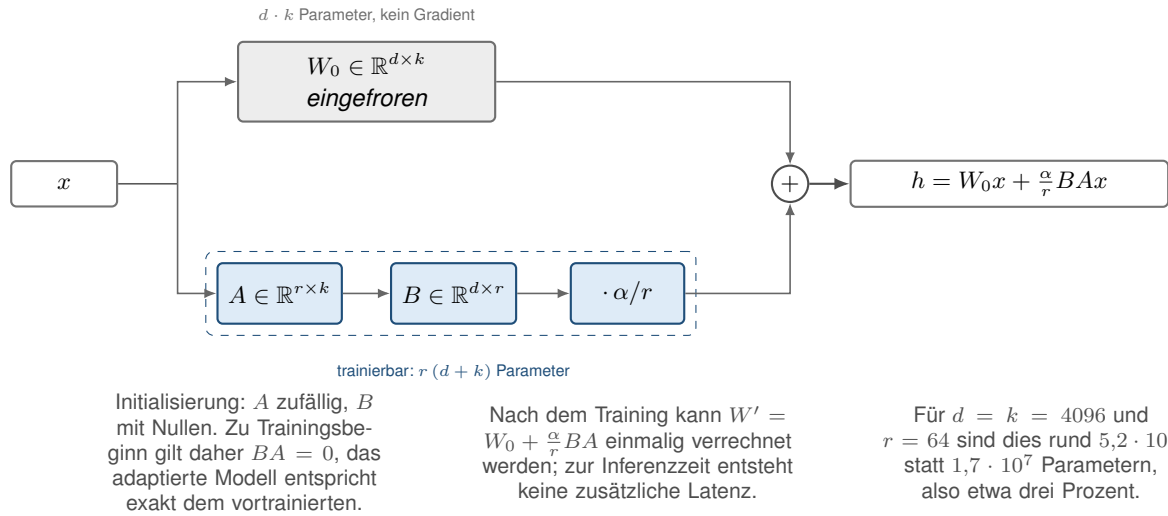


Abbildung 2.11: Aufbau einer mit LoRA [28] adaptierten linearen Schicht nach (2.37). Die vortrainierte Gewichtsmatrix W_0 bleibt eingefroren; parallel dazu wird eine Änderung niedrigen Rangs als Produkt zweier schmaler Matrizen gelernt und mit dem Faktor α/r skaliert. Beide Pfade werden addiert. Da nur A , B und die Skalierung trainiert werden, entfallen Gradienten und Optimiererzustände für den weitaus größten Teil des Modells. In dieser Arbeit wird LoRA ausschließlich auf das Sprachmodell angewandt; die Fusions- und Scoring-Module liegen architektonisch außerhalb und werden vollständig trainiert (Abschnitt 4.6).

sämtliche linearen Module einschließlich der Feed-Forward-Schichten. Eine speichereffizientere Variante kombiniert LoRA mit einer quantisierten Ablage des eingefrorenen Backbones [15]. Die in dieser Arbeit gewählte Konfiguration wird in Abschnitt 4.6 dokumentiert.

Zu beachten ist, dass die in Kapitel 4 eingeführten Fusions- und Scoring-Komponenten – die Projektionen der Fusionsfunktion sowie der Scoring-Kopf ϕ – nicht Teil des vortrainierten Backbones sind. Sie werden zufällig initialisiert und vollständig trainiert, da die Low-Rank-Adaption ausschließlich auf das Sprachmodell angewandt wird und diese Module architektonisch außerhalb davon liegen. Ihre Gewichte werden folgerichtig getrennt von den Adaptergewichten gespeichert. Die Zahl der tatsächlich aktualisierten Parameter setzt sich damit aus zwei Anteilen zusammen: den Adaptergewichten nach (2.38) und den vollständig trainierten Fusions- und Scoring-Parametern, deren Umfang je nach Fusionsvariante erheblich schwankt (Abschnitt 4.2).

2.8 Verwandte Arbeiten zu Fusionsmechanismen bei Rubrik-Antwort-Alignment

Die in Abschnitt 2.5 eingeführte Fusionsfunktion g aus (2.19) ist der Punkt, an dem sich die rubrik-geleiteten Architekturen im ASAG voneinander unterscheiden. Der vorliegende Abschnitt ordnet die in dieser Arbeit verglichenen Mechanismen in diese Linie ein.

Ähnlichkeitsbasierte Verfahren

Die einfachste Form der Verknüpfung verzichtet auf gelernte Parameter und bewertet über ein Ähnlichkeitsmaß. Bexte, Horbach und Zesch kodieren Schülerantwort und Ankerantworten der einzelnen Bewertungsstufen getrennt und vergeben die Stufe mit der höchsten Kosinusähnlichkeit [5]. Die Fusion ist hier also eine feste Funktion $g(u, v) = \cos(u, v)$ ohne trainierbare Komponente. Der Vorteil liegt in der unmittelbaren Übertragbarkeit auf neue Stufenmengen, der Nachteil darin, dass die Passung zwischen Antwort und Kriterium auf eine einzige skalare Größe reduziert wird und keine merkmalsweise Gewichtung möglich ist.

Paarweise Klassifikation

Ein zweiter Ansatz expandiert jede Instanz in Antwort-Rubrik-Paare und lässt ein Modell für jedes Paar entscheiden, ob die Antwort das jeweilige Kriterium erfüllt [52]. Die Fusion findet hier nicht auf der Ebene extrahierter Vektoren statt, sondern implizit innerhalb des Encoders, der beide Texte gemeinsam verarbeitet. Das erlaubt beliebig komplexe Interaktionen, erkauft dies aber mit K Vorwärtspässen je Instanz (Abschnitt 2.5) und verhindert, dass das Modell die Kriterien einer Aufgabe in Beziehung zueinander sieht.

Bilineares Alignment

Gombert u. a. kodieren Antwort und Rubriken gemeinsam und richten die Repräsentation des *Antwort*-Spans über eine bilineare Form

$$g_{\text{bl}}(u, v) = u^{\top} \mathcal{W} v + b, \quad \mathcal{W} \in \mathbb{R}^{d \times d}, \quad (2.39)$$

an den Repräsentationen der einzelnen Rubrik-Spans aus [24]. Die bilineare Form erfasst sämtliche paarweisen Merkmalsinteraktionen zwischen beiden Vektoren und ist damit ausdrucksstärker als eine lineare Abbildung über der Konkatenation. Der Preis sind d^2 zusätzliche Parameter, die bei großen Modelldimensionen erheblich ins Gewicht fallen und die Überanpassungsneigung erhöhen. Diese Arbeit ist der unmittelbare methodische Bezugspunkt der vorliegenden Untersuchung: Sie etabliert das Prinzip, Rubriken als Spans in einer gemeinsam kodierten Sequenz zu behandeln, wählt als Bezugsgröße jedoch den Antwort-Span statt der Sequenzrepräsentation.

Konkatenative und multiplikative Fusion

Die in dieser Arbeit verglichenen Mechanismen setzen an genau dieser Stelle an und variieren die Konstruktion von v_k bei gleichbleibender übriger Architektur. CONCAT verknüpft beide Repräsentationen rein additiv im nachgeschalteten linearen Kopf, DIFF verwendet allein ihre komponentenweise Differenz, CONDIFF vereint beides, GATE führt eine multiplikative Konditionierung der Kontextrepräsentation durch die Rubrik ein. Die formale Spezifikation erfolgt in Abschnitt 4.2.

Der Vergleich dieser vier Varianten ist im ASAG-Kontext deshalb aufschlussreich, weil sie sich systematisch in der Zahl zusätzlicher, vollständig zu trainierender Parameter unterscheiden – von keinen (CONCAT, DIFF, CONDIFF) bis zu d^2 (GATE) – und damit erlauben, den Beitrag zusätzlicher Fusionskapazität von dem der Repräsentationsqualität des Backbones zu trennen. Eine systematische Gegenüberstellung dieser Varianten in Verbindung mit der Sequenzrepräsentation als Bezugsgröße und über mehrere Backbone-Größen hinweg liegt nach Kenntnisstand dieser Arbeit im ASAG bislang nicht vor; sie bildet den ersten Kernbeitrag der vorliegenden Untersuchung.

3 Datensätze

Die vorliegende Arbeit evaluiert das in Kapitel 4 beschriebene RUSPAN-System auf drei öffentlich verfügbaren ASAG-Benchmarks: ASAP-SAS, Beetle und SciEntsBank. Dieses Kapitel beschreibt Struktur, Statistik und Label-Skala jedes Benchmarks (Abschnitte 3.1 bis 3.3) dokumentiert die Herkunft der Rubriktexte und Musterlösungen (Abschnitt 3.4) und begründet abschließend die Auswahl dieser drei Datensätze für die vergleichende Evaluation (Abschnitt 3.5). Alle berichteten Kennzahlen wurden aus den in dieser Arbeit tatsächlich verwendeten Trainings- und Testdateien berechnet, nicht aus den ursprünglichen Wettbewerbs- bzw. Korpusbeschreibungen, da die konkreten Splits von der jeweiligen Originalveröffentlichung geringfügig abweichen können.

3.1 ASAP-SAS: Struktur, Statistik, Label-Skala

Herkunft und Format

Das Automated Student Assessment Prize – Short Answer Scoring (ASAP-SAS) ist ein von der Hewlett Foundation in Zusammenarbeit mit Kaggle im Jahr 2012 ausgerichteter Wettbewerb zur automatisierten Bewertung kurzer, konstruierter Schülerantworten [57]. Der Datensatz umfasst zehn unabhängige Aufgaben (*items*) aus den Fächern Naturwissenschaften (*Science*), Biologie und Englisch/Sprachkunde (*English, English Language Arts*), bearbeitet von Schülerinnen und Schülern der achten beziehungsweise zehnten Klassenstufe. Jede Instanz besteht aus der Frage q , der Schülerantwort a sowie einer vom Aufgabenersteller vergebenen ganzzahligen Bewertungsstufe y . ASAP-SAS enthält – anders als Beetle und SciEntsBank – von den Aufgabenerstellern verfasste, aufgabenspezifische Bewertungsrichtlinien (*general_scoring_rubrics*), jedoch keine Musterlösung s ; Letztere wird für die vorliegende Arbeit sprachmodellgestützt ergänzt (Abschnitt 3.4).

Variable Label-Skala

Ein struktureller Sonderfall von ASAP-SAS gegenüber den beiden anderen Benchmarks ist die je Aufgabe unterschiedliche Bewertungsskala: Vier der zehn Aufgaben verwenden eine vierstufige Skala $\{0, 1, 2, 3\}$, die übrigen sechs eine dreistufige Skala $\{0, 1, 2\}$. Tabelle 3.1 listet Fach, Klassenstufe, Label-Skala sowie Instanzzahlen je Aufgabe auf. Diese Variabilität ist der

unmittelbare empirische Anlass für die in Abschnitt 2.1 eingeführte rubrik-konditionierte Aufgabenformulierung (2.5): Ein Modell mit festem Klassifikationskopf gemäß (2.4) kann nicht ohne Weiteres über alle zehn Aufgaben hinweg gemeinsam trainiert werden, da die Ausgabedimension zwischen 4 und 3 wechselt.

Die Stufen sind ungleich verteilt. Über alle Aufgaben des Trainingssplits entfallen 39,5 % der Antworten auf Stufe 0, 32,7 % auf Stufe 1, 23,4 % auf Stufe 2 und 4,3 % auf Stufe 3. Innerhalb einzelner Aufgaben ist das Ungleichgewicht teils ausgeprägt: Bei den Biology-Items 5 und 6 erhalten 77,5 % beziehungsweise 84,3 % der Antworten die Stufe 0, bei Item 4 entfallen nur 7,7 % auf die höchste Stufe. Die Science-Items 1 und 2 sind dagegen vergleichsweise ausgeglichen.

Tabelle 3.1: ASAP-SAS: Aufgabenübersicht mit Fach, Klassenstufe, Label-Skala und Instanzzahlen (berechnet aus `train.csv` / `test_ua.csv`).

Item	Fach	Klasse	Label-Skala	Train n	Test n (UA)
1	Science	10	{0, 1, 2, 3}	1 672	557
2	Science	10	{0, 1, 2, 3}	1 278	426
3	English Language Arts	10	{0, 1, 2}	1 808	406
4	English Language Arts	10	{0, 1, 2}	1 657	295
5	Biology	10	{0, 1, 2, 3}	1 795	598
6	Biology	10	{0, 1, 2, 3}	1 797	599
7	English	10	{0, 1, 2}	1 799	599
8	English	10	{0, 1, 2}	1 799	599
9	English	10	{0, 1, 2}	1 798	599
10	Science	8	{0, 1, 2}	1 640	546
Gesamt				17 043	5 224

Antwortlänge und Splits

Die Schülerantworten sind im Mittel 42,3 Wörter lang (Median 39, Maximum 325 Wörter im Trainingssplit) und damit deutlich länger als die Antworten in Beetle und SciEntsBank (vgl. Abschnitte 3.2 und 3.3). Dies spiegelt den Aufgabentyp wider: Mehrere Items verlangen die Nennung mehrerer Teilaspekte (z. B. die Beschreibung mehrerer Schritte eines biologischen Prozesses), was die Bewertungsskala mit bis zu vier Stufen erklärt – die Stufe kodiert dabei häufig die Anzahl korrekt genannter Teilaspekte. Für ASAP-SAS liegt in dieser Arbeit ausschließlich der Unseen-Answers-Split (UA) vor (vgl. Abschnitt 2.1): Die Testantworten stammen zu denselben zehn Aufgaben wie das Training, sodass ASAP-SAS keine Aussage über die Generalisierung auf neue Fragen erlaubt, wohl aber die Bewertungskonvention einer variablen, aufgabenspezifischen Skala testet.

3.2 Beetle: Domäne und Test-Splits

Herkunft und Domäne

Der Beetle-Korpus entstand im Rahmen der Evaluation des BEETLE-II-Tutorendialogsystems und enthält Schülerantworten auf Erklärungs- und Definitionsfragen aus dem Bereich elementarer Elektrizitätslehre und Elektronik (Stromkreise, Spannung, Kurzschlüsse). Er wurde zusammen mit SciEntsBank im Rahmen von SemEval-2013 Task 7 als gemeinsamer Benchmark für die automatisierte Bewertung von Schülerantworten veröffentlicht [19]. Anders als ASAP-SAS liegen für Beetle sowohl eine Musterlösung s als auch Bewertungsrichtlinien in Textform vor; jede Instanz besteht damit vollständig aus dem Tupel $(q, s, a, \mathcal{R})$ gemäß (2.2).

Aufgabenstruktur und Label-Skala

Der Datensatz umfasst 56 Fragen, die sich in drei thematische Gruppen mit den Präfixen FF (23 Fragen), PC (21 Fragen) und SC (12 Fragen) gliedern. Inhaltlich betreffen die FF-Fragen überwiegend Spannungsmessungen an einzelnen Anschlussklemmen, während unter den SC-Fragen mehrfach explizit nach Kurzschlüssen (*short circuits*) gefragt wird. Für jede Frage liegt eine einheitliche, dreistufige ordinale Bewertungsskala vor, deren Stufen inhaltlich *Incorrect*, *Partially Correct* und *Correct* entsprechen und in den Rubriktexten explizit so benannt werden. Im Trainingssplit entfallen 34,4 % der Antworten auf *Incorrect*, 23,3 % auf *Partially Correct* und 42,2 % auf *Correct*; die mittlere Stufe ist damit die seltenste.

Test-Splits

Beetle stellt zwei Testsplits bereit (Tabelle 3.2). Der Unseen-Answers-Split (UA) verwendet dieselben 47 Fragen wie das Training, jedoch disjunkte Schülerantworten. Der Unseen-Questions-Split (UQ) enthält 9 Fragen, die im Training nicht vorkommen; eine Prüfung der Fragen-IDs bestätigt, dass die Schnittmenge zwischen Trainings- und UQ-Fragen leer ist. Damit erlaubt Beetle – im Unterschied zu ASAP-SAS – eine Aussage über die Generalisierung auf neue, aber domänengleiche Fragen.

Tabelle 3.2: Beetle: Instanzzahlen und Fragenzahlen je Split (berechnet aus den bereitgestellten CSV-Dateien).

Split	Instanzen n	Fragen	Bezug zu Training
Train	3 941	47	—
Test (UA)	439	47	dieselben Fragen, neue Antworten
Test (UQ)	819	9	neue, disjunkte Fragen

Die Schülerantworten sind mit durchschnittlich 9,8 Wörtern (Median 9) deutlich kürzer als in ASAP-SAS, was der Aufgabenstellung entspricht: Die Fragen verlangen typischerweise eine ein- bis zweisätzige Begründung eines beobachteten Phänomens (z. B. einer Spannungsmessung) und nicht die Aufzählung mehrerer Teilaspekte.

3.3 SciEntsBank: Domäne und Test-Splits

Herkunft und Domäne

SciEntsBank wurde gemeinsam mit Beetle im Rahmen von SemEval-2013 Task 7 veröffentlicht [19] und deckt im Unterschied zu dessen eng gefasster Domäne Elektrizitätslehre ein breites Spektrum naturwissenschaftlicher Themen ab. Die in dieser Arbeit verwendete Fassung umfasst 200 Fragen, die sich anhand ihrer Fragen-ID-Präfixe in 15 Themenblöcke gliedern (u. a. ME, WA, MX, PS, SE, HB, EM, LF, VB, FN, MS, ST, EV, II, LP mit zwischen 3 und 25 Fragen je Block). Wie bei Beetle liegen für jede Instanz Frage, Musterlösung und Rubriktexte vor.

Label-Skala

SciEntsBank verwendet in seiner ursprünglichen SemEval-Fassung ein fünfstufiges nominales Bewertungsschema (*correct*, *partially_correct_incomplete*, *contradictory*, *irrelevant*, *non_domain*), das – wie in Abschnitt 2.2 diskutiert – keine Ordinalstruktur aufweist. Die vorliegende Arbeit verwendet stattdessen, wie in der jüngeren ASAG-Literatur üblich, eine reduzierte Fassung mit den drei ordinalen Stufen *incorrect*, *partially correct* und *correct*. Von den 200 Fragen sind 169 mit dieser vollständigen dreistufigen Skala annotiert; die verbleibenden 31 Fragen liegen nur zweistufig vor (*incorrect* vs. *correct*, ohne mittlere Stufe). Diese Heterogenität der Stufenzahl *innerhalb* eines einzelnen Benchmarks ist ein weiterer empirischer Beleg für die Notwendigkeit der variablen Kandidatenmenge $\mathcal{Y}_i$ aus Formel (2.5). Im Trainingssplit verteilen sich die Antworten zu 32,9 % auf *incorrect*, 26,6 % auf *partially correct* und 40,4 % auf *correct*.

Test-Splits

SciEntsBank stellt als einziger der drei verwendeten Benchmarks alle drei in Abschnitt 2.1 eingeführten Generalisierungsbedingungen bereit (Tabelle 3.3): Unseen Answers (UA), Unseen Questions (UQ) und Unseen Domains (UD). Eine Prüfung der Fragen-ID-Präfixe bestätigt die intendierte Struktur der Splits: Die 46 Fragen des UD-Splits stammen ausschließlich aus den drei Themenblöcken WA, HB und EV, die im Trainingssplit *vollständig* fehlen – der UD-Split testet damit tatsächlich den Transfer auf gänzlich neue thematische Bereiche und nicht nur auf neue Einzelfragen. Der UQ-Split umfasst 15 Fragen aus denselben Themenblöcken wie das Training, jedoch ohne Überschneidung der konkreten Fragen-IDs. Der UA-Split schließlich verwendet zu 131 der 135 Trainingsfragen zusätzliche, disjunkte Testantworten; die geringfügige Restdifferenz von vier Fragen ist auf die konkrete Splitbildung zurückzuführen.

Die Schülerantworten sind mit durchschnittlich 11,4 Wörtern (UD) bis 12,6 Wörtern (Training) ähnlich kurz wie in Beetle. Aufgrund der großen Zahl an Fragen und Themenblöcken sowie der Verfügbarkeit des UD-Splits ist SciEntsBank unter den drei verwendeten Benchmarks derjenige mit der größten thematischen Diversität und der anspruchsvollsten Generalisierungsbedingung.

Tabelle 3.3: SciEntsBank: Instanzzahlen und Fragenzahlen je Split (berechnet aus den bereitgestellten CSV-Dateien).

Split	Instanzen n	Fragen	Bezug zu Training
Train	4 969	135	—
Test (UA)	540	135	überwiegend dieselben Fragen, neue Antworten
Test (UQ)	733	15	neue Fragen, bekannte Themenblöcke
Test (UD)	4 562	46	neue Fragen aus ungesehenen Themenblöcken

3.4 Herkunft der Rubriktexte und Musterlösungen

Keiner der drei verwendeten Benchmarks stellt sämtliche Felder des Tupels (2.2) nativ bereit. Da die vorliegende Arbeit die Rubriktexte zum zentralen Gegenstand macht, ist ihre Herkunft ausdrücklich festzuhalten.

ASAP-SAS enthält von den Aufgabenerstellenden verfasste, aufgabenspezifische Bewertungsrichtlinien, aber keine Musterlösung. Letztere wurde sprachmodellgestützt allein aus der Aufgabenstellung erzeugt.

Beetle und SciEntsBank enthalten umgekehrt native Musterlösungen, aber keine Bewertungsrichtlinien. Die in dieser Arbeit verwendeten Rubriktexte wurden für beide Benchmarks mit `gpt-4o-mini` erzeugt. Als Eingabe dienten dabei die Aufgabenstellung, die verfügbaren Referenzantworten sowie bis zu zehn Beispiel-Schülerantworten je Bewertungsstufe.

Diese Herkunft ist für die Interpretation der Ergebnisse wesentlich. Die erzeugten Texte sind keine von der Aufgabenbeschreibung unabhängige Informationsquelle, sondern eine aus ihr abgeleitete Umformulierung, die deren Inhalt erkennbar mit aufnimmt. Für die SciEntsBank-Frage *What is the difference between a rock and a mineral?* etwa hält die Beschreibung der höchsten Stufe fest, korrekte Antworten unterschieden zutreffend zwischen Gestein und Mineral und stellten dar, dass Gestein aus mehreren Mineralen bestehe, ein Mineral dagegen aus einer einzigen Substanz – also genau den Inhalt der zugehörigen Musterlösung. Abschnitt 5.3 zeigt, dass sich diese Nähe empirisch niederschlägt: Rubriktexte und Aufgabenkontext erweisen sich dort als weitgehend austauschbar.

Zwei Einschränkungen sind festzuhalten. Erstens sind die für die Erzeugung verwendeten Arbeitsabläufe nicht Bestandteil der bereitgestellten Codebasis; die Prompts und die genaue Auswahl der Beispielantworten konnten daher nicht eingesehen werden. Zweitens und daraus folgend lässt sich nicht überprüfen, aus welchem Split die herangezogenen Beispiel-Schülerantworten stammen. Wurden ausschließlich Trainingsantworten verwendet, ist das Vorgehen unbedenklich. Wären dagegen auch Testantworten eingeflossen, so enthielten die Rubriktexte Information über die Testmenge, und die rubrik-konditionierten Ergebnisse wären insbesondere auf dem UA-Split optimistisch verzerrt. Diese Frage konnte im Rahmen der vorliegenden Arbeit nicht geklärt werden und ist bei der Bewertung der Absolutwerte zu berücksichtigen; die in Kapitel 5 geführten Vergleiche *zwischen* Konfigurationen sind davon nicht betroffen, da sämtliche Bedingungen dieselben Rubriktexte verwenden.

3.5 Vergleich der Benchmarks und Auswahlbegründung

Abbildung 3.1 stellt Umfang und Split-Struktur der drei Benchmarks im Größenvergleich dar; Tabelle 3.4 ergänzt die qualitativen Merkmale. Die Auswahl ist nicht redundant, sondern gezielt komplementär in drei Hinsichten.



Abbildung 3.1: Umfang der Trainings- und Testsplits der drei verwendeten Benchmarks; die Balkenlängen sind proportional zur Instanzzahl. Deutlich werden drei für die Auswertung wesentliche Asymmetrien. Erstens ist ASAP-SAS mit 17 043 Trainingsinstanzen etwa vier- bis fünfmal so umfangreich wie die beiden anderen Benchmarks, stellt aber ausschließlich die UA-Bedingung bereit. Zweitens sind die UA- und UQ-Testsplits von Beetle und SciEntsBank mit einigen hundert Instanzen klein, sodass einzelne Fehlklassifikationen dort spürbar auf die Metrik durchschlagen. Drittens ist ausgerechnet der anspruchsvollste Split – SciEntsBank UD – der mit Abstand größte und damit der statistisch belastbarste. Die Zahlen entstammen den in dieser Arbeit tatsächlich verwendeten CSV-Dateien.

Variabilität der Label-Skala. ASAP-SAS ist unter den drei Benchmarks der einzige mit einer zwischen Einzelaufgaben wechselnden Stufenzahl und stellt damit den unmittelbaren empirischen Testfall für die in Abschnitt 2.1 motivierte rubrik-konditionierte Formulierung (2.5) dar, unter der ein Modell mit festem Klassifikationskopf nicht anwendbar ist. Beetle und SciEntsBank besitzen dagegen überwiegend eine einheitliche, dreistufige Skala und dienen so als Kontrollfälle, in denen der Vorteil der variablen Formulierung nicht aus einer unterschiedlichen Stufenzahl, sondern ausschließlich aus der semantischen Rubrik-Konditionierung selbst resultieren muss.

Verfügbarkeit von Generalisierungssplits. Nur SciEntsBank stellt mit dem UD-Split eine echte Domänentransfer-Bedingung bereit; Beetle erlaubt mit UQ immerhin den Test auf neue Fragen innerhalb derselben engen Domäne, während ASAP-SAS ausschließlich die UA-Bedingung zulässt. Diese Abstufung erlaubt es, die in Kapitel 5 berichteten Ergebnisse nach Generalisierungsanforderung gestaffelt zu interpretieren, anstatt sich auf eine einzelne, womöglich durch lexikalische Überlappung begünstigte Testbedingung zu verlassen.

Antwortlänge und Domänenbreite. ASAP-SAS unterscheidet sich von Beetle und SciEntsBank auch strukturell: Die Antworten sind im Mittel drei- bis viermal so lang, die Aufgaben

Tabelle 3.4: Gegenüberstellung der drei verwendeten ASAG-Benchmarks.

	ASAP-SAS	Beetle	SciEntsBank
Domäne	Naturwiss. / Sprache, 10 Einzelaufgaben	Elektrizitätslehre (eng, 1 Domäne)	breites naturwiss. Spektrum (15 Themen)
Native Rubriken	ja (aufgabenspez.)	nein (generiert)	nein (generiert)
Native Musterlösung	nein (generiert)	ja	ja
Label-Skala	variabel, 3–4-stufig, je Aufgabe fest	einheitlich, 3-stufig ordinal	einheitlich, überw. 3-stufig ordinal
Verfügbare Splits	UA	UA, UQ	UA, UQ, UD
Antwortlänge (Median)	39 Wörter	9 Wörter	11 Wörter
Trainingsumfang	17 043	3 941	4 969

stammen aus unterschiedlichen Schulfächern statt aus einer einzelnen naturwissenschaftlichen Domäne, und die Zahl der Trainingsinstanzen ist deutlich größer. Beetle deckt mit seiner engen Fokussierung auf Elektrizitätslehre das gegenteilige Extrem ab – eine einzelne, in sich konsistente technische Domäne mit kurzen, präzisen Antworten –, während SciEntsBank mit 15 Themenblöcken eine mittlere Domänenbreite bei ebenfalls kurzen Antworten bietet. Die drei Benchmarks unterscheiden sich damit systematisch in Antwortlänge, Domänenbreite und Aufgabenfach, wodurch überprüft werden kann, ob die in Kapitel 4 entwickelte Architektur robust über diese Variationsachsen hinweg funktioniert oder ob ihre Wirksamkeit an bestimmte strukturelle Eigenschaften eines Benchmarks gebunden ist.

Weitere im RUSPAN-Softwareprojekt verfügbare Benchmarks – insbesondere ein deutschsprachiger Datensatz (ALICE), ein portugiesischsprachiger Datensatz (PT-ASAG) sowie iStudio – werden im Rahmen dieser Arbeit nicht ausgewertet, da der Fokus auf dem monolingualen, englischsprachigen Vergleich der Fusionsmechanismen und der bidirektionalen Attention liegt; ihre Auswertung wird in Kapitel 7 als Ausblick auf mögliche Erweiterungen angesprochen.

4 Methodik: Das RUSPAN-System

Dieses Kapitel entwickelt das in dieser Arbeit untersuchte System. Abschnitt 4.1 beschreibt die Gesamtarchitektur, das Eingabeformat und die Extraktion der Span-Repräsentationen. Abschnitt 4.2 spezifiziert die vier verglichenen Fusionsmechanismen CONDIFF, CONCAT, DIFF und GATE. Abschnitt 4.3 entwickelt die anteilige Aufhebung der kausalen Attention-Maske. Abschnitt 4.4 stellt die verwendeten Backbone-Modelle vor, Abschnitt 4.5 spezifiziert die für die Ablationsexperimente benötigten Varianten, und Abschnitt 4.6 dokumentiert die Trainingskonfiguration.

Die in Kapitel 2 eingeführte Notation wird durchgängig weitergeführt: $H_i \in \mathbb{R}^{T \times d}$ bezeichnet die Zustandsmatrix der letzten Encoder-Schicht, z_i die Sequenzrepräsentation, $r_{i,k}$ die Repräsentation des k -ten Rubrik-Spans und g die Fusionsfunktion. Der Instanzindex i wird im Folgenden weggelassen, wo er nicht zur Unterscheidung benötigt wird.

4.1 Systemarchitektur: Span-Alignment-Modell

Überblick

Das System realisiert die in Abschnitt 2.5 skizzierte Architekturfamilie in vier Verarbeitungsschritten:

1. **Serialisierung.** Die textuellen Felder einer Instanz werden zu einer einzigen Zeichenkette zusammengesetzt. Dabei werden die Zeichenpositionen der Antwort und jedes Rubrik-Spans mitgeführt.
2. **Kodierung.** Die Sequenz wird in *einem* Vorwärtspass durch das Sprachmodell verarbeitet; das Ergebnis ist die Zustandsmatrix $H \in \mathbb{R}^{T \times d}$.
3. **Span-Pooling.** Aus H werden die Sequenzrepräsentation z und für jede Bewertungsstufe k die Rubrik-Repräsentation r_k extrahiert.
4. **Alignment und Scoring.** Jedes Paar (z, r_k) wird über die Fusionsfunktion g verknüpft und durch einen gemeinsamen Scoring-Kopf auf einen Skalar s_k abgebildet. Über die Skalare wird eine Verteilung gebildet und listwise optimiert.

Der entscheidende Unterschied zu einem Klassifikator mit festem Ausgabekopf liegt in Schritt 4: Der Scoring-Kopf bildet auf *einen* Skalar je Kandidatenstufe ab, nicht auf einen Vektor fester

Länge. Die Zahl der Ausgaben ergibt sich damit zur Laufzeit aus der Zahl der Rubrik-Spans in der Eingabe und nicht aus der Architektur. Abbildung 4.1 zeigt die vollständige Verarbeitungskette.

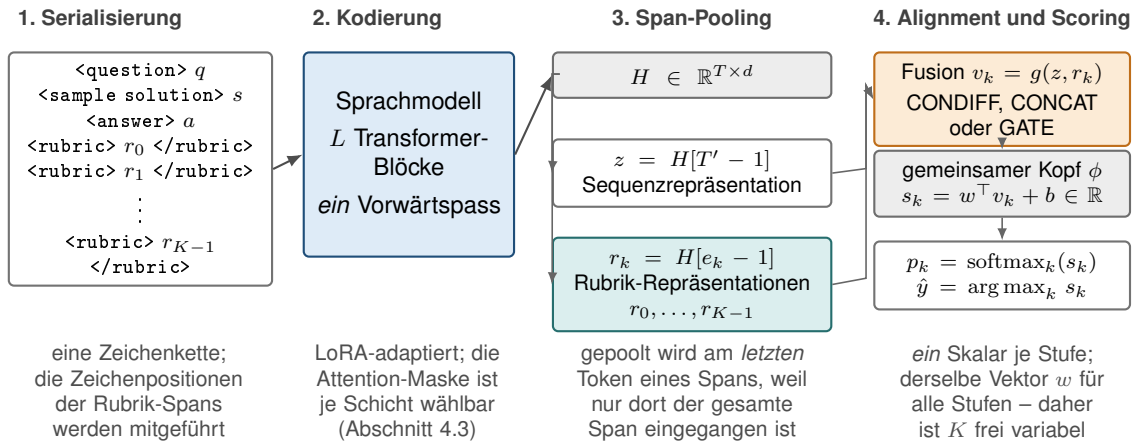


Abbildung 4.1: Verarbeitungskette des untersuchten Span-Alignment-Modells. Sämtliche Felder einer Instanz werden zu einer einzigen Zeichenkette serialisiert und in *einem* Vorwärtspass kodiert. Aus derselben Zustandsmatrix H werden anschließend die Sequenzrepräsentation z nach (4.4) und die K Rubrik-Repräsentationen r_k nach (4.2) extrahiert. Der entscheidende architektonische Punkt liegt im vierten Schritt: Der Scoring-Kopf bildet auf *einen* Skalar je Kandidatenstufe ab und nicht auf einen Vektor fester Länge, sodass sich die Zahl der Ausgaben zur Laufzeit aus der Zahl der Rubrik-Spans in der Eingabe ergibt. Die drei in Abschnitt 4.2 verglichenen Varianten unterscheiden sich ausschließlich in der Fusionsfunktion g ; alle übrigen Bestandteile bleiben identisch.

4.1.1 Eingabeformat

Serialisierung mit Feldmarkierungen

Die Felder einer Instanz werden mit XML-artigen Marken versehen und zeilenweise konkateniert. Für eine Instanz mit K Bewertungsstufen entsteht die Zeichenkette

```

<question> q </question>
<sample solution> s </sample solution>
<answer> a </answer>
<rubric> r0 </rubric>
<rubric> r1 </rubric>
⋮
<rubric> rK-1 </rubric>
[Instruktion]

```

Die Marken erfüllen zwei Funktionen. Erstens strukturieren sie die Eingabe für das Sprachmodell, das aus dem Vortraining mit XML-artiger Auszeichnung vertraut ist. Zweitens – und für die

Architektur wesentlich – liefern die schließenden Marken `</rubric>` eindeutige, im Zeichenstrom lokalisierbare Endpositionen der Rubrik-Spans, an denen später gepoolt wird.

Die Reihenfolge ist nicht beliebig: Die Rubriken stehen *nach* allen übrigen Feldern und damit insbesondere nach der Schülerantwort. Unter kausaler Attention (Abschnitt 2.6.2) folgt daraus, dass jede Rubrik-Repräsentation die vorausgehende Eingabe einschließlich der Schülerantwort gesehen hat – die Rubrik-Repräsentation ist also nicht die Einbettung eines isolierten Textes, sondern eine *kontextualisierte* Repräsentation der Bewertungsstufe angesichts der konkret vorliegenden Antwort. Diese Eigenschaft ist der eigentliche Mechanismus, über den die Architektur Antwort und Kriterium in Beziehung setzt.

Die Kandidatenstufen werden in aufsteigender Reihenfolge der Bewertungsstufe serialisiert. Die niedrigste Stufe steht damit stets an erster, die höchste an letzter Position.

Optionale Bestandteile

Sowohl der Aufgabenkontext (Frage und Musterlösung) als auch die abschließende Instruktion sind konfigurierbar und können weggelassen werden. Für die vorliegende Arbeit ist diese Konfiguration ausdrücklich festzuhalten: **Sämtliche Hauptläufe wurden ohne Aufgabenkontext und ohne abschließende Instruktion durchgeführt** (`add_context = False`, `add_suffix = False`). Die einzige Ausnahme bildet das in Abschnitt 4.5.3 beschriebene Kontext-Experiment, das diese Festlegung gezielt aufhebt. Die tatsächlich verarbeitete Eingabe besteht damit ausschließlich aus der Schülerantwort und den K Rubriktexten:

```
<answer> a </answer>
<rubric> r0 </rubric>
⋮
<rubric> rK-1 </rubric>
```

Der Einfluss dieser Festlegung ist keine bloße Formalie, sondern für die Interpretation der Ergebnisse wesentlich. Ohne Frage und Musterlösung muss die Sequenzrepräsentation z die gesamte Bewertungsgrundlage aus der Antwort und den Rubriktexten rekonstruieren; das Modell erfährt insbesondere nicht, *worauf* die vorliegende Antwort überhaupt antwortet. Die Rubriktexte sind damit die einzige aufgabenbezogene Information in der Eingabe – ein Umstand, der die in Abschnitt 5.3 berichtete Ablation besonders scharf macht: Wird auch sie entfernt, verbleibt allein der Antworttext.

Tokenisierung

Die serialisierte Zeichenkette wird mit dem Tokenizer des jeweiligen Backbones kodiert. Die maximale Sequenzlänge beträgt 2048 Token; längere Eingaben werden rechtsseitig abgeschnitten. Die Auffüllung kürzerer Sequenzen innerhalb eines Batches erfolgt ebenfalls rechtsseitig (*right padding*); als Füll-Token dient bei den Decoder-Modellen das EOS-Token. Diese Festlegung ist für das Pooling relevant und wird in Abschnitt 4.1.2 wieder aufgegriffen.

4.1.2 Span-Extraktion und Pooling

Abbildung von Zeichen- auf Token-Positionen

Die Serialisierung liefert die Spans zunächst als Zeichenintervalle. Da die Tokenisierung Zeichenpositionen nicht längentreu auf Token-Positionen abbildet, werden die Intervalle über das vom Tokenizer bereitgestellte *offset mapping* übersetzt. Für ein Zeichenintervall $[c_{\text{start}}, c_{\text{end}})$ wird die Token-Startposition als dasjenige Token bestimmt, dessen Zeichenintervall c_{start} enthält, und die Token-Endposition analog als das erste Token, dessen Intervall bei c_{end} endet oder darüber hinausreicht. Die resultierenden Token-Spans folgen der Konvention halboffener Intervalle

$$\text{Span}_k = [b_k, e_k), \quad b_k, e_k \in \{0, \dots, T\}, \quad (4.1)$$

das heißt e_k ist die Position *nach* dem letzten Token des Spans.

Rubrik-Repräsentationen

Für jede Bewertungsstufe k wird die Repräsentation als Zustand des *letzten* Tokens des zugehörigen Spans gebildet:

$$r_k = H[e_k - 1] \in \mathbb{R}^d. \quad (4.2)$$

Die Wahl der Endposition folgt unmittelbar aus der Kausalität: Unter der Maske (2.33) ist $H[j]$ eine Funktion aller Positionen $\leq j$; nur an der Endposition des Spans ist der gesamte Rubriktext in die Repräsentation eingegangen. Konkret fällt diese Position auf ein Token der schließenden Marke `</rubric>`, was den zusätzlichen Vorteil hat, dass der Pooling-Anker für alle Stufen und alle Benchmarks strukturell identisch ist.

Alternativ unterstützt die Implementierung eine Mittelung über alle Token des Spans,

$$r_k^{\text{mean}} = \frac{1}{e_k - b_k} \sum_{j=b_k}^{e_k-1} H[j], \quad (4.3)$$

die in den hier berichteten Experimenten jedoch nicht verwendet wird.

Sequenzrepräsentation

Die Sequenzrepräsentation wird als Zustand des letzten *gültigen* Tokens der Sequenz gebildet. Bezeichnet $m \in \{0, 1\}^T$ die Padding-Maske und $T' = \sum_j m_j$ die tatsächliche Sequenzlänge – das Symbol L bleibt der Zahl der Transformer-Schichten vorbehalten –, so gilt bei rechtsseitiger Auffüllung

$$z = H[T' - 1] \in \mathbb{R}^d. \quad (4.4)$$

Vor dem Pooling werden die Zustände an Padding-Positionen auf null gesetzt und ein Dropout mit Rate 0,1 angewandt. Da die Rubriken am Ende der Sequenz stehen, hat z unter kausaler Attention die gesamte Eingabe einschließlich *aller* Rubriktexte gesehen. z ist damit keine reine

Antwortrepräsentation, sondern eine Zusammenfassung der gesamten Bewertungssituation. Abbildung 4.2 zeigt Serialisierung, Span-Grenzen und Pooling-Anker im Zusammenhang.

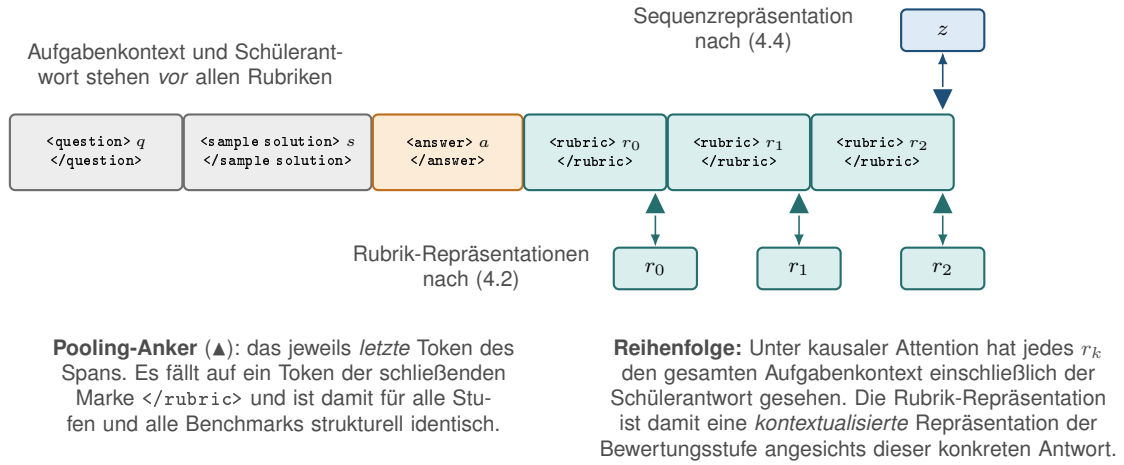


Abbildung 4.2: Serialisierung einer Instanz und Extraktion der Repräsentationen am Beispiel von drei Bewertungsstufen. Die Felder werden mit XML-artigen Marken versehen und zu einer einzigen Sequenz konkateniert; die Zeichenintervalle der Rubrik-Spans werden über das *offset mapping* des Tokenizers in Token-Intervalle nach (4.1) übersetzt. Gepoolt wird jeweils am letzten Token eines Spans, da unter kausaler Maske nur dort der vollständige Span in die Repräsentation eingegangen ist. Die Sequenzrepräsentation z wird am letzten gültigen Token der gesamten Sequenz gebildet und hat damit auch sämtliche Rubriktexte gesehen; sie ist folglich keine reine Antwortrepräsentation, sondern eine Zusammenfassung der gesamten Bewertungssituation.

Abgrenzung zur Antwort-Span-Repräsentation

Die Implementierung extrahiert zusätzlich eine Repräsentation des Antwort-Spans nach demselben Verfahren wie (4.2). Diese wird für die in dieser Arbeit untersuchten Varianten jedoch nicht als Kontext verwendet: Sämtliche berichteten Fusionsmechanismen (Abschnitt 4.2) verknüpfen die Rubrik-Repräsentation mit der *Sequenz*-Repräsentation z , nicht mit der Antwort-Repräsentation. Die Alternative – Alignment von Antwort- und Rubrik-Span – entspricht dem Ansatz von Gombert u. a. [24] und dient in dieser Arbeit als konzeptioneller Bezugspunkt.

Umgang mit variabler Stufenzahl

Da die Zahl der Bewertungsstufen K_i zwischen Instanzen variiert (vgl. Kapitel 3), werden die Instanzen eines Batches auf die maximale vorkommende Stufenzahl $K_{\max}$ aufgefüllt. Eine Gültigkeitsmaske $m^{\text{rub}} \in \{0, 1\}^{B \times K_{\max}}$ markiert die realen Stufen. Die Scores der Auffüllpositionen werden vor der Normalisierung auf $-\infty$ gesetzt und tragen damit weder zur Verteilung noch zum Gradienten bei.

4.2 Fusionsmechanismen

Die Fusionsfunktion g aus (2.19) verknüpft die Sequenzrepräsentation $z \in \mathbb{R}^d$ mit einer Rubrik-Repräsentation $r_k \in \mathbb{R}^d$ zu einem Vektor v_k , den ein gemeinsamer, für alle Stufen identischer linearer Scoring-Kopf auf einen Skalar abbildet:

$$s_k = w^\top \text{Dropout}(v_k) + b, \quad w \in \mathbb{R}^{d_g}, \quad b \in \mathbb{R}, \quad (4.5)$$

wobei d_g die von der jeweiligen Fusionsfunktion erzeugte Dimension ist und die Dropout-Rate 0,1 beträgt. Dass *derselbe* Vektor w für alle Stufen verwendet wird, ist die architektonische Bedingung dafür, dass die Zahl der Stufen frei variieren kann: Der Kopf lernt kein stufenspezifisches Muster, sondern eine allgemeine Passungsbewertung zwischen Bewertungssituation und Kriterienbeschreibung.

Die vier verglichenen Mechanismen unterscheiden sich ausschließlich in der Konstruktion von v_k und damit in der Art der modellierten Interaktion.

4.2.1 CONDIFF: Conditional-Difference-Fusion

CONDIFF konkateniert beide Repräsentationen und ergänzt ihre komponentenweise Differenz:

$$v_k^{\text{CONDIFF}} = [z \parallel r_k \parallel (z - r_k)] \in \mathbb{R}^{3d}, \quad d_g = 3d. \quad (4.6)$$

Der Differenzterm ist der eigentliche Gegenstand der Variante. Ein linearer Kopf über der reinen Konkatenation berechnet $w_z^\top z + w_r^\top r_k$, also eine additive Verrechnung zweier unabhängig gewichteter Beiträge. Ein solcher Ausdruck kann Übereinstimmung zwischen z und r_k nur indirekt darstellen. Der Differenzterm stellt dagegen die komponentenweise Abweichung explizit bereit: Ist die Bewertungssituation in denjenigen Dimensionen, die das Kriterium beschreibt, mit der Rubrik verträglich, so liegen die entsprechenden Komponenten von $z - r_k$ nahe null. Der Kopf kann damit Passung als Nähe zum Nullvektor auswerten, statt sie aus zwei separaten Termen rekonstruieren zu müssen. Diese Konstruktion überträgt das aus der Satzpaarmodellierung bekannte heuristische Matching [42, 14, 11, 47] auf das Paar aus Sequenz- und Rubrik-Repräsentation.

Bemerkenswert ist, dass der Differenzterm keine zusätzlichen Parameter im Fusionsschritt selbst einführt – er ist eine parameterfreie Transformation. Der Mehraufwand entsteht ausschließlich im Scoring-Kopf, dessen Eingabedimension von $2d$ auf $3d$ wächst, was d zusätzliche Parameter bedeutet.

4.2.2 CONCAT: Einfache Konkatenation als Baseline

CONCAT verzichtet auf den Differenzterm und konkateniert lediglich:

$$v_k^{\text{CONCAT}} = [z \parallel r_k] \in \mathbb{R}^{2d}, \quad d_g = 2d. \quad (4.7)$$

CONCAT ist als *ablative* Baseline konzipiert: Sie ist mit CONDIFF in allen Komponenten identisch – gleiche Span-Extraktion, gleiches Pooling, gleicher Kopf-Typ, gleiche Verlustfunktion – und unterscheidet sich ausschließlich durch das Fehlen von $(z - r_k)$. Ein Leistungsunterschied zwischen beiden Varianten ist daher dem Differenzterm zurechenbar und nicht auf konfundierende Architekturunterschiede zurückzuführen. Umgekehrt zeigt ein ausbleibender Unterschied an, dass der Kopf die Vergleichsoperation auch ohne explizite Bereitstellung hinreichend gut aus der Konkatenation lernen kann.

4.2.3 DIFF: Differenz als alleinige Eingabe

DIFF bildet das Gegenstück zu CONCAT: Statt der beiden Repräsentationen erhält der Kopf allein ihre komponentenweise Differenz,

$$v_k^{\text{DIFF}} = z - r_k \in \mathbb{R}^d, \quad d_g = d. \quad (4.8)$$

Zusammen mit CONCAT zerlegt DIFF die Konstruktion von CONDIFF in ihre beiden Bestandteile: CONDIFF (4.6) enthält genau die Komponenten von CONCAT (4.7) und DIFF (4.8) zugleich. Die drei Varianten erlauben damit eine vollständige Zurechnung. Ein Vorteil von CONDIFF gegenüber CONCAT ist dem Differenzterm zuzuschreiben; erreicht DIFF allein dasselbe Niveau wie CONDIFF, tragen die unveränderten Einzelrepräsentationen keine zusätzliche verwertbare Information. Wie CONCAT und CONDIFF ist DIFF im Fusionsschritt parameterfrei; der Kopf ist mit $d_g = d$ sogar kleiner als bei beiden.

4.2.4 GATE: Gated Fusion

GATE ersetzt die konkatenative durch eine multiplikative Verknüpfung. Aus der Rubrik-Repräsentation wird ein komponentenweises Gate berechnet, das die Sequenzrepräsentation moduliert; eine Residualverbindung und eine Layer-Normalisierung stabilisieren das Ergebnis:

$$\gamma_k = \sigma(W_g r_k + b_g) \in (0, 1)^d, \quad (4.9)$$

$$v_k^{\text{GATE}} = \text{LayerNorm}(z + z \odot \gamma_k) \in \mathbb{R}^d, \quad d_g = d, \quad (4.10)$$

mit $W_g \in \mathbb{R}^{d \times d}$, σ als logistischer Funktion und $\odot$ als komponentenweisem Produkt.

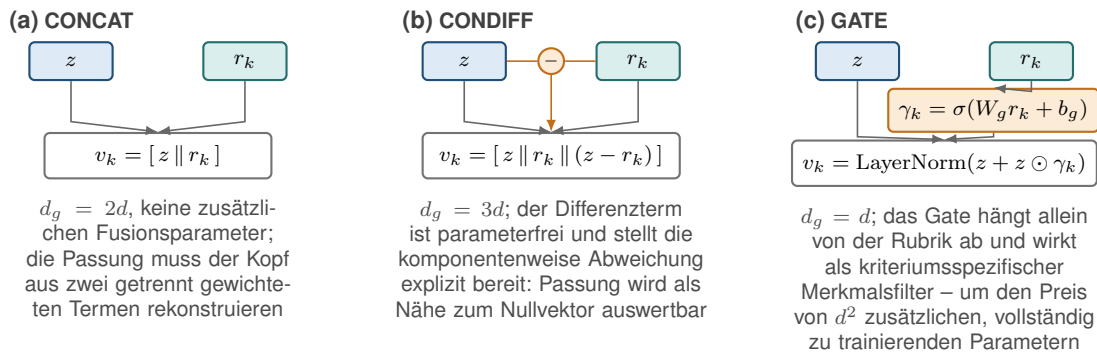
Die Konstruktion ist wie folgt zu lesen. Das Gate γ_k hängt *ausschließlich* von der Rubrik ab und wirkt als kriteriumsspezifischer Merkmalsfilter: Für jede Dimension der Sequenzrepräsentation entscheidet die Rubrik, wie stark diese Dimension in die Bewertung dieser Stufe eingeht. Merkmale, die für das betrachtete Kriterium irrelevant sind, können unterdrückt werden. Der Residualterm z in (4.10) stellt sicher, dass die ungefilterte Sequenzinformation erhalten bleibt und das Gate nur eine additive Verstärkung beisteuert; wegen $\gamma_k \in (0, 1)^d$ wirkt der Ausdruck $z + z \odot \gamma_k$ als komponentenweise Skalierung von z mit Faktoren in $(1, 2)$. Die Layer-Normalisierung fängt die dadurch entstehende Skalenverschiebung ab.

Anders als CONDIFF, CONCAT und DIFF führt GATE zusätzliche Parameter im Fusionsschritt ein: die Gate-Projektion W_g mit d^2 Parametern sowie die Normalisierungsparameter. Bei $d =$

4096 (Mistral-7B) sind dies allein für W_g rund $1,7 \cdot 10^7$ zusätzliche, vollständig zu trainierende Parameter. Da die Fusionsmodule nicht Teil des LoRA-adaptierten Backbones sind, sondern von Grund auf trainiert werden (Abschnitt 4.6), ist GATE damit die parameterreichste der vier Varianten und potenziell anfälliger für Überanpassung – ein Aspekt, der bei der Interpretation der Ergebnisse in Kapitel 5 zu berücksichtigen ist.

Übersicht

Abbildung 4.3 stellt die drei strukturell verschiedenen Mechanismen schematisch gegenüber – DIFF entspricht dem Differenzzweig von CONDIFF und wird dort nicht eigens gezeigt –, Tabelle 4.1 fasst die Kennzahlen aller vier zusammen.



In allen Varianten identisch: der gemeinsame Scoring-Kopf $s_k = w^\top \text{Dropout}(v_k) + b$. Lediglich seine Eingabedimension d_g unterscheidet sich zwischen den Varianten.

Abbildung 4.3: Drei der vier verglichenen Konstruktionen von v_k aus (2.19). Alle übrigen Bestandteile der Architektur – Serialisierung, Span-Extraktion, Pooling, Kopf-Typ und Verlustfunktion – sind identisch, sodass ein beobachteter Leistungsunterschied ausschließlich der Fusionsfunktion zurechenbar ist. CONCAT (a) dient dabei als ablative Baseline zu CONDIFF (b): Beide unterscheiden sich einzig durch den parameterfreien Differenzterm. Die vierte Variante DIFF (4.8) verwendet diesen Differenzterm allein und entspricht damit dem orangefarbenen Zweig in (b). GATE (c) ersetzt die konkatentative durch eine multiplikative Verknüpfung und ist mit der Gate-Projektion $W_g \in \mathbb{R}^{d \times d}$ die einzige Variante, die zusätzliche Parameter im Fusionsschritt einführt.

Tabelle 4.1: Die vier verglichenen Fusionsmechanismen. d bezeichnet die Modelldimension, d_g die Eingabedimension des Scoring-Kopfs.

Variante	Konstruktion von v_k	d_g	Zusätzl. Parameter
CONCAT	$[z \parallel r_k]$	$2d$	—
DIFF	$z - r_k$	d	—
CONDIFF	$[z \parallel r_k \parallel (z - r_k)]$	$3d$	—
GATE	$\text{LayerNorm}(z + z \odot \sigma(W_g r_k + b_g))$	d	$W_g \in \mathbb{R}^{d \times d}$, LayerNorm

Trainingsziel

Unabhängig vom Fusionsmechanismus werden die Skalare s_k über die gültigen Kandidatenstufen normalisiert:

$$p_k = \frac{\exp(s_k)}{\sum_{k' \in \mathcal{Y}_i} \exp(s_{k'})}, \quad (4.11)$$

und über die Trainingsmenge wird der Kreuzentropieverlust bezüglich der Referenzstufe y_i minimiert:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log p_{i, y_i}. \quad (4.12)$$

Diese Formulierung ist *listwise*: Sie bewertet alle Kandidatenstufen einer Instanz gemeinsam und in Konkurrenz zueinander, statt jede Antwort-Rubrik-Paarung isoliert als binäre Entscheidung zu behandeln. Der Gradient drückt den Score der korrekten Stufe nach oben und die Scores der übrigen Stufen *derselben Instanz* nach unten; das Modell lernt damit eine relative Rangordnung zwischen den Kriterien einer konkreten Aufgabe. Formal ist (4.12) identisch mit dem Kreuzentropieverlust eines Klassifikators, jedoch über eine instanzabhängige Kandidatenmenge statt über einen festen Labelraum. Die Vorhersage erfolgt als Argmax über die Kandidatenstufen gemäß (2.5).

4.3 Bidirektionale Attention in Decoder-only-Modellen

4.3.1 Motivation

Die in Abschnitt 2.6.2 entwickelte Beobachtung wird hier auf die konkrete Architektur angewandt. Unter kausaler Maske gilt für die Repräsentationen aus (4.2) und (4.4) eine Asymmetrie, die sich aus der Serialisierungsreihenfolge ergibt:

- Die Tokens der **Schülerantwort** stehen vor allen Rubriken. Sie können daher *keine* Rubrik sehen. Die Antwort wird also kodiert, ohne dass das Modell weiß, gegen welche Kriterien sie zu prüfen ist.
- Die Repräsentation r_k der *k*-ten **Rubrik** sieht den Kontext, die Antwort und alle *vorangehenden* Rubriken $r_0, \dots, r_{k-1}$, nicht aber die nachfolgenden. Die Repräsentationen der Stufen sind damit untereinander asymmetrisch konditioniert: Die erste Stufe wird ohne Kenntnis der Alternativen kodiert, die letzte mit vollständiger Kenntnis.

Beide Asymmetrien sind sachlich nicht motiviert. Für die Bewertungsaufgabe ist die zeitliche Ordnung der Felder ein Artefakt der Serialisierung, kein inhaltliches Merkmal: Eine menschliche Bewertungskraft liest Kriterien und Antwort in beliebiger Reihenfolge und wechselt zwischen beiden. Die Aufhebung der Kausalität stellt genau diese wechselseitige Sichtbarkeit her – die Antwort-Tokens können die Kriterien einbeziehen, und alle Stufen werden unter gleichen Sichtbarkeitsbedingungen kodiert.

Dem steht der in Abschnitt 2.6.2 diskutierte Einwand gegenüber: Das Backbone wurde ausschließlich unter kausaler Maske vortrainiert. Eine vollständige Aufhebung stellt eine Abwei-

chung von der Vortrainingsverteilung dar und kann etablierte Mechanismen wie den Attention-Sink [60] stören [35]. Statt sich zwischen beiden Extremen zu entscheiden, behandelt diese Arbeit den Grad der Aufhebung als kontinuierlich steuerbaren Parameter.

4.3.2 Umsetzung: Anteilige Aufhebung der kausalen Maske

Schichtweise Maskenwahl

Sei L die Zahl der Transformer-Schichten des Backbones und $n_{\text{bidir}} \in \{0, \dots, L\}$ die Zahl der bidirektional rechnenden Schichten. Die bidirektionalen Schichten werden stets als *oberste* zusammenhängende Gruppe gewählt; die Schichtindizes werden hier, abweichend von (2.34), entsprechend der Implementierung ab 0 gezählt:

$$\mathcal{B} = \{ \ell : L - n_{\text{bidir}} \leq \ell < L \}, \quad (4.13)$$

und die in Schicht ℓ verwendete Maske ist

$$M^{(\ell)} = \begin{cases} M^{\text{bidir}}, & \ell \in \mathcal{B}, \\ M^{\text{kausal}}, & \text{sonst,} \end{cases} \quad (4.14)$$

mit M^{bidir} und M^{kausal} gemäß (2.32) und (2.33). Die Wahl der obersten Schichten folgt der Überlegung, dass untere Schichten überwiegend lokale lexikalische und syntaktische Merkmale extrahieren und daher näher an den Vortrainingsbedingungen bleiben sollten, während die aufgabenspezifische Aggregation in den oberen Schichten stattfindet und dort von vollständiger Sichtbarkeit profitieren kann.

Eingriffstiefe

Der Eingriff beschränkt sich auf den additiven Maskenterm in (2.31). Unverändert bleiben insbesondere:

- sämtliche Gewichte des Backbones einschließlich der Projektionen W_Q, W_K, W_V, W_O ,
- die Positionskodierung: RoPE wird unverändert angewandt, die Positionsindizes bleiben die ursprünglichen. Dies ist möglich, weil RoPE gemäß (2.30) nur relative Positionen kodiert und keine Richtungsannahme enthält,
- die Berechnungsvorschrift der skalierten Punktprodukt-Attention selbst.

Konkret wird für die betroffenen Schichten lediglich die Maskenmatrix ausgetauscht: Statt der unteren Dreiecksstruktur wird eine reine Padding-Maske übergeben, die alle gültigen Positionen wechselseitig sichtbar macht. Zusätzlich wird das Kausalitäts-Flag der betroffenen Attention-Module deaktiviert, damit effiziente Attention-Implementierungen nicht ihrerseits eine kausale Maske ergänzen. Der Eingriff ist damit minimal und erlaubt eine saubere Zuschreibung beobachteter Effekte.

Parametrisierung über einen Anteil

Da die drei Backbones unterschiedlich viele Schichten besitzen (Abschnitt 4.4), wird n_{bidir} nicht absolut, sondern über einen Anteil ρ spezifiziert. Die Umrechnung erfolgt nach

$$n_{\text{bidir}}(\rho) = \begin{cases} 0, & \rho = 0, \\ \max(1, \lfloor \rho \cdot L \rfloor), & 0 < \rho < 1, \\ \rho, & \rho \geq 1 \text{ und } \rho \in \mathbb{N}, \end{cases} \quad (4.15)$$

das heißt Werte im offenen Intervall $(0, 1)$ werden als Anteil der Schichtzahl interpretiert und abgerundet, während Werte ab 1 als absolute Schichtzahl gelesen werden. Tabelle 4.2 gibt die resultierenden Schichtzahlen für die untersuchten Anteile und Backbones an; Abbildung 2.10 veranschaulicht die Aufteilung am Beispiel von Llama-3.2-1B.

Tabelle 4.2: Aus dem Anteil ρ resultierende Zahl bidirektionaler Schichten n_{bidir} je Backbone nach Vorschrift (4.15). Die Zeilen sind nach der tatsächlichen Zahl bidirektionaler Schichten geordnet, nicht nach dem nominellen Wert von ρ . Die vollständig bidirektionale Bedingung $\rho = L$ liegt für Llama-3.2-1B und -3B vor, für Mistral-7B nicht; unter drei Zufallssaaten liegt sie für beide Llama-Größen vor.

	Llama-3.2-1B	Llama-3.2-3B	Mistral-7B
ρ	$(L = 16)$	$(L = 28)$	$(L = 32)$
1,0	1	1	1
0,2	3	5	6
0,4	6	11	12
0,6	9	16	19
0,8	12	22	25
$\rho = L$	16	28	32

Auf die Randfälle sei ausdrücklich hingewiesen. Für $\rho = 0$ ist das Modell mit dem unveränderten Decoder identisch; diese Bedingung entspricht den Läufen ohne Bidir-Kennzeichnung. Die Untergrenze $\max(1, \cdot)$ in (4.15) stellt sicher, dass ein von null verschiedener Anteil stets mindestens eine Schicht umfasst. Der Wert $\rho = 1$ fällt gemäß (4.15) bereits in den Zweig der absoluten Angabe und bezeichnet daher *eine* Schicht, nicht das vollständig bidirektionale Modell; für Letzteres ist $\rho = L$ zu setzen.

Verhältnis zum Fusionsmechanismus

Die Variation von ρ ist orthogonal zur Wahl des Fusionsmechanismus. Um beide Effekte getrennt beurteilen zu können, wird der Fusionsmechanismus in allen Läufen mit variiertem ρ auf CONDIFF festgehalten. Unterschiede zwischen diesen Läufen sind damit ausschließlich der Attention-Maske zuzuschreiben.

4.4 Verwendete Backbone-Modelle

Die Architektur ist backbone-agnostisch: Sie setzt lediglich voraus, dass das Modell eine Zustandsmatrix $H \in \mathbb{R}^{T \times d}$ liefert und dass Token-Spans in dieser Matrix lokalisierbar sind. Untersucht werden drei Decoder-only-Modelle unterschiedlicher Größe sowie zwei Encoder-Modelle als Vergleichsbasis.

4.4.1 Llama-3.2-1B/3B

Die beiden kleineren Backbones stammen aus der Llama-3.2-Familie [25]. Es handelt sich um Decoder-only-Transformer mit Pre-Norm-Aufbau, RMS-Normalisierung [62], SwiGLU-Feed-Forward-Schichten [51], Grouped-Query Attention und RoPE als Positionskodierung [54]. Die 1B-Variante besitzt 16 Schichten bei einer Modelldimension von $d = 2048$, die 3B-Variante 28 Schichten bei $d = 3072$.

Der Größenunterschied zwischen beiden Varianten ist für diese Arbeit von eigenem Interesse: Er erlaubt es zu prüfen, ob die Wirksamkeit der Fusionsmechanismen und der bidirektionalen Attention von der Kapazität des Backbones abhängt. Insbesondere ist zu erwarten, dass ein parameterreicher Fusionsmechanismus wie GATE bei kleineren Backbones relativ stärker ins Gewicht fällt, da das Verhältnis zwischen neu trainierten Fusionsparametern und adaptierten Backbone-Parametern günstiger ausfällt.

4.4.2 Mistral-7B

Als größtes Backbone dient Mistral-7B-v0.1 [29] mit 32 Schichten und einer Modelldimension von $d = 4096$. Das Modell verwendet ebenfalls Pre-Norm, RMS-Normalisierung, SwiGLU und RoPE, ergänzt um Grouped-Query Attention und Sliding-Window Attention zur Effizienzsteigerung bei langen Sequenzen.

Die Einbindung erfolgt über dieselbe Erweiterung wie bei Llama: Die schichtweise Maskenwahl nach (4.14) ist für beide Modellfamilien identisch implementiert, sodass die Bidir-Bedingungen über alle drei Decoder-Backbones hinweg vergleichbar sind.

4.4.3 BERT/RobERTa als Encoder-Baseline

Als Vergleichsbasis dienen BERT-base-uncased [16] und RoBERTa-base [37]. Beide sind Encoder-only-Modelle mit 12 Schichten und $d = 768$, die bereits im Vortraining vollständig bidirektional rechnen. Sie erlauben damit die Beantwortung einer naheliegenden Frage: Wenn bidirektionale Sichtbarkeit für die Aufgabe vorteilhaft ist, wie schneidet dann ein Modell ab, das diese Eigenschaft von Grund auf besitzt – allerdings bei erheblich geringerer Kapazität und ohne das Weltwissen großer Sprachmodelle?

Die Span-Alignment-Architektur wird für die Encoder-Baselines unverändert übernommen; angepasst wird lediglich die Serialisierung. Da die Encoder-Tokenizer keine XML-artigen Marken

als strukturelle Trenner vortrainiert haben, werden die Felder stattdessen durch das modelleigene [SEP]-Token getrennt, und der Rubrik-Span wird so festgelegt, dass er das nachfolgende [SEP]-Token einschließt. Der Pooling-Anker nach (4.2) fällt damit auf dieses Trennzeichen, das die strukturelle Rolle der schließenden Marke `</rubric>` übernimmt. Die Fusionsmechanismen, der Scoring-Kopf und die Verlustfunktion bleiben identisch, sodass der Vergleich zwischen Encoder- und Decoder-Backbones die Architektur konstant hält.

4.5 Varianten für die Ablationsexperimente

Neben den vier Fusionsmechanismen und der Variation der Attention-Maske werden in Kapitel 5 vier weitere Konfigurationen ausgewertet. Sie dienen nicht der Verbesserung des Systems, sondern der Zurechnung beobachteter Leistung auf einzelne Bestandteile. Dieser Abschnitt spezifiziert sie.

4.5.1 Fester Klassifikationskopf als Baseline

Die grundlegendste Vergleichsbedingung verzichtet vollständig auf das Span-Alignment. Die Eingabe wird unverändert serialisiert und kodiert, und auch die Sequenzrepräsentation z wird nach (4.4) gebildet; die Rubrik-Repräsentationen r_k werden jedoch nicht in die Bewertung einbezogen. Stattdessen bildet eine einzelne lineare Abbildung

$$(s_0, \dots, s_{C-1})^\top = W_{\text{cls}} \text{Dropout}(z) + b_{\text{cls}}, \quad W_{\text{cls}} \in \mathbb{R}^{C \times d}, \quad (4.16)$$

die Sequenzrepräsentation direkt auf C Logits ab, und optimiert wird der gewöhnliche Kreuzentropieverlust über den festen Labelraum. Diese Variante entspricht damit exakt der klassischen Formulierung (2.4) aus Abschnitt 2.1.

Zwei Eigenschaften sind für die Auswertung wesentlich. Erstens *sieht* das Modell die Rubriktexte weiterhin – sie sind Teil der serialisierten Eingabe und gehen über die kausale Attention in z ein; sie werden lediglich nicht als Spans ausgelesen. Ein Vergleich dieser Variante mit dem Span-Alignment misst daher den Beitrag der expliziten Span-Extraktion, nicht den der Rubrikinformation (Abschnitt 5.6). Zweitens setzt (4.16) eine über alle Instanzen konstante Stufenzahl C voraus. Die Variante ist deshalb auf ASAP-SAS nicht anwendbar, dessen Stufenzahl je Aufgabe zwischen drei und vier wechselt (Kapitel 3) – ein unmittelbarer praktischer Beleg für die in Abschnitt 2.1 gegebene Begründung der rubrik-konditionierten Formulierung.

4.5.2 Ablation der Rubriktexte

Für die Ablation werden sämtliche Rubriktexte vor der Serialisierung aus der Instanz entfernt. Die Eingabe besteht dann nur noch aus denjenigen Feldern, die nach der Konfiguration des Aufgabenkontexts verbleiben. Da die Rubrik-Spans dadurch entfallen, ist diese Bedingung ausschließlich mit dem festen Klassifikationskopf aus Abschnitt 4.5.1 kombinierbar; die Implementierung erzwingt dies. Alle übrigen Bestandteile – Backbone, LoRA-Konfiguration, Pooling, Kopf,

Verlustfunktion, sämtliche Hyperparameter – bleiben unverändert, sodass ein beobachteter Unterschied allein dem Vorhandensein der Rubriktexte zurechenbar ist.

4.5.3 Variation des Aufgabenkontexts

Wie in Abschnitt 4.1.1 dokumentiert, enthalten die Hauptläufe dieser Arbeit weder Frage noch Musterlösung. Um den Beitrag der Rubriktexte von dem der Aufgabenbeschreibung zu trennen, wird diese Festlegung in einem eigenen Experiment aufgehoben: Frage und Musterlösung werden der Serialisierung vorangestellt, alles Übrige bleibt unverändert. In Verbindung mit der Ablation aus Abschnitt 4.5.2 ergibt sich ein vollständiges 2×2 -Schema über die Verfügbarkeit beider Informationsquellen, das in Abschnitt 5.3 ausgewertet wird.

4.5.4 Antwort-Span als Bezugsgröße der Fusion

Die in Abschnitt 4.2 spezifizierten Mechanismen verwenden durchgängig die Sequenzrepräsentation z als Bezugsgröße. Zur Prüfung dieser Entwurfsentscheidung wird eine Variante untersucht, die stattdessen die Repräsentation des Antwort-Spans heranzieht. Diese wird nach demselben Verfahren wie die Rubrik-Repräsentationen gebildet, also als Zustand des letzten Tokens des Antwort-Spans,

$$t = H[e_{\text{ans}} - 1] \in \mathbb{R}^d, \quad (4.17)$$

und tritt in der Fusion an die Stelle von z :

$$v_k^{\text{t-concat}} = [t \parallel r_k] \in \mathbb{R}^{2d}. \quad (4.18)$$

Die Variante ist damit die exakte Entsprechung von CONCAT (4.7) mit ausgetauschter Bezugsgröße; jeder Unterschied ist allein dieser Vertauschung zuzuschreiben. Inhaltlich unterscheiden sich beide Größen darin, welchen Kontext sie gesehen haben: Unter kausaler Maske steht t vor allen Rubriktexten und kennt diese nicht, während z am Sequenzende steht und sämtliche Stufen gesehen hat. Diese Variante entspricht zugleich dem Ansatz von Gombert u. a., die das Alignment am Antwort-Span verankern [24]; ihre Auswertung erfolgt in Abschnitt 5.5.

4.6 Trainingskonfiguration

Parameter-effizientes Finetuning

Die Backbones werden nicht vollständig, sondern mittels LoRA [28] adaptiert (vgl. Abschnitt 2.7). Die Konfiguration verwendet Rang $r = 64$ und Skalierungsfaktor $\alpha = 64$, woraus nach (2.37) ein effektiver Skalierungsfaktor $\alpha/r = 1$ folgt. Die LoRA-Dropout-Rate beträgt 0,1; Bias-Terme werden nicht adaptiert. Adapter werden auf *sämtliche* linearen Module der Transformer-Blöcke angewandt, das heißt bei den Llama- und Mistral-Backbones auf die vier Attention-Projektionen W_Q, W_K, W_V, W_O sowie auf die drei Projektionen der SwiGLU-Feed-Forward-Schicht.

Von der LoRA-Adaption ausgenommen sind die architektureigenen Komponenten – die Gate-Projektion und Layer-Normalisierung der GATE-Variante sowie der Scoring-Kopf (4.5). Diese sind nicht Teil des vortrainierten Backbones, werden zufällig initialisiert und vollständig trainiert.

Optimierung

Tabelle 4.3 fasst die Trainingskonfiguration zusammen. Optimiert wird mit AdamW; bei den Decoder-Backbones kommt eine speichereffiziente, seitenweise ausgelagerte Variante in 32-Bit-Genauigkeit zum Einsatz, bei den Encoder-Baselines die Standardimplementierung. Die Lernrate folgt einem Kosinus-Zeitplan mit kurzer Aufwärmphase. Die Decoder-Backbones rechnen in Training und Inferenz in `bf16` und verwenden zusätzlich Gradient Checkpointing, um den Speicherbedarf der Aktivierungen zu senken; die Encoder-Baselines rechnen in 32-Bit-Gleitkommagenauigkeit.

Tabelle 4.3: Trainingskonfiguration der Decoder-Backbones. Die Werte entsprechen der in den ausgewerteten Decoder-Läufen verwendeten Konfiguration; die Abweichungen der Encoder-Baselines beschreibt der folgende Unterabschnitt.

Parameter	Wert
LoRA-Rang r / Skalierung α	64 / 64
LoRA-Dropout	0,1
LoRA-Zielmodule	alle linearen Module
Batchgröße (pro Gerät)	4
Gradientenakkumulation	4
Effektive Batchgröße	16
Maximale Epochenzahl	5
Lernrate	$5 \cdot 10^{-5}$
Lernraten-Zeitplan	Kosinus
Aufwärmanteil	0,01
Gewichtsverfall	0,01
Adam- ϵ	10^{-8}
Gradienten-Clipping (Norm)	1,0
Dropout (Fusion / Scoring-Kopf)	0,1
Numerisches Format	<code>bf16</code>
Maximale Sequenzlänge	2048 Token
Aufgabenkontext (Frage, Musterlösung)	nicht enthalten
Abschließende Instruktion	nicht enthalten
Zufallssaaten	114 514 (Referenz), 1, 2

Abweichende Konfiguration der Encoder-Baselines

Die Encoder-Backbones werden nicht in derselben Konfiguration trainiert wie die Decoder-Modelle. Da BERT und RoBERTa eine maximale Kontextlänge von 512 Token besitzen, ist

die für die Decoder verwendete Grenze von 2048 Token dort nicht anwendbar; entsprechend wird auf 512 Token gekürzt. Ferner werden die Encoder über zehn statt fünf Epochen trainiert und ohne LoRA vollständig feinabgestimmt, da die Zahl ihrer Parameter dies ohne weiteres zulässt. Sie rechnen in 32-Bit-Gleitkommagenauigkeit, mit einer Batchgröße von 16 und ohne Gradientenakkumulation.

Für die Encoder-Hauptläufe ist diese Konfiguration allerdings nicht durch Dateien belegt: Sie stammen aus einer frühen Trainingswelle, von der ausschließlich die Vorhersagen erhalten sind. Insbesondere ihre Lernrate lässt sich nicht nachweisen. Vollständig dokumentiert ist dagegen die in Abschnitt 5.9 berichtete Lernratenvariation. Deren Konfigurationsdateien weisen zehn Epochen, vollständiges Finetuning ohne LoRA, 32-Bit-Gleitkommagenauigkeit und eine Batchgröße von 16 ohne Gradientenakkumulation aus, bei einem BERT-Lauf eine Batchgröße von 32. Dass die Hauptläufe ebenso konfiguriert waren, ist plausibel, aber nicht belegt.

Diese Unterschiede sind bei der Interpretation des Encoder-Decoder-Vergleichs zu berücksichtigen: Der Vergleich hält die *Architektur* konstant – Serialisierung, Span-Pooling, Fusionsmechanismus, Scoring-Kopf und Verlustfunktion sind identisch –, nicht jedoch sämtliche Optimierungsbedingungen. Insbesondere erweist sich die Lernrate für die Encoder als kritischer Parameter, wie die in Abschnitt 5.9 berichtete Variation zeigt.

Modellauswahl und Validierung

Da die drei verwendeten Benchmarks keinen offiziellen Entwicklungssplit bereitstellen, werden 10 % des Trainingssplits zufällig als Validierungsmenge abgetrennt; die Aufteilung erfolgt mit fester Zufallsaat und ist damit über alle Läufe hinweg identisch. Nach jeder Epoche wird auf dieser Menge evaluiert. Als Auswahlkriterium dient die Accuracy; der Prüfpunkt mit dem höchsten Wert wird am Ende des Trainings geladen und für die Testevaluation verwendet. Ergänzend beendet ein Early-Stopping-Kriterium das Training, wenn sich die Validierungsleistung über drei aufeinanderfolgende Epochen nicht verbessert.

Dass die Modellauswahl über Accuracy und nicht über die primär berichteten Metriken QWK oder Macro- F_1 erfolgt, ist eine Eigenschaft der verwendeten Konfiguration und bei der Interpretation der Ergebnisse zu berücksichtigen: Unter unbalancierten Labelverteilungen (vgl. Kapitel 3) kann die Accuracy Prüfpunkte bevorzugen, die auf häufigen Stufen stark und auf seltenen Stufen schwach abschneiden.

Evaluationsprotokoll

Die Testevaluation erfolgt auf den in Kapitel 3 beschriebenen Splits. Berichtet werden die in Abschnitt 2.2 definierten Metriken: Quadratic Weighted Kappa mit quadratischer Gewichtungsmatrix (2.12), Macro- F_1 (2.9) und Accuracy (2.7). Die Berechnung erfolgt über alle Testinstanzen eines Splits gemeinsam. Zusätzlich werden die Metriken je Aufgabe getrennt berechnet und protokolliert, was insbesondere für ASAP-SAS mit seinen zehn heterogenen Aufgaben eine differenzierte Auswertung erlaubt (vgl. Kapitel 5).

Die Erhebung erfolgt in zwei Stufen. In der ersten Stufe wurde jede Konfiguration einmal mit der Referenzsaat 114514 trainiert; diese Läufe decken den gesamten Untersuchungsraum ab

und dienen der Übersicht. In der zweiten Stufe wurden diejenigen Konfigurationen, auf denen die inhaltlichen Aussagen dieser Arbeit beruhen, zusätzlich mit den Saaten 1 und 2 wiederholt. Über sämtliche Benchmarks, Backbones, Varianten und Splits hinweg liegen dadurch 72 Zellen mit je drei Läufen vor.

Diese Zweiteilung schlägt auf die Auswertung durch: Einzelläufe werden in Kapitel 5 ausschließlich dokumentarisch berichtet, während jede inhaltliche Aussage auf Mittelwerten über drei Saaten beruht und gegen die in Abschnitt 5.2 bestimmte Rauschgrenze geprüft wird. Wo möglich stammen die drei Läufe einer Gruppe aus derselben Softwareumgebung, da ein Umgebungswechsel eine eigene, von der Zufallssaat unabhängige Störgröße darstellt (Abschnitt 5.2).

5 Experimente und Ergebnisse

Dieses Kapitel berichtet die empirischen Ergebnisse. Es folgt dabei nicht der Reihenfolge der Benchmarks, sondern der Reihenfolge der Befunde: Abschnitt 5.1 beschreibt das experimentelle Design, Abschnitt 5.2 bestimmt die Rauschgrenze, an der anschließend jeder berichtete Unterschied gemessen wird. Abschnitt 5.3 berichtet den einzigen Eingriff mit großem Effekt – die Verfügbarkeit aufgabenbezogener Information – und schränkt dabei zugleich die Ausgangsannahme dieser Arbeit ein. Die Abschnitte 5.4 bis 5.6 untersuchen die Ausgestaltung der Fusion; keine ihrer Varianten erzeugt dort einen Unterschied oberhalb der Rauschgrenze. Abschnitt 5.7 behandelt die Attention-Maske, deren Wirkung als einzige Architekturvariante die Rauschgrenze deutlich überschreitet, dabei aber je nach Benchmark und Backbone das Vorzeichen wechselt. Die Abschnitte 5.8 bis 5.11 behandeln die Backbone-Größe – für die sich zeigt, dass der vorliegende Vergleich keine Aussage trägt –, den Encoder-Vergleich, die aufgabenweise Auswertung und die instabilen Läufe. Abschnitt 5.12 prüft abschließend, welche Befunde unter Macro- F_1 bestehen bleiben.

Sofern nicht anders angegeben, sind sämtliche Zahlen kaufmännisch auf drei Nachkommastellen gerundet; Differenzen werden aus den ungerundeten Werten gebildet. Die Rohwerte liegen den Auswertungsdateien der jeweiligen Läufe bei.

5.1 Experimentelles Design

Untersuchte Bedingungen

Untersucht werden fünf Variationsachsen. Die ersten vier betreffen die Architektur, die fünfte das Backbone:

1. **Verfügbarkeit des Rubriktexts.** Der Kontrast zwischen einem Modell, dem die Rubriktexte als Eingabe zur Verfügung stehen, und einem sonst identischen Modell, dem sie entzogen werden (Abschnitt 5.3).
2. **Fusionsmechanismus** bei unveränderter kausaler Attention: CONDIFF, CONCAT, DIFF und GATE (Abschnitt 4.2); DIFF wurde ausschließlich auf Llama-3.2-3B untersucht.
3. **Bezugsgröße der Fusion:** die Sequenzrepräsentation z gegenüber der Repräsentation des Antwort-Spans (Abschnitt 5.5).

4. **Attention-Maske** bei fest auf CONDIFF gesetztem Fusionsmechanismus: die Anteile $\rho \in \{0,2; 0,4; 0,6; 0,8; 1,0\}$ sowie die vollständig bidirektionale Bedingung $n_{\text{bidir}} = L$ (Abschnitt 4.3.2).
5. **Backbone**: Llama-3.2-1B, Llama-3.2-3B und Mistral-7B als Decoder-Modelle sowie BERT-base-uncased und RoBERTa-base als Encoder-Vergleichsbasis (Abschnitt 4.4).

Da die Läufe mit variierter Maske durchgängig CONDIFF verwenden, ist der Lauf mit CONDIFF und unveränderter kausaler Maske zugleich die Referenzbedingung $\rho = 0$ für die vierte Achse. Fusions- und Maskeneffekte sind damit gegeneinander abgrenzbar.

Interpretation des Anteilsparameters

Für die vierte Achse ist die in Abschnitt 4.3.2 beschriebene Umrechnungsvorschrift (4.15) von zentraler Bedeutung. Da Werte ab 1 als absolute Schichtzahl gelesen werden, entspricht die Bedingung $\rho = 1,0$ einer *einzigsten* bidirektionalen Schicht und nicht dem vollständig bidirektionalen Modell; Letzteres erfordert die Angabe $\rho = L$. Abbildung 2.10 und Tabelle 4.2 in Kapitel 4 geben die resultierenden Schichtzahlen an.

Die Bedingungen bilden daher *keine* monoton ansteigende Reihe in ihrer nominellen Bezeichnung. Geordnet nach dem tatsächlichen Grad der Bidirektionalität ergibt sich

$$\rho = 0 \prec \rho = 1,0 \prec \rho = 0,2 \prec \rho = 0,4 \prec \rho = 0,6 \prec \rho = 0,8 \prec \rho = L, \quad (5.1)$$

das heißt die nominell als „vollständig“ erscheinende Bedingung $\rho = 1,0$ liegt dem kausalen Ausgangsmodell am nächsten. Sämtliche Ergebnistabellen dieses Kapitels sind daher nach der tatsächlichen Schichtzahl n_{bidir} sortiert und geben diese explizit an.

Vollständigkeit der Auswertung

Die Auswertung ist nicht in allen Zellen vollständig. Für Mistral-7B liegen auf Beetle und SciEntsBank ausschließlich die Bedingungen CONDIFF, $\rho = 0,2$, $\rho = 0,6$ und $\rho = 1,0$ vor; die Fusionsvarianten CONCAT und GATE sowie die Anteile $\rho = 0,4$ und $\rho = 0,8$ wurden für dieses Backbone auf diesen beiden Benchmarks nicht ausgeführt. Für BERT liegt auf Beetle nur der UA-Split vor. Die vollständig bidirektionale Bedingung liegt für beide Llama-Größen unter drei Zufallssaaten vor. Die Variation der Bezugsgröße und die Fusionsvariante DIFF wurden unter drei Saaten nur auf Llama-3.2-3B untersucht, jeweils auf Beetle und SciEntsBank; die Ablation des Rubriktexts wurde auf beiden Llama-Größen wiederholt. Die Lernratenvariation der Encoder umfasst RoBERTa auf Beetle und SciEntsBank sowie BERT auf SciEntsBank. Fehlende Zellen sind in den Tabellen mit „—“ gekennzeichnet.

Die Einzelläufe der Bezugsgrößen-Variante auf Llama-3.2-1B werden nicht in eigenen Tabellen geführt, da sie mit einer Zufallssaat keine Interpretation tragen. Dasselbe gilt für t-concat auf ASAP-SAS (0,814 und 0,837 QWK für Llama-3.2-1B und -3B gegenüber 0,828 und 0,842 für CONCAT) sowie für einen Einzellauf des festen Klassifikationskopfs mit Llama-3.2-3B auf Beetle (0,730 / 0,534 QWK auf UA / UQ). Die vollständig bidirektionale Bedingung auf ASAP-SAS ist dagegen in Tabelle 5.14 aufgeführt.

Wo mehrere Zufallssaaten vorliegen, wird der Mittelwert über die Saaten berichtet und die Spannweite mitgeführt. Wo nur ein Lauf vorliegt, ist dies im Tabellenkopf vermerkt; solche Werte dienen der Dokumentation, nicht der Interpretation.

5.2 Rauschgrenze: Streuung zwischen Läufen

Bevor einzelne Konfigurationen verglichen werden, ist die in Abschnitt 2.2 eingeführte Rauschgrenze empirisch zu bestimmen. Andernfalls ließe sich nicht entscheiden, ob ein beobachteter Unterschied auf die untersuchte Variation oder auf die stochastischen Anteile des Trainings zurückgeht.

Reproduzierbarkeit bei identischer Zufallssaat

Ein erster Befund betrifft die Wiederholbarkeit selbst, und er fällt eindeutiger aus als erwartet. Zwei Konfigurationen wurden unter identischer Zufallssaat, identischen Hyperparametern und identischer Softwareumgebung ein zweites Mal ausgeführt. Die resultierenden Vorhersagedateien sind *byteweise identisch* mit denen der ersten Ausführung; sämtliche vier Messwerte (zwei Konfigurationen $\times$ zwei Splits) stimmen exakt überein. Innerhalb einer festen Umgebung ist das Training in diesem Aufbau also vollständig deterministisch: Die in Abschnitt 2.2 als mögliche Störquelle genannte nicht-assoziative Gleitkommaaddition auf dem Grafikprozessor schlägt hier nicht durch.

Dieser Befund korrigiert eine naheliegende Fehldeutung. In den Daten existiert ein weiteres Paar von Läufen mit derselben Zufallssaat, das *nicht* übereinstimmt: Für CONDIFF mit Llama-3.2-1B auf Beetle ergab die Wiederholung 0,718 statt 0,696 QWK auf UA und 0,472 statt 0,532 auf UQ. Dieses Paar stammt jedoch aus zwei verschiedenen Softwareumgebungen und misst damit nicht die Wiederholbarkeit, sondern den im folgenden Unterabschnitt behandelten Umgebungseinfluss.

Die praktische Konsequenz ist günstig: Die weiter unten bestimmte Spannweite über mehrere Zufallssaaten ist ein reiner Saat-Effekt und nicht zusätzlich mit Wiederholungsrauschen überlagert.

Höhe der Rauschgrenze

Zur Bestimmung der Streuung wurden Konfigurationen unter je drei Zufallssaaten wiederholt. Über sämtliche Benchmarks, Backbones, Varianten und Splits hinweg liegen dadurch 72 Zellen mit drei Läufen vor. Tabelle 5.1 fasst die Verteilung der Spannweiten (2.14) zusammen.

Die typische Spannweite beträgt rund 0,037 QWK, gut drei Viertel aller Zellen liegen bei höchstens 0,05. Der Maximalwert von 0,118 entfällt auf GATE mit Llama-3.2-1B auf dem Beetle-UQ-Split. Bemerkenswert ist die Breite der Verteilung selbst: Sie reicht von 0,006 bis 0,118, sodass eine pauschale Schwelle nicht ausreicht. Im Folgenden wird daher zweistufig geprüft – gegen den Median 0,037 als Orientierung und, wo drei Saaten vorliegen, gegen die tatsächliche Spannweite der betroffenen Zellen.

Tabelle 5.1: Spannweite Δ des QWK über drei Zufallssaaten, aggregiert über 72 Zellen.

Kennwert	Spannweite Δ (QWK)
Median	0,037
Mittelwert	0,038
1. Quartil	0,023
3. Quartil	0,049
Minimum	0,006
Maximum	0,118
Anteil der Zellen mit $\Delta \leq 0,05$	76 %

Einfluss der Softwareumgebung

Ein Teil der Läufe wurde zu einem späteren Zeitpunkt und damit unter einer neueren Version der verwendeten Bibliotheken ausgeführt (`transformers` 5.16.1 statt 4.57.6). Da eine Konfiguration in beiden Umgebungen mit identischer Zufallssaat vorliegt, lässt sich der Einfluss dieser Änderung direkt beziffern (Tabelle 5.2).

Tabelle 5.2: Einfluss der Bibliotheksversion bei identischer Konfiguration und identischer Zufallssaat (SciEntsBank, Llama-3.2-3B, fester Klassifikationskopf).

	UA	UQ	UD
<code>transformers</code> 4.57.6	0,747	0,557	0,648
<code>transformers</code> 5.16.1	0,763	0,543	0,648
Differenz	+0,016	−0,014	−0,001

Der Versionswechsel bewirkt eine Änderung von höchstens 0,016 QWK ohne einheitliche Richtung. Eine zweite, weniger kontrollierte Messung liefert einen größeren Wert: Das im vorigen Unterabschnitt erwähnte CONDIFF-Paar auf Beetle, dessen beide Läufe aus unterschiedlichen Umgebungen stammen, weicht um +0,022 beziehungsweise −0,060 QWK ab. Der Umgebungseinfluss liegt damit in derselben Größenordnung wie der Saat-Effekt, im ungünstigen Fall etwa bei der Hälfte der maximalen Saat-Spannweite.

Daraus ergeben sich drei klar getrennte Störgrößen, die im Folgenden auseinanderzuhalten sind:

- gleiche Umgebung, gleiche Saat: keine Abweichung (bytwweise identisch),
- gleiche Umgebung, andere Saat: Spannweite im Median 0,037, maximal 0,118 QWK,
- andere Umgebung, gleiche Saat: bis 0,060 QWK.

Für die Auswertung folgt daraus die Forderung, Drei-Seed-Gruppen möglichst innerhalb einer Umgebung zu bilden. Erfüllt ist sie für 11 der 28 Gruppen: Innerhalb von `transformers` 4.57.6 liegen die Ablation des Rubriktexts auf Beetle, die beiden Bedingungen mit Aufgabenkontext

auf SciEntsBank, CONDIFF auf Beetle mit Llama-3.2-1B, die vollständig bidirektionale Bedingung auf Llama-3.2-1B und DIFF; innerhalb von 5.16.1 die Ablation auf Llama-3.2-3B. In den übrigen 17 Gruppen stammt der Lauf mit der Saat 114514 aus einer anderen Umgebung als die beiden weiteren Läufe. Bei 11 Gruppen – sämtlichen Gruppen von CONDIFF, CONCAT und GATE mit Ausnahme der genannten – stammt er aus der ersten Trainingswelle, deren Umgebung nicht dokumentiert ist. Bei sechs Gruppen stammt er aus 4.57.6, die beiden anderen Läufe aus 5.16.1: den beiden SciEntsBank-Bedingungen ohne Aufgabenkontext auf Llama-3.2-1B sowie der vollständig bidirektionalen Bedingung und t-concat auf Llama-3.2-3B, jeweils auf Beetle und SciEntsBank.

In diesen Gruppen ist der Umgebungsbeitrag in der ausgewiesenen Spannweite enthalten. Für Vergleiche ohne nachweisbaren Unterschied wirkt das konservativ, da eine vergrößerte Spannweite die Auswertungsregel strenger macht. Für Vergleiche mit Befund ist dagegen zu prüfen, ob die Richtung auch innerhalb einer einzigen Umgebung besteht. Beschränkt man die Vergleiche auf Läufe aus 4.57.6, bleiben die Vorzeichen erhalten: Kontext allein liegt gegenüber dem einzigen Lauf mit Rubrik allein aus dieser Umgebung um $+0,103 / +0,080 / +0,068$ QWK vorn; die vollständig bidirektionale Bedingung auf Llama-3.2-3B ergibt gegenüber den beiden kausalen Läufen derselben Umgebung $-0,085 / -0,110$ auf Beetle und $+0,053 / +0,048 / -0,011$ auf SciEntsBank, t-concat auf Beetle $+0,030 / +0,010$ gegenüber CONCAT. Da hier jeweils nur ein Lauf einer Bedingung eingeht, bestätigt diese Prüfung die Richtung, nicht die Höhe der Effekte. Insbesondere liegt der Gewinn von $0,060$ QWK auf SciEntsBank-UQ in der Größenordnung des oben bezifferten Umgebungseinflusses; innerhalb einer Umgebung beträgt er $0,048$.

Auswertungsregel

Aus den vorstehenden Befunden folgt die Regel, nach der dieses Kapitel durchgängig verfährt. Ein Unterschied zwischen zwei Konfigurationen wird nur dann als Effekt bezeichnet, wenn die Differenz der Mittelwerte die größere der beiden Spannweiten übersteigt. Liegt sie darunter, überlappen die Spannweiten aber nicht, wird dies als schwacher Hinweis vermerkt und nicht als Ergebnis behauptet. Überlappen die Spannweiten, gilt der Unterschied als nicht nachweisbar. Abbildung 2.4 stellt diese Regel schematisch dar.

5.3 Der Beitrag aufgabenbezogener Information

Die erste und für die Fragestellung dieser Arbeit grundlegendste Frage lautet, welchen Beitrag die Rubriktexte leisten. Zur Beantwortung wird eine Ablation im Sinne von Abschnitt 2.2 durchgeführt: Aus einer funktionierenden Konfiguration wird gezielt eine Informationsquelle entfernt, während Serialisierung, Backbone, Pooling, Scoring-Kopf, Verlustfunktion und sämtliche Hyperparameter unverändert bleiben. Alle Bedingungen verwenden den festen Klassifikationskopf, sodass der Vergleich nicht durch die Fusionsarchitektur konfundiert wird.

Entscheidend für die Anlage des Experiments ist die in Abschnitt 4.1.1 dokumentierte Festlegung, dass die Hauptläufe dieser Arbeit *weder Frage noch Musterlösung* enthalten. Die Rubriktexte sind dort die einzige aufgabenbezogene Information; ihr Entzug lässt allein den Antworttext übrig. Eine Ablation unter dieser Bedingung kann daher nicht zwischen dem spezifischen

Beitrag der Kriterienbeschreibung und dem allgemeinen Beitrag *irgendeiner* Aufgabeninformation unterscheiden. Um beides zu trennen, wird die Ablation hier in einem vollständigen 2×2 -Schema durchgeführt, das zusätzlich die Verfügbarkeit von Frage und Musterlösung variiert.

Das vollständige Schema

Tabelle 5.3: SciEntsBank, Llama-3.2-1B, fester Klassifikationskopf: Verfügbarkeit von Aufgabenkontext (Frage und Musterlösung) und Rubriktext im vollständigen 2×2 -Schema. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.

Kontext	Rubriktext	UA	UQ	UD
ja	ja	0,769 (0,024)	0,630 (0,026)	0,617 (0,016)
ja	nein	0,782 (0,041)	0,615 (0,022)	0,611 (0,038)
nein	ja	0,682 (0,007)	0,555 (0,051)	0,534 (0,016)
nein	nein	0,447 (0,055)	0,035 (0,023)	0,172 (0,038)

Tabelle 5.3 führt zu einem klar strukturierten Befund, der die naheliegende Deutung der einfachen Ablation korrigiert.

Aufgabenbezogene Information ist unverzichtbar. Fehlt sie vollständig, bricht die Leistung zusammen: auf UQ von 0,555 auf 0,035 QWK, auf UD von 0,534 auf 0,172. Der Effekt beträgt $+0,235 / +0,520 / +0,361$ und übersteigt die Rauschgrenze um das Sechs- bis Vierzehnfache; die Spannweiten der beiden Bedingungen überlappen in keiner der drei Zellen.

Der Rubriktext trägt jedoch nichts bei, sobald Frage und Musterlösung vorliegen. Die Differenz zwischen den beiden oberen Zeilen beträgt $-0,012 / +0,015 / +0,006$ QWK. Alle drei Werte liegen unterhalb der Rauschgrenze, und in allen drei Fällen überlappen die Spannweiten. Nach der in Abschnitt 5.2 formulierten Regel ist ein Beitrag des Rubriktexts unter dieser Bedingung nicht nachweisbar. Unter Macro- F_1 gilt dies nur eingeschränkt: Dort trägt der Rubriktext bei vorhandenem Kontext auf UD nachweisbar bei (Abschnitt 5.12).

Frage und Musterlösung sind dem Rubriktext sogar überlegen. Der Vergleich der zweiten mit der dritten Zeile – Kontext ohne Rubrik gegen Rubrik ohne Kontext – ergibt $+0,100 / +0,060 / +0,077$ QWK zugunsten des Kontexts. Alle drei Werte übersteigen die Rauschgrenze. Umgekehrt bringt das Hinzufügen des Kontexts zu einer bereits vorhandenen Rubrik $+0,088 / +0,075 / +0,083$ QWK, ebenfalls durchgängig oberhalb der Rauschgrenze. Auf UA bestehen beide Befunde auch unter Macro- F_1 . Auf den Transfersplits beruht der Vorsprung des Kontexts dagegen vor allem darauf, dass grobe Fehlbewertungen seltener werden; unter Macro- F_1 ist er dort nicht oder nur als schwacher Hinweis nachweisbar (Abschnitt 5.12).

Bemerkenswert ist schließlich, dass die beste Einzelkonfiguration dieses Schemas – Kontext ohne Rubriktext – auf dem UA-Split mit 0,782 QWK im Mittel und 0,799 im besten Lauf den höchsten SciEntsBank-Wert der gesamten Arbeit erreicht, oberhalb sämtlicher Läufe mit Llama-3.2-3B und Mistral-7B. Ein 1,2-Milliarden-Modell mit Frage, Musterlösung und Antwort schlägt damit alle größeren Backbones, die stattdessen Rubriktexte erhalten.

Konsequenz für die Deutung

Der in der einfachen Ablation gemessene Effekt von bis zu 0,52 QWK ist demnach *nicht* als Beitrag der Kriterienbeschreibung zu lesen, sondern als Beitrag aufgabenbezogener Information überhaupt. Rubriktext einerseits sowie Frage und Musterlösung andererseits erweisen sich als weitgehend austauschbare Träger derselben Information – mit einem Vorteil für Letztere, der sich unter QWK auf allen Splits, unter Macro- F_1 nur auf UA zeigt. Dieser Befund ist konsistent mit der Herkunft der Rubriktexte: Für Beetle und SciEntsBank liegen keine originalen Bewertungsrichtlinien vor, die verwendeten Texte wurden sprachmodellgestützt aus Frage und Musterlösung erzeugt (Kapitel 3) und nehmen deren Inhalt erkennbar mit auf. Für die SciEntsBank-Frage *What is the difference between a rock and a mineral?* etwa hält die Beschreibung der höchsten Stufe fest, korrekte Antworten unterschieden zutreffend zwischen Gestein und Mineral und stellten dar, dass Gestein aus mehreren Mineralen bestehe, ein Mineral dagegen aus einer einzigen Substanz – also genau der Inhalt der zugehörigen Musterlösung.

Damit ist die Prämisse der rubrik-konditionierten Formulierung (2.5) empirisch eingeschränkt. Ihr Nutzen liegt nach diesen Daten nicht darin, dass Kriterienbeschreibungen eine der Aufgabenbeschreibung überlegene Informationsquelle wären. Er liegt in der strukturellen Eigenschaft, mit einer zur Laufzeit variablen Stufenzahl umgehen zu können (Abschnitt 4.1) und auf ungesehene Rubriksätze übertragbar zu sein – Eigenschaften, die ein fester Klassifikationskopf grundsätzlich nicht besitzt und die sich in einem Genauigkeitsvergleich auf festen Labelräumen auch nicht zeigen können.

Zwei Präzisierungen zum Zusammenbruch

Der Zusammenbruch ohne jede Aufgabeninformation ist kein degenerierter Klassifikator. Es wäre denkbar, dass das Modell auf die Vorhersage einer einzigen Stufe zusammenfällt und das niedrige QWK lediglich diesen Kollaps abbildet. Eine Prüfung der Vorhersagedateien widerlegt dies für alle sechs Läufe ohne jede Aufgabeninformation, die drei der untersten Tabellenzeile sowie die drei entsprechenden Läufe mit Llama-3.2-3B (Tabelle 5.4): Jeder von ihnen sagt sämtliche drei Stufen vorher, und der Anteil der häufigsten vorhergesagten Stufe übersteigt in keinem Fall 54 %.

Auf den Transfersplits liegt die Leistung dann auf oder unter dem Zufallsniveau. Ohne jede Aufgabeninformation erreichen die Modelle auf dem UQ-Split eine Accuracy zwischen 0,381 und 0,416 gegenüber einer Mehrheitsschranke von 0,411; in fünf der sechs Läufe liegt sie *unter* dieser Schranke. Auf UD ergibt sich mit 0,353 bis 0,440 gegenüber einer Schranke von 0,420 dasselbe Bild. Auf UA dagegen bleibt die Accuracy mit 0,574 bis 0,611 deutlich über der dortigen Schranke von 0,431. Der Kontrast ist aufschlussreich: Ohne Aufgabeninformation kann das Modell die Bewertungskonvention bekannter Fragen weiterhin teilweise reproduzieren – offenbar über sprachliche Regelmäßigkeiten der Antworten –, während es auf neuen Fragen und Domänen jede Grundlage verliert. Damit ist empirisch belegt, was in Abschnitt 2.1 als methodisches Argument eingeführt wurde: UA und UQ messen verschiedene Fähigkeiten.

Modellgröße und die Ablation

Die einfache Ablation ohne Kontext wurde zusätzlich auf Llama-3.2-3B durchgeführt (Tabelle 5.4). Der Effekt bleibt dort unverändert groß; auf UD fällt er mit 0,440 gegenüber 0,361 QWK nominell größer aus. Ob diese Zunahme mehr ist als Rauschen, lässt sich nicht entscheiden: Beide Modellgrößen wurden bei derselben, für sie nicht vorgesehenen Lernrate trainiert (Abschnitt 5.8). Sicher ist allein das Vorzeichen des Vergleichs *innerhalb* jeder Größe: Zusätzliche Modellkapazität kompensiert fehlende Aufgabeninformation nicht: Ohne sie liegen beide Modellgrößen gleichermaßen am Boden.

Tabelle 5.4: SciEntsBank: Ablation der Rubriktexte ohne Aufgabenkontext, über zwei Modellgrößen. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. *Lücke* bezeichnet den Abstand zwischen dem schlechtesten Lauf mit und dem besten Lauf ohne Rubriktext.

Backbone	Split	mit Rubriktext	ohne	Effekt	Lücke
Llama-3.2-1B	UA	0,682 (0,007)	0,447 (0,055)	+0,235	0,206
	UQ	0,555 (0,051)	0,035 (0,023)	+0,520	0,485
	UD	0,534 (0,016)	0,172 (0,038)	+0,361	0,338
Llama-3.2-3B	UA	0,746 (0,028)	0,491 (0,025)	+0,255	0,236
	UQ	0,580 (0,076)	0,052 (0,063)	+0,528	0,454
	UD	0,641 (0,026)	0,201 (0,029)	+0,440	0,406

Eine Ausnahme auf Beetle

Auf Beetle fällt die Ablation ohne Kontext anders aus als auf SciEntsBank. Auch dieser Vergleich beruht auf je drei Zufallssaaten derselben Umgebung.

Tabelle 5.5: Beetle, Llama-3.2-1B, fester Klassifikationskopf, ohne Aufgabenkontext: Ablation der Rubriktexte. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.

Split	mit Rubriktext	ohne	Effekt	Urteil
UA	0,641 (0,055)	0,587 (0,011)	+0,054	Spannweiten disjunkt
UQ	0,456 (0,068)	0,457 (0,037)	−0,001	im Rauschen

Auf UA liegen alle drei Läufe mit Rubriktext über allen drei Läufen ohne; die Differenz von 0,054 QWK bleibt allerdings knapp unter der Spannweite der rubrikhaltigen Bedingung und gilt nach der Auswertungsregel daher als Hinweis, nicht als gesicherter Effekt. Unter Macro- F_1 besteht derselbe Vergleich die Regel als Effekt (+0,040). Auf UQ ist der Unterschied mit −0,001 QWK exakt null, und beide Bedingungen liegen mit einer Accuracy von 0,581 bis 0,611 beziehungsweise 0,586 bis 0,602 klar über der Mehrheitsschranke von 0,420.

Auf Beetle generalisiert das Modell also auch ohne jede Aufgabeninformation auf ungesehene Fragen, während es auf SciEntsBank vollständig scheitert. Eine plausible Erklärung liegt in der Domänenstruktur: Beetle umfasst 47 Trainingsfragen aus einer einzigen, eng umgrenzten

Domäne (Elektrizitätslehre), deren neun UQ-Fragen inhaltlich sehr nahe an den Trainingsfragen liegen. SciEntsBank verteilt 135 Trainingsfragen über zwölf Themenblöcke (Kapitel 3). Je enger die Domäne, desto eher lässt sich eine Bewertungskonvention allein aus dem Antworttext von einer Frage auf die nächste übertragen.

Das Schema auf dem zweiten Benchmark

Um zu prüfen, ob der Befund zur Austauschbarkeit beider Informationsquellen auf SciEntsBank beschränkt ist, wurde das 2×2 -Schema auf Beetle wiederholt. Die beiden Bedingungen mit Aufgabenkontext liegen dort mit je einem Lauf vor und sind daher zurückhaltender zu bewerten als die SciEntsBank-Werte.

Tabelle 5.6: Beetle, Llama-3.2-1B, fester Klassifikationskopf: Verfügbarkeit von Aufgabenkontext und Rubriktext. Sämtliche Werte bei der Zufallssaat 114514, um die Zellen untereinander vergleichbar zu halten. Für die beiden unteren Zeilen liegen zusätzlich drei Saaten vor (Mittelwerte 0,641 / 0,456 beziehungsweise 0,587 / 0,457).

Kontext	Rubriktext	UA	UQ
ja	ja	0,764	0,554
ja	nein	0,799	0,602
nein	ja	0,669	0,443
nein	nein	0,588	0,445

Tabelle 5.6 bestätigt das Muster von SciEntsBank und verschärft es in einem Punkt. Der Aufgabenkontext wirkt auch hier deutlich: Er bringt +0,095 / +0,111 QWK bei vorhandener Rubrik und +0,211 / +0,157 ohne sie. Kontext allein übertrifft Rubrik allein um +0,131 / +0,158 und damit noch klarer als auf SciEntsBank.

Der Rubriktext dagegen bringt bei vorhandenem Kontext nicht nur nichts, sondern kostet: –0,036 auf UA und –0,047 auf UQ. Beide Werte liegen in der Größenordnung der Rauschgrenze und beruhen auf je einem Lauf, sind also nicht als gesicherter Schaden zu lesen. Festzuhalten ist aber, dass das Vorzeichen auf beiden Benchmarks gegen einen zusätzlichen Nutzen der Rubriktexte spricht, sobald die Aufgabe selbst beschrieben ist.

Abbildung 5.1 stellt beide Schemata nebeneinander. Unter QWK ist das Muster auf beiden Benchmarks dasselbe: Die zwei Bedingungen mit Aufgabenkontext sind auf keinem Split sicher voneinander zu unterscheiden – auf Beetle beruhen sie auf Einzelläufen, deren Abstände in der Größenordnung der Rauschgrenze liegen –, die Bedingung mit Rubriktext allein liegt durchgehend darunter, und ohne jede aufgabenbezogene Information bricht die Leistung auf den Transfersplits ein.

Auch auf Beetle ist damit – wie schon auf SciEntsBank – die stärkste der untersuchten kleinen Konfigurationen diejenige *ohne* Rubriktexte. Ein Llama-3.2-1B mit Frage, Musterlösung und Antwort erreicht auf UA 0,799 QWK und übertrifft damit sämtliche Läufe mit Llama-3.2-3B. Auf UQ erreicht dieselbe Konfiguration 0,602 QWK und liegt damit praktisch gleichauf mit Mistral-7B, das dort mit Rubriktexten und ohne Kontext 0,606 erzielt – bei etwa dem Sechsfachen an

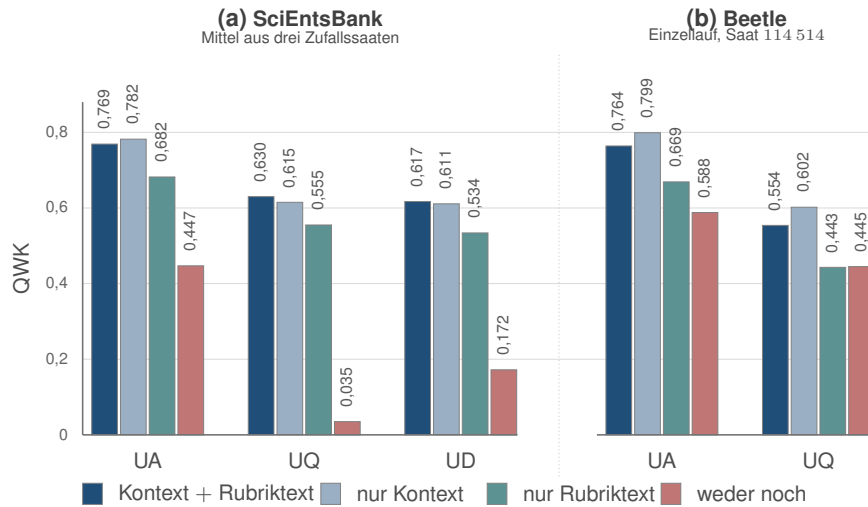


Abbildung 5.1: Verfügbarkeit von Aufgabenkontext (Frage und Musterlösung) und Rubriktext, Llama-3.2-1B mit festem Klassifikationskopf. Die beiden Bedingungen mit Kontext sind unter QWK auf keinem der fünf Splits sicher zu unterscheiden (unter Macro- F_1 trägt der Rubriktext auf SciEntsBank-UD bei, vgl. Abschnitt 5.12); ohne jede aufgabenbezogene Information bricht die Leistung auf den Transfersplits zusammen. Werte aus den Tabellen 5.3 und 5.6.

Parametern. Höher liegt unter den beiden kleinen Backbones auf UQ allein der in Abschnitt 5.8 berichtete Lauf mit abweichender Lernrate (0,605).

Eine Einschränkung bleibt für beide Benchmarks bestehen: Frage und Musterlösung werden in der verwendeten Implementierung gemeinsam zu- oder abgeschaltet. Ob der beobachtete Kontext-Effekt auf die Aufgabenstellung, auf die Musterlösung oder auf beide zurückgeht, lässt sich mit den vorliegenden Läufen nicht entscheiden und bleibt offen (Kapitel 7).

Einordnung

Dieser Abschnitt liefert den einzigen großen Effekt dieser Arbeit – und zugleich eine Einschränkung ihrer Ausgangsannahme. Entscheidend ist, dass dem Modell aufgabenbezogene Information zur Verfügung steht, nicht, in welcher Form. Wie diese Information anschließend extrahiert und verrechnet wird, ist Gegenstand der folgenden drei Abschnitte. Keine der dort untersuchten Varianten erzeugt einen Unterschied oberhalb der Rauschgrenze.

5.4 Fusionsmechanismen

Einzelläufe je Benchmark

Die Tabellen 5.7 bis 5.9 dokumentieren die Ergebnisse der Fusionsvarianten, auf ASAP-SAS als Einzelläufe, auf Beetle und SciEntsBank als Mittelwerte über drei Zufallssaaten. DIFF wur-

de ausschließlich auf Llama-3.2-3B untersucht. Die Tabellen dienen der Vollständigkeit; die Interpretation erfolgt anschließend auf Grundlage der Mittelwerte über drei Zufallssaaten.

Tabelle 5.7: ASAP-SAS (UA): Fusionsmechanismen bei kausaler Attention. Einzelläufe, Zufallssaat 114514.

	Llama-3.2-1B			Llama-3.2-3B			Mistral-7B		
	QWK	F_1	Acc	QWK	F_1	Acc	QWK	F_1	Acc
CONDIFF	0,818	0,729	0,767	0,838	0,762	0,786	0,860	0,803	0,809
CONCAT	0,828	0,750	0,781	0,842	0,769	0,796	0,855	0,788	0,800
GATE	0,829	0,750	0,777	0,841	0,765	0,789	0,858	0,790	0,801

Tabelle 5.8: Beetle und SciEntsBank: Fusionsmechanismen bei kausaler Attention, QWK je Split. Mittelwerte über drei Zufallssaaten; für Mistral-7B liegt nur CONDIFF mit einer Saat vor.

Benchmark	Split	Llama-3.2-1B		Llama-3.2-3B		Mistral-7B
		CONDIFF	CONCAT	CONDIFF	CONCAT	CONDIFF
Beetle	UA	0,729	0,725	0,730	0,735	0,803
	UQ	0,509	0,503	0,544	0,546	0,606
SciEntsBank	UA	0,702	0,698	0,734	0,720	0,743
	UQ	0,544	0,569	0,576	0,582	0,621
	UD	0,560	0,577	0,617	0,626	0,635

Tabelle 5.9: Beetle und SciEntsBank: GATE- und DIFF-Fusion, QWK je Split. Mittelwerte über drei Zufallssaaten. DIFF liegt nur für Llama-3.2-3B vor.

Benchmark	Split	Llama-3.2-1B	Llama-3.2-3B	
		GATE	GATE	DIFF
Beetle	UA	0,696	0,757	0,740
	UQ	0,471	0,550	0,572
SciEntsBank	UA	0,718	0,737	0,722
	UQ	0,564	0,624	0,579
	UD	0,559	0,630	0,623

Der Unterschied zwischen den Varianten liegt unterhalb der Rauschgrenze

Tabelle 5.10 stellt den Unterschied *zwischen* den Fusionsmechanismen der Streuung *innerhalb* einer Fusionsvariante gegenüber. In allen zehn Konfigurationen übersteigt die Seed-Streuung den Abstand zwischen den Varianten, im Extremfall um mehr als das Dreifache. Nach der in Abschnitt 5.2 formulierten Regel ist ein Unterschied zwischen den Fusionsvarianten auf dieser Datengrundlage folglich *nicht als Effekt nachweisbar*.

Tabelle 5.10: Unterschied zwischen den Fusionsmechanismen (Spannweite der Mittelwerte von CONDIFF, CONCAT und GATE, bei Llama-3.2-3B zusätzlich DIFF) gegenüber der größten Seed-Spannweite derselben Zellen.

Benchmark	Backbone	Split	zw. Varianten	max. Seed- Δ	Verhältnis
Beetle	Llama-1B	UA	0,033	0,051	1,5×
Beetle	Llama-1B	UQ	0,038	0,118	3,1×
Beetle	Llama-3B	UA	0,027	0,042	1,6×
Beetle	Llama-3B	UQ	0,028	0,048	1,7×
SciEntsBank	Llama-1B	UA	0,020	0,039	1,9×
SciEntsBank	Llama-1B	UQ	0,025	0,058	2,3×
SciEntsBank	Llama-1B	UD	0,017	0,047	2,7×
SciEntsBank	Llama-3B	UA	0,016	0,047	2,8×
SciEntsBank	Llama-3B	UQ	0,049	0,060	1,2×
SciEntsBank	Llama-3B	UD	0,013	0,035	2,7×

Drei Paarvergleiche erfüllen allerdings das schwächere Kriterium: GATE gegenüber CONDIFF auf Beetle-UA und auf SciEntsBank-UQ sowie GATE gegenüber DIFF auf Beetle-UA, sämtlich mit Llama-3.2-3B. Dort überlappen die Spannweiten der beiden Bedingungen nicht, während die Differenz der Mittelwerte (0,027, 0,049 und 0,017 QWK) jeweils unter der größeren der beiden Spannweiten bleibt. Nach der Auswertungsregel ist dies ein schwacher Hinweis, kein Ergebnis. Unter Macro- F_1 verschieben sich acht Paarurteile zwischen diesen Kategorien; das Gesamtbild bleibt davon unberührt (Abschnitt 5.12).

Die Konsequenz reicht weiter, als es zunächst scheint. Tabelle 5.11 zeigt, welche Variante in welcher Zelle den höchsten Einzelwert erreicht.

Tabelle 5.11: Beste Fusionsvariante je Benchmark, Split und Backbone nach dem Mittelwert über drei Zufallssaaten. Die Rangfolge ist nach Tabelle 5.10 nicht belastbar und wird hier ausschließlich zur Illustration ihrer Beliebigkeit angegeben.

Benchmark	Split	Llama-1B	Llama-3B
Beetle	UA	CONDIFF	GATE
	UQ	CONDIFF	DIFF
SciEntsBank	UA	GATE	GATE
	UQ	CONCAT	GATE
	UD	CONCAT	GATE

Beim kleineren Backbone wechselt die Rangfolge zwischen Benchmarks und Splits ohne erkennbares Muster – genau das zu erwartende Bild, wenn die zugrunde liegenden Unterschiede tatsächlich Null sind und die Reihenfolge allein durch die Zufallssaat bestimmt wird. Beim größeren Backbone liegt dagegen in vier von fünf Zellen GATE vorn; in der fünften, Beetle-UQ, führt DIFF. Dieses Bild ist mit den drei oben genannten disjunkten Spannweiten konsistent, die sämtlich GATE betreffen, und es ist zurückhaltend zu bewerten: Keine der Differenzen übersteigt die Spannweite der beteiligten Bedingungen, und die fünf Zellen sind nicht unabhängig

voneinander, da sie auf denselben Läufen beruhen und lediglich nach Split getrennt ausgewertet werden.

Festzuhalten bleibt, dass dieser Hinweis der in Abschnitt 4.4.1 formulierten Erwartung *entgegenläuft*: Dort war vermutet worden, ein parameterreicher Mechanismus wie GATE falle bei kleineren Backbones relativ stärker ins Gewicht. Die nominelle Verteilung weist eher in die Gegenrichtung: Beim kleineren Backbone liegt GATE in einer von fünf Zellen vorn, beim größeren in vier. Eine Deutung einzelner Rangfolgen – etwa dass der Differenzterm von CONDIFF der Generalisierung zugutekomme – ist auf dieser Grundlage gleichwohl nicht zu rechtfertigen. Solche Deutungen wären an Einzelläufen durchaus plausibel gewesen; sie halten der Wiederholung unter mehreren Zufallssaaten nicht stand. Ob GATE bei größeren Backbones tatsächlich gewinnt, wäre durch Wiederholung mit weiteren Zufallssaaten zu prüfen.

Für die Fragestellung dieser Arbeit ist das ein aussagekräftiges negatives Ergebnis: Der parameterfreie Differenzterm von CONDIFF (4.6) bringt gegenüber der reinen Konkatenation keinen nachweisbaren Vorteil, und umgekehrt tragen die unveränderten Einzelrepräsentationen gegenüber dem Differenzterm allein nichts bei: Auf Llama-3.2-3B liegt DIFF (4.8) in vier von fünf Zellen nominell vor CONDIFF, durchweg im Rauschen. Die d^2 zusätzlichen Parameter der Gate-Projektion (4.9) rechtfertigen ihren Aufwand nach der Auswertungsregel ebenfalls nicht. Da CONCAT und DIFF die einfachsten Konstruktionen sind – DIFF mit dem kleinsten Scoring-Kopf –, folgt daraus eine praktische Empfehlung zugunsten einer einfachen Konstruktion; für DIFF stützt sie sich allerdings allein auf Llama-3.2-3B.

5.5 Bezugsgröße der Fusion

Die zweite Frage zur Ausgestaltung der Fusion betrifft die Wahl der Kontextrepräsentation in (2.19). Die vorliegende Arbeit verwendet durchgängig die Sequenzrepräsentation z ; Gombert u. a. richten stattdessen die Repräsentation des *Antwort*-Spans an den Rubrik-Spans aus [24]. Zur Prüfung wurde die Variante „t-concat“ untersucht, die sich von CONCAT ausschließlich durch die Bezugsgröße unterscheidet: Statt $[z \parallel r_k]$ bildet sie $[t \parallel r_k]$, wobei t die nach demselben Verfahren gepoolte Repräsentation des Antwort-Spans bezeichnet (Abschnitt 4.1.2).

Tabelle 5.12: Bezugsgröße der Fusion auf Llama-3.2-3B: Antwort-Span (t-concat) gegenüber Sequenzrepräsentation (CONCAT, CONDIFF). Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.

Benchmark	Split	t-concat	CONCAT	CONDIFF	Δ zu CONCAT
Beetle	UA	0,761 (0,006)	0,735 (0,042)	0,730 (0,041)	+0,026
	UQ	0,552 (0,019)	0,546 (0,046)	0,544 (0,044)	+0,006
SciEntsBank	UA	0,734 (0,066)	0,720 (0,042)	0,734 (0,047)	+0,013
	UQ	0,584 (0,059)	0,582 (0,049)	0,576 (0,009)	+0,002
	UD	0,615 (0,019)	0,626 (0,035)	0,617 (0,028)	−0,011

Tabelle 5.12 zeigt in vier der fünf Zellen vollständig überlappende Spannweiten bei Differenzen zwischen 0,002 und 0,013 QWK. Die einzige Ausnahme ist Beetle-UA: Dort überlappen die

Spannweiten von t-concat und CONCAT nicht, die Lücke beträgt jedoch lediglich 0,007 QWK, und die Differenz der Mittelwerte (0,026) bleibt unter der Spannweite von CONCAT (0,042). Nach der Auswertungsregel ist dies ein schwacher Hinweis, kein Effekt. Unter Macro- F_1 besteht derselbe Vergleich die Regel knapp als Effekt (+0,014); die Differenz bleibt aber klein (Abschnitt 5.12).

Dieses Ergebnis verdient eine methodische Anmerkung, weil es die Notwendigkeit der Wiederholung besonders anschaulich belegt. Läge nur der Lauf mit der Zufallssaat 114514 vor, ergäbe sich ein deutlich anderes Bild: t-concat erreichte dort auf SciEntsBank-UA 0,762 gegenüber einem CONCAT-Bereich von 0,701 bis 0,743 und auf SciEntsBank-UQ 0,601 gegenüber 0,550 bis 0,598 – in beiden Fällen oberhalb sämtlicher Vergleichsläufe. Auch der Lauf mit der Saat 1 liegt mit 0,744 und 0,604 knapp oberhalb dieser Bereiche. Der Lauf mit der Saat 2 fällt dagegen auf 0,696 beziehungsweise 0,545 und damit unter den jeweils schwächsten CONCAT-Lauf. Im Mittel bleibt dadurch ein Vorsprung von nur 0,013 und 0,002 QWK, der vollständig innerhalb der Spannweiten liegt. Zwei von drei Ziehungen hätten als Einzellauf einen Vorteil nahegelegt, den die dritte aufhebt.

Für die Arbeit folgt daraus: Ob als Bezugsgröße der Antwort-Span oder die Sequenzrepräsentation dient, macht keinen Unterschied oberhalb der Rauschgrenze. Das ist insofern bemerkenswert, als beide Repräsentationen unterschiedliche Informationen tragen – t endet vor den Rubriktexten und hat diese unter kausaler Maske nicht gesehen, z steht am Sequenzende und hat sie alle gesehen (Abschnitt 4.1.2). Dass dieser Unterschied folgenlos bleibt, deutet darauf hin, dass die für die Bewertung ausschlaggebende Information bereits in der Antwortrepräsentation enthalten ist und die zusätzliche Kenntnis der übrigen Stufen keinen verwertbaren Beitrag leistet.

5.6 Span-Alignment gegenüber einem festen Klassifikationskopf

Die dritte Frage zur Architektur betrifft den Vergleich mit der einfachsten denkbaren Alternative: einem Modell, das die Rubriktexte zwar in der Eingabe sieht, sie aber nicht als Spans extrahiert, sondern die Bewertung über einen festen Kopf mit C Ausgaben aus der Sequenzrepräsentation ableitet. Der Unterschied zum Span-Alignment liegt damit ausschließlich im Scoring-Mechanismus.

Tabelle 5.13 zeigt ein zweigeteiltes Bild. Beim kleineren Backbone liegt das Span-Alignment auf UA und UD vorn; die Vorteile betragen 0,021 und 0,026 QWK. Beide liegen *unterhalb* des globalen Medians der Seed-Spannweite von 0,037 QWK und sind allein deshalb nachweisbar, weil die betroffenen Zellen ungewöhnlich eng streuen (0,007 auf UA). Auf UD ist die Entscheidung zudem denkbar knapp: Die Differenz übersteigt die maßgebliche Spannweite nur um 0,0003 QWK. Beim größeren Backbone liegen alle drei Differenzen im Rauschen, und alle drei sind nominell negativ.

Der Kontrast zwischen beiden Modellgrößen ist mit Vorsicht zu lesen. Beide Llama-Varianten wurden bei $5 \cdot 10^{-5}$ statt bei der für sie vorgesehenen Rate $2 \cdot 10^{-4}$ trainiert (Abschnitt 5.8); ob sie dadurch in gleichem Maß untertrainiert sind, ist nicht geprüft. Der Vergleich *innerhalb* jeder Modellgröße bleibt davon unberührt, die Aussage über das Verhältnis beider Größen zueinander nicht.

Tabelle 5.13: SciEntsBank: Span-Alignment (CONDIFF) gegenüber festem Klassifikationskopf. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. *Über Spannweite* bedeutet, dass die Differenz die größere der beiden zellspezifischen Spannweiten übersteigt; sämtliche Differenzen dieser Tabelle liegen unterhalb des globalen Medians von 0,037 QWK.

Backbone	Split	Span-Alignment	fester Kopf	Δ	Urteil
Llama-3.2-1B	UA	0,702 (0,007)	0,682 (0,007)	+0,021	über Spannweite
	UQ	0,544 (0,058)	0,555 (0,051)	−0,011	im Rauschen
	UD	0,560 (0,026)	0,534 (0,016)	+0,026	über Spannweite (knapp)
Llama-3.2-3B	UA	0,734 (0,047)	0,746 (0,028)	−0,011	im Rauschen
	UQ	0,576 (0,009)	0,580 (0,076)	−0,004	im Rauschen
	UD	0,617 (0,028)	0,641 (0,026)	−0,024	im Rauschen

Bei der Deutung ist entscheidend, dass beide Bedingungen die Rubriktexte in der Eingabe enthalten. Der Vergleich misst also nicht den Wert der Rubrikinformation – dieser ist Gegenstand von Abschnitt 5.3 und beträgt ein Vielfaches –, sondern lediglich den Wert ihrer expliziten Extraktion als Span. Dass dieser Zusatznutzen klein ist, steht fest; dass er beim größeren Backbone nicht mehr nachweisbar ist, legt die Lesart nahe, ein hinreichend großes Sprachmodell verdichte die relevante Rubrikinformation bereits in der Sequenzrepräsentation, sodass das gezielte Auslesen der einzelnen Stufen keinen weiteren Beitrag liefere. Diese Lesart ist mit den vorliegenden zwei Modellgrößen und der oben genannten Einschränkung zur Lernrate nicht zu sichern.

Der praktische Wert der Span-Architektur liegt damit weniger in einer höheren Genauigkeit als in ihrer strukturellen Eigenschaft, mit einer zur Laufzeit variablen Stufenzahl umgehen zu können (Abschnitt 4.1) – eine Eigenschaft, die dem festen Kopf grundsätzlich fehlt und die auf Benchmarks wie ASAP-SAS mit wechselnder Skalenlänge erst die gemeinsame Modellierung aller Aufgaben ermöglicht.

5.7 Attention-Maske

Die vierte Achse ist die einzige, auf der eine Architekturvariante die Rauschgrenze deutlich überschreitet. Sie wird in zwei Schritten ausgewertet: zunächst die anteilige Aufhebung der Kausalität auf Einzelläufen, anschließend die vollständige Aufhebung unter drei Zufallssaaten auf beiden Llama-Größen.

Anteilige Aufhebung

Über die Einzelläufe hinweg zeigt sich *kein* monotoner Zusammenhang zwischen der Zahl bidirektionaler Schichten und der Leistung. Die stärkste Bedingung ist je nach Zelle die kausale Referenz, eine einzelne bidirektionale Schicht oder eine mittlere Schichtzahl. Da sämtliche Werte dieser beiden Tabellen auf Einzelläufen beruhen und die beobachteten Abstände weit

Tabelle 5.14: ASAP-SAS (UA): Attention-Maske bei CONDIFF-Fusion, sortiert nach der tatsächlichen Zahl bidirektionaler Schichten n_{bidir} . Einzelläufe, Zufallssaat 114514. Die vollständig bidirektionale Bedingung liegt für Mistral-7B nicht vor.

ρ	Llama-3.2-1B ($L = 16$)			Llama-3.2-3B ($L = 28$)			Mistral-7B ($L = 32$)		
	n_{bidir}	QWK	F_1	n_{bidir}	QWK	F_1	n_{bidir}	QWK	F_1
0 (kausal)	0	0,818	0,729	0	0,838	0,762	0	0,860	0,803
1,0	1	0,821	0,743	1	0,843	0,773	1	0,849	0,782
0,2	3	0,822	0,745	5	0,836	0,764	6	0,851	0,784
0,4	6	0,822	0,738	11	0,834	0,765	12	0,845	0,783
0,6	9	0,819	0,744	16	0,834	0,765	19	0,855	0,792
0,8	12	0,823	0,748	22	0,839	0,767	25	0,847	0,787
L (vollst.)	16	0,825	0,754	28	0,840	0,770	32	—	—

Tabelle 5.15: Beetle und SciEntsBank: Attention-Maske bei CONDIFF-Fusion, QWK je Split, sortiert nach n_{bidir} . Einzelläufe, Zufallssaat 114514. Für Llama-3.2-3B und Mistral-7B liegen dieselben Bedingungen ebenfalls als Einzelläufe vor; sie werden hier nicht aufgeführt, da sie zu keiner Aussage beitragen, die nicht schon aus der vollständigen Aufhebung folgt.

ρ	Beetle, Llama-3.2-1B			SciEntsBank, Llama-3.2-1B			
	n_{bidir}	UA	UQ	n_{bidir}	UA	UQ	UD
0 (kausal)	0	0,696	0,532	0	0,705	0,582	0,572
1,0	1	0,707	0,544	1	0,691	0,547	0,558
0,2	3	0,711	0,526	3	0,697	0,551	0,555
0,4	6	0,691	0,472	6	0,683	0,571	0,561
0,6	9	0,724	0,503	9	0,681	0,533	0,541
0,8	12	0,750	0,413	12	0,704	0,567	0,589

überwiegend unterhalb der Rauschgrenze von 0,037 QWK liegen, sind sie einzeln nicht interpretierbar. Auch die vollständig bidirektionale Bedingung weicht auf ASAP-SAS mit +0,006 und +0,003 QWK kaum von der kausalen Referenz ab.

Eine Beobachtung überschreitet diese Schwelle: Bei Llama-3.2-1B auf Beetle steigt der UA-Wert über den Bereich von null bis zwölf bidirektionalen Schichten um 0,053 QWK, während der UQ-Wert im selben Bereich um 0,119 fällt. Diese gegenläufige Entwicklung ist deutlich genug, um bemerkenswert zu sein, beruht jedoch auf je einem Lauf und bedürfte der Absicherung durch Wiederholung.

Vollständige Aufhebung

Die aussagekräftigere Bedingung ist die vollständige Aufhebung der Kausalität in allen Schichten. Sie wurde für Llama-3.2-3B (28 Schichten) und Llama-3.2-1B (16 Schichten) unter je drei Zufallssaaten ausgeführt und der kausalen CONDIFF-Referenz desselben Backbones gegenübergestellt. Tabelle 5.16 berichtet zunächst das größere Backbone.

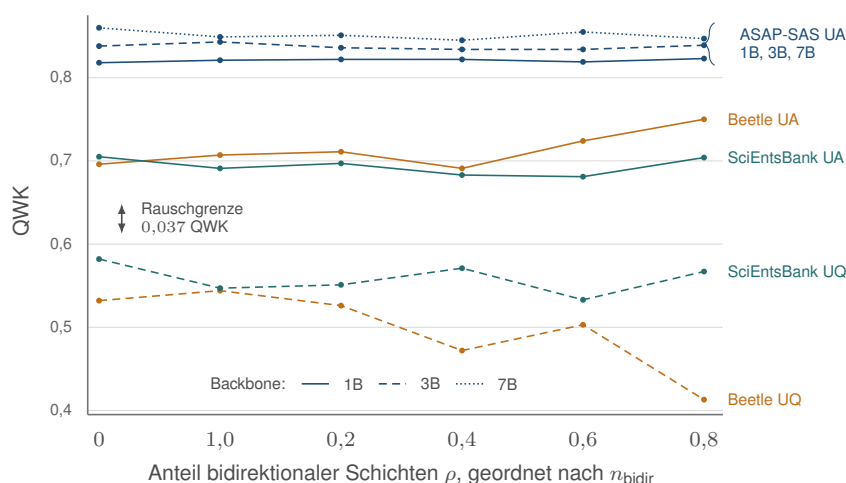


Abbildung 5.2: QWK in Abhängigkeit vom Anteil bidirektionaler Schichten, CONDIFF-Fusion, Einzeläufe bei Zufallssaat 114 514. Die Bedingungen sind nach der tatsächlichen Schichtzahl n_{bidir} geordnet, weshalb $\rho = 1,0$ (genau eine Schicht) vor $\rho = 0,2$ steht. Ein monotoner Zusammenhang besteht nicht; nahezu alle Abstände bleiben unterhalb der eingezeichneten Rauschgrenze. Die Ausnahme ist der gegenläufige Verlauf von Beetle UA und UQ. Werte aus den Tabellen 5.14 und 5.15.

Tabelle 5.16: Vollständig bidirektionale Attention ($n_{\text{bidir}} = L = 28$) gegenüber kausaler Attention, Llama-3.2-3B, CONDIFF-Fusion. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.

Benchmark	Split	bidirektional	kausal	Δ	Urteil
Beetle	UA	0,657 (0,061)	0,730 (0,041)	-0,073	über Rauschgrenze
	UQ	0,358 (0,094)	0,544 (0,044)	-0,186	weit über Rauschgrenze
SciEntsBank	UA	0,767 (0,024)	0,734 (0,047)	+0,032	Spannweiten disjunkt
	UQ	0,636 (0,026)	0,576 (0,009)	+0,060	über Rauschgrenze
	UD	0,625 (0,036)	0,617 (0,028)	+0,008	im Rauschen

Tabelle 5.16 enthält für Llama-3.2-3B den deutlichsten Architektureffekt dieser Arbeit – und zugleich den einzigen, der zwischen Benchmarks das Vorzeichen wechselt.

Auf Beetle schadet die vollständige Aufhebung erheblich. Der Verlust beträgt 0,073 QWK auf UA und 0,186 auf UQ. Auf beiden Splits überlappen die Spannweiten nicht; auf UQ liegt der beste bidirektionale Lauf (0,419) noch um 0,109 unter dem schlechtesten kausalen (0,528). Bemerkenswert ist zudem die erhöhte Streuung: Die Spannweite steigt von 0,044 auf 0,094, das Training wird also nicht nur schlechter, sondern auch instabiler.

Auf SciEntsBank nützt sie. Der Gewinn beträgt 0,060 QWK auf UQ – mit disjunkten Spannweiten und einer Lücke von 0,043 – sowie 0,032 auf UA, wo die Spannweiten ebenfalls nicht überlappen, die Differenz der Mittelwerte jedoch unter der Spannweite der kausalen Referenz bleibt. Auf UD ist kein Unterschied nachweisbar.

Unter Macro- F_1 bestehen die beiden UQ-Befunde, der Verlust auf Beetle und der Gewinn auf SciEntsBank, unverändert als Effekt. Der Verlust auf Beetle-UA und der Hinweis auf

SciEntsBank-UA sind dort dagegen nicht nachweisbar (Abschnitt 5.12).

Auf dem kleineren Backbone ergibt sich ein anderes Bild. Tabelle 5.17 stellt dieselbe Bedingung für Llama-3.2-1B dar, ebenfalls unter drei Zufallssaaten.

Tabelle 5.17: Vollständig bidirektionale Attention ($n_{\text{bidir}} = L = 16$) gegenüber kausaler Attention, Llama-3.2-1B, CONDIFF-Fusion. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. Über Spannweite wie in Tabelle 5.13.

Benchmark	Split	bidirektional	kausal	Δ	Urteil
Beetle	UA	0,765 (0,017)	0,729 (0,023)	+0,036	über Spannweite im Rauschen
	UQ	0,493 (0,058)	0,509 (0,073)	−0,016	
SciEntsBank	UA	0,692 (0,049)	0,702 (0,007)	−0,010	im Rauschen
	UQ	0,553 (0,087)	0,544 (0,058)	+0,009	im Rauschen
	UD	0,588 (0,046)	0,560 (0,026)	+0,028	im Rauschen

Auf SciEntsBank ist beim kleineren Backbone auf keinem der drei Splits ein Unterschied nachweisbar; sämtliche Spannweiten überlappen. Aufschlussreich ist dabei die Rolle der einzelnen Saaten: Der Lauf mit der Saat 114 514 ist auf allen drei Splits der schwächste der drei und hätte als Einzellauf gegenüber der kausalen Referenz derselben Saat einen Verlust von bis zu 0,084 QWK nahegelegt. Wie beim Vorsprung von t-concat in Abschnitt 5.5 erweist sich ein zunächst deutliches Muster erst unter Wiederholung als Artefakt einer einzelnen Ziehung.

Auf Beetle kehrt sich das Vorzeichen dagegen um. Auf UA gewinnt das kleinere Backbone 0,036 QWK; die Spannweiten sind disjunkt, und die Differenz übersteigt die größere von ihnen (0,023), sodass nach der Auswertungsregel ein Effekt vorliegt. Er liegt allerdings knapp unterhalb des globalen Medians von 0,037 und besteht die Regel nur, weil beide Bedingungen ungewöhnlich eng streuen. Auf UQ, wo das größere Backbone 0,186 QWK verliert, ist beim kleineren unter QWK kein Unterschied nachweisbar. Unter Macro- F_1 verliert allerdings auch das kleinere Backbone auf Beetle-UQ (−0,041), und dieser Verlust besteht die Auswertungsregel; der Gewinn auf UA bleibt unter beiden Metriken bestehen.

Der Kontrast zwischen beiden Benchmarks beim größeren Backbone ist erklärungsbedürftig, da sie dieselbe Aufgabenstruktur, dieselbe dreistufige Skala und dasselbe Backbone verwenden. Eine naheliegende Hypothese knüpft an die Datenstruktur an (Kapitel 3): Beetle umfasst 3 941 Trainingsinstanzen zu 47 Fragen aus einer einzigen eng umgrenzten Domäne, SciEntsBank 4 969 Instanzen zu 135 Fragen aus zwölf Themenblöcken. Die vollständige Aufhebung der Kausalität entfernt die Bedingung, unter der das Backbone vortrainiert wurde (Abschnitt 2.6.2); das Modell muss die Nutzung des beidseitigen Kontexts im Finetuning erst erlernen. Eine schmale und homogene Datengrundlage bietet dafür offenbar zu wenig Material, eine breitere genügt. Diese Deutung ist mit den vorliegenden Daten nicht prüfbar; sie ließe sich durch eine Variation des Trainingsumfangs bei festgehaltener Maske testen. Mit dem Befund für das kleinere Backbone ist sie in dieser Form zudem nicht vereinbar: Dieselben Beetle-Daten genügen Llama-3.2-1B, um aus der vollständigen Aufhebung auf UA sogar Nutzen zu ziehen. Die Wirkung der Maske hängt demnach nicht allein von der Datengrundlage ab, sondern auch vom Backbone.

Festzuhalten bleibt in jedem Fall der methodische Kern: Die Wirkung der Attention-Maske

ist weder über Benchmarks noch über Backbones hinweg generalisierbar. Eine Empfehlung, Decoder-Modelle für ASAG grundsätzlich bidirektional zu betreiben, lässt sich aus diesen Daten ebenso wenig ableiten wie das Gegenteil.

5.8 Effekt der Modellgröße

Tabelle 5.18: Effekt der Backbone-Größe bei CONDIFF und kausaler Attention (QWK). Für Llama-3.2-1B und -3B Mittelwerte über drei Zufallssaaten, für Mistral-7B Einzelläufe.

Benchmark	Split	Llama-1B	Llama-3B	Mistral-7B
ASAP-SAS	UA	0,818	0,838	0,860
Beetle	UA	0,729	0,730	0,803
	UQ	0,509	0,544	0,606
SciEntsBank	UA	0,702	0,734	0,743
	UQ	0,544	0,576	0,621
	UD	0,560	0,617	0,635

Dem Augenschein nach ist der Größeneffekt der stabilste Befund dieser Arbeit (Tabelle 5.18). Von Llama-3.2-1B zu Mistral-7B beträgt der Gewinn auf Beetle 0,073 QWK auf UA und 0,097 auf UQ, auf SciEntsBank 0,041 / 0,077 / 0,075 und auf ASAP-SAS 0,042; alle sechs Zellen lägen damit oberhalb der Rauschgrenze. Eine ergänzende Prüfung zeigt jedoch, dass dieser Eindruck nicht trägt.

Der Vergleich ist mit der Lernrate konfundiert

Sämtliche Decoder-Hauptläufe dieser Arbeit verwenden die Lernrate $5 \cdot 10^{-5}$ (Abschnitt 4.6). Die Referenzimplementierung sieht diesen Wert jedoch erst für Modelle ab sieben Milliarden Parametern vor; für die beiden Llama-Backbones nennt sie $2 \cdot 10^{-4}$. Das größte Backbone wurde also bei seiner vorgesehenen Rate trainiert, die beiden kleineren bei einem Viertel der ihren.

Zur Abschätzung der Folgen wurde CONDIFF auf Llama-3.2-1B zusätzlich bei $2 \cdot 10^{-4}$ trainiert und den drei vorhandenen Läufen bei $5 \cdot 10^{-5}$ gegenübergestellt (Tabelle 5.19).

Der Befund ist zweigeteilt und für die Auswertung dieser Achse entscheidend.

Auf Beetle war das kleinste Backbone erheblich untertrainiert. Bei der Referenzlernrate erreicht es 0,779 QWK auf UA und 0,605 auf UQ – in beiden Fällen oberhalb sämtlicher Läufe bei $5 \cdot 10^{-5}$. Der Zuwachs gegenüber dem Mittelwert beträgt 0,050 beziehungsweise 0,096 QWK. Damit schrumpft der in Tabelle 5.18 ausgewiesene Abstand zwischen Llama-3.2-1B und Mistral-7B auf Beetle von 0,073 auf 0,023 QWK (UA) und von 0,097 auf 0,001 (UQ). Auf dem Transfersplit verschwindet der Größeneffekt vollständig.

Tabelle 5.19: Llama-3.2-1B mit CONDIFF bei der Referenzlernrate $2 \cdot 10^{-4}$ gegenüber den Läufen bei $5 \cdot 10^{-5}$ (Mittelwert und Spanne über drei Zufallssaaten). Der Lauf bei $2 \cdot 10^{-4}$ beruht auf einer einzelnen Zufallssaat.

Benchmark	Split	$2 \cdot 10^{-4}$	$5 \cdot 10^{-5}$	Lage zur Spanne
Beetle	UA	0,779	0,729 (0,718–0,741)	darüber (+0,038)
	UQ	0,605	0,509 (0,472–0,545)	darüber (+0,060)
SciEntsBank	UA	0,679	0,702 (0,698–0,705)	darunter (–0,019)
	UQ	0,537	0,544 (0,524–0,582)	innerhalb
	UD	0,496	0,560 (0,546–0,572)	darunter (–0,050)

Auf SciEntsBank wirkt dieselbe Änderung entgegengesetzt. Dort liegt der Lauf bei $2 \cdot 10^{-4}$ auf zwei von drei Splits unterhalb der gesamten Spanne bei $5 \cdot 10^{-5}$, auf UD um 0,050 QWK. Es gibt folglich keine über beide Benchmarks hinweg günstigere Lernrate; sie interagiert mit dem Datensatz.

Konsequenz

Aus beidem folgt, dass Tabelle 5.18 keine Aussage über den Einfluss der Modellgröße erlaubt. Die drei Backbones unterscheiden sich in den vorliegenden Läufen nicht nur in ihrer Größe, sondern zugleich darin, wie weit ihre Lernrate von der für sie vorgesehenen abweicht – und das Ausmaß dieser Abweichung ist auf Beetle groß genug, um den gesamten beobachteten Unterschied zu erklären. Eine belastbare Untersuchung dieser Achse erforderte, jedes Backbone bei seiner jeweils geeigneten Lernrate zu trainieren; das war im Rahmen dieser Arbeit nicht zu leisten.

Die vorliegende Arbeit trifft daher *keine* Aussage über den Effekt der Backbone-Größe. Tabelle 5.18 wird als Dokumentation der durchgeführten Läufe beibehalten und ist ausdrücklich nicht als Größenvergleich zu lesen.

Zwei Einschränkungen betreffen auch diesen Befund selbst: Er beruht auf je einem Lauf bei $2 \cdot 10^{-4}$, und er wurde nur für Llama-3.2-1B erhoben. Für Llama-3.2-3B ist offen, wie stark die Lernrate dort wirkt.

Unberührt bleiben sämtliche Vergleiche *innerhalb* eines Backbones, in denen die Lernrate durchgängig identisch ist – und damit die Befunde zu Fusionsmechanismus, Bezugsgröße und Attention-Maske. Betroffen sind dagegen die wenigen Stellen, an denen dieses Kapitel Effektgrößen *zwischen* zwei Modellgrößen vergleicht: die Zunahme des Ablationseffekts von 1B auf 3B (Abschnitt 5.3) das Verschwinden des Span-Vorteils beim größeren Backbone (Abschnitt 5.6) sowie die Deutung der aufgabenweisen Unterschiede zwischen Llama-3.2-1B und Mistral-7B (Abschnitt 5.10). Alle drei sind dort entsprechend eingeschränkt. Der Befund fügt sich allerdings in ein Muster, das dieses Kapitel wiederholt zeigt: Mehrere zunächst deutlich erscheinende Effekte erweisen sich bei genauerer Prüfung als Eigenschaften der Konfiguration und nicht des untersuchten Gegenstands.

5.9 Encoder- gegenüber LLM-basierten Ansätzen

Tabelle 5.20: Encoder-Baselines gegenüber Llama-3.2-1B (jeweils CONDIFF, kausal), QWK. Encoder-Werte sind Einzelläufe einer frühen Trainingswelle, von der nur die Vorhersagen erhalten sind. [‡]Lauf mit Ergebnis auf Zufallsniveau, siehe Text.

Benchmark	Split	BERT-base	RoBERTa-base	Llama-3.2-1B
ASAP-SAS	UA	0,788	0,821	0,818
Beetle	UA	0,478	0,694	0,729
	UQ	—	0,289	0,509
SciEntsBank	UA	0,052 [‡]	0,633	0,702
	UQ	0,061 [‡]	0,451	0,544
	UD	−0,004 [‡]	0,472	0,560

Der Vergleich in Tabelle 5.20 erweist die Wahl des Evaluationssplits erneut als entscheidend.

Auf UA-Splits sind Encoder konkurrenzfähig. Auf ASAP-SAS erreicht RoBERTa mit 0,821 QWK das Niveau von Llama-3.2-1B (0,818) – bei einem Bruchteil der Parameter und ohne LoRA-Overhead. Auf Beetle-UA liegt RoBERTa mit 0,694 um 0,035 hinter dem Llama-Mittelwert und damit im Bereich der Rauschgrenze. Auf SciEntsBank-UA gilt dies nicht: Dort beträgt der Rückstand 0,069 QWK und übersteigt die Rauschgrenze deutlich. Hätte diese Arbeit nur die UA-Splits von ASAP-SAS und Beetle ausgewertet, wäre der Schluss naheliegend gewesen, dass Decoder-only-LLMs für diese Aufgabe keinen Mehrwert bieten.

Auf Transfersplits bricht der Encoder-Vorteil zusammen. Auf Beetle-UQ fällt RoBERTa auf 0,289 QWK, während Llama-3.2-1B 0,509 hält – ein Abstand von 0,221, also ein Vielfaches der Rauschgrenze. Der Abfall von UA zu UQ beträgt für RoBERTa 0,406, für Llama-3.2-1B lediglich 0,220. Auf SciEntsBank liegt Llama-3.2-1B auf allen drei Splits vorn, mit Abständen von 0,069 bis 0,093.

Die Deutung ist konsistent mit der in Abschnitt 2.4 referierten Motivation: Wenn ein Modell die Bewertungskonvention einer bekannten Frage lernen kann, genügt die Repräsentationsfähigkeit eines mittelgroßen Encoders. Sobald die Frage neu ist und die Bewertung aus dem Rubriktext und fachlichem Verständnis erschlossen werden muss, wirkt sich das im Vortraining erworbene Wissen der größeren Sprachmodelle aus.

Lernratensensitivität der Encoder-Baselines

Die BERT-Werte auf SciEntsBank liegen mit 0,052, 0,061 und −0,004 auf allen drei Splits praktisch auf Zufallsniveau. Zur Klärung wurde für beide Encoder-Backbones eine Lernratenvariation durchgeführt (maximale Sequenzlänge 512, zehn Epochen, vollständiges Finetuning). Anders als bei den Hauptläufen, von denen nur die Vorhersagen erhalten sind, ist die Konfiguration dieser Läufe vollständig dokumentiert. Tabelle 5.21 weist die Hauptläufe deshalb gesondert aus, statt sie einer Lernrate zuzuordnen.

Tabelle 5.21: Lernratenvariation der Encoder-Baselines mit CONDIFF-Fusion, QWK. Einzelläufe, Zufallsaat 114 514. Die BERT-Läufe verwenden Batchgröße 16 bei $1 \cdot 10^{-5}$ und $2 \cdot 10^{-5}$ sowie 32 bei $3 \cdot 10^{-5}$, die RoBERTa-Läufe durchgängig 16. Die letzte Spalte gibt den Hauptlauf aus Tabelle 5.20 wieder, dessen Lernrate nicht dokumentiert ist.

Modell	Benchmark	Split	$1 \cdot 10^{-5}$	$2 \cdot 10^{-5}$	$3 \cdot 10^{-5}$	$5 \cdot 10^{-5}$	Hauptlauf
RoBERTa-base	Beetle	UA	0,665	0,768	—	0,688	0,694
		UQ	0,362	0,337	—	0,288	0,289
	SciEntsBank	UA	0,634	−0,054	—	0,049	0,633
BERT-base	SciEntsBank	UA	0,006	0,009	−0,026	—	0,052

Tabelle 5.21 führt zu drei Befunden.

Auf Beetle streut RoBERTa erheblich, bricht auf UQ aber bei jeder Rate ein. Auf UA reichen die Werte von 0,665 bis 0,768. Die Spanne von 0,103 QWK liegt allerdings unter der größten in Abschnitt 5.2 beobachteten Seed-Spannweite, sodass sich Lernrateneinfluss und Saatrauschen mit je einem Lauf nicht trennen lassen. Bei $2 \cdot 10^{-5}$ übertrifft RoBERTa auf UA mit 0,768 sogar den Llama-Mittelwert um 0,039 QWK. Auf UQ bleibt es dagegen bei allen drei Raten zwischen 0,288 und 0,362 und damit mindestens 0,148 QWK hinter Llama-3.2-1B. Der Kontrast zwischen beiden Splits ist folglich nicht an eine bestimmte Lernrate gebunden. Der Hauptlauf liegt mit 0,694 und 0,289 nahe am Lauf bei $5 \cdot 10^{-5}$.

Auf SciEntsBank arbeitet RoBERTa nur bei der niedrigsten Rate. Bei $1 \cdot 10^{-5}$ erreicht es 0,634 QWK und damit auf UA praktisch denselben Wert wie der Hauptlauf (0,633). Bei $2 \cdot 10^{-5}$ und $5 \cdot 10^{-5}$ scheitert es dagegen mit −0,054 und 0,049; die Accuracy fällt dort mit 0,324 und 0,374 unter die Mehrheitsschranke von 0,431. Auch bei seiner besten Rate liegt RoBERTa auf allen drei Splits hinter Llama-3.2-1B, um 0,068, 0,058 und 0,133 QWK.

BERT ist bei keiner Rate funktionsfähig. Auf SciEntsBank bleibt das Modell bei allen geprüften Lernraten wie im Hauptlauf auf Zufallsniveau; die Werte bewegen sich zwischen −0,026 und 0,052 QWK. Bei den drei dokumentierten Raten liegt die Accuracy mit 0,350 bis 0,374 sogar unter der Mehrheitsschranke von 0,431, die der Hauptlauf exakt erreicht. Die naheliegende Vermutung, der Ausfall sei ein bloßes Optimierungsproblem der Lernrate, ist damit widerlegt.

Was den Ausfall verursacht, lässt sich mit den vorliegenden Daten nicht bestimmen. Eine Zuschreibung an die spezifische Kombination aus BERT-base und SciEntsBank trägt nach der Lernratenvariation nur eingeschränkt, denn auch RoBERTa scheitert auf diesem Benchmark bei zwei von drei Raten. Die Encoder erweisen sich auf SciEntsBank insgesamt als fragil; BERT unterscheidet sich lediglich darin, dass es auch bei der Rate versagt, bei der RoBERTa funktioniert. Die BERT-Zeile für SciEntsBank in Tabelle 5.20 ist folglich von jeder inhaltlichen Deutung auszunehmen.

Für den Vergleich in Tabelle 5.20 folgt daraus zweierlei. Erstens ist die dort berichtete RoBERTa-Leistung auf SciEntsBank nicht wesentlich unterschätzt: Die beste dokumentierte Rate erreicht auf UA praktisch denselben Wert wie der Hauptlauf, liegt auf UQ 0,035 darüber und auf UD 0,046 darunter, und der Vorsprung von Llama-3.2-1B bleibt auf allen drei Splits bestehen. Zweitens gewinnt der qualitative Kernbefund an Gewicht: Dass RoBERTa auf

UA konkurrenzfähig ist und auf UQ einbricht, zeigt sich auf Beetle nicht mehr nur in einem Lauf, sondern bei allen drei geprüften Lernraten. Einschränkend bleibt festzuhalten, dass die Encoder-Läufe auf je einer Saat beruhen und die Decoder-Spalte mit einer anderen Konfiguration (2048 Token, fünf Epochen, LoRA) entstanden ist.

5.10 Ergebnisse je Aufgabe auf ASAP-SAS

Da ASAP-SAS aus zehn inhaltlich und strukturell heterogenen Aufgaben besteht (Kapitel 3), verdeckt der über alle Instanzen aggregierte Wert erhebliche Unterschiede. Tabelle 5.22 schlüsselt das QWK nach Aufgabe auf und vergleicht das kleinste mit dem größten Backbone.

Tabelle 5.22: ASAP-SAS (UA): QWK je Aufgabe für CONDIFF mit kausaler Attention. Einzelläufe.

Item	Fach	Skala	Llama-1B	Mistral-7B	Differenz
1	Science	{0..3}	0,829	0,881	+0,052
2	Science	{0..3}	0,776	0,898	+0,122
5	Biology	{0..3}	0,832	0,852	+0,020
6	Biology	{0..3}	0,801	0,892	+0,091
3	English Language Arts	{0..2}	0,663	0,653	−0,011
4	English Language Arts	{0..2}	0,728	0,734	+0,007
7	English	{0..2}	0,702	0,738	+0,035
8	English	{0..2}	0,625	0,697	+0,072
9	English	{0..2}	0,834	0,832	−0,002
10	Science	{0..2}	0,731	0,772	+0,041
Mittel vierstufige Items			0,810	0,881	+0,071
Mittel dreistufige Items			0,714	0,738	+0,024

Die Streuung zwischen Aufgaben übertrifft alle untersuchten Architektureffekte. Beim 1B-Backbone reicht das QWK von 0,625 (Item 8) bis 0,834 (Item 9), beim 7B-Backbone von 0,653 bis 0,898. Diese Spanne von rund 0,21 beziehungsweise 0,25 liegt um ein Vielfaches über den Unterschieden zwischen Fusionsmechanismen, Bezugsgrößen oder Attention-Bedingungen auf demselben Benchmark, die höchstens 0,015 QWK erreichen, und in derselben Größenordnung wie der Effekt des Rubriktexts. Sie übertrifft auch den größten Architektureffekt dieser Arbeit, den Verlust von 0,186 QWK durch vollständige Bidirektionalität auf Beetle-UQ, wenn auch nur knapp. Welche Aufgaben ein Benchmark enthält, wirkt sich damit stärker aus als jede der hier untersuchten Architekturvarianten – ein Umstand, der bei der Verallgemeinerung aggregierter Benchmarkwerte zu berücksichtigen ist.

Abbildung 5.3 macht dieses Größenverhältnis sichtbar: Die Spannweite über die zehn Aufgaben ist dort der Rauschgrenze gegenübergestellt, an der sämtliche Architekturvergleiche dieses Kapitels zu messen sind.

Der Abstand zwischen beiden Backbones konzentriert sich auf die vierstufigen naturwissenschaftlichen Aufgaben. Auf den vier Items mit Skala $\{0, \dots, 3\}$ – durchweg Science

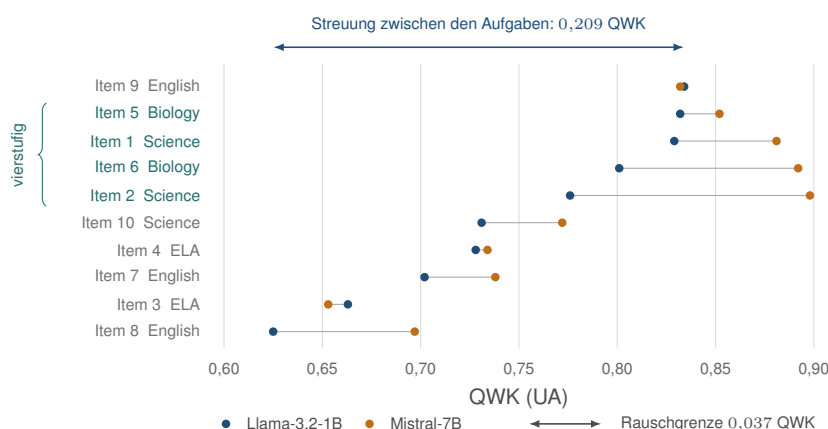


Abbildung 5.3: ASAP-SAS (UA): QWK je Aufgabe für CONDIFF mit kausaler Attention, aufsteigend nach dem Wert des 1B-Backbones geordnet. Die Streuung zwischen den Aufgaben liegt um ein Vielfaches über den Architektureffekten auf ASAP-SAS; der Abstand zwischen beiden Backbones konzentriert sich auf die vierstufigen naturwissenschaftlichen Aufgaben, ist wegen der unterschiedlichen Lernraten aber nicht als Größeneffekt zu lesen. Einzelläufe, Werte aus Tabelle 5.22.

oder Biology – liegt Mistral-7B im Mittel 0,071 QWK vorn, auf den sechs dreistufigen lediglich 0,024. Auf den beiden Aufgaben aus dem Bereich English Language Arts ist der Abstand praktisch null: $-0,011$ auf Item 3 und $+0,007$ auf Item 4. Das Muster ließe sich mit der Beschreibung dieser Items in Kapitel 3 verbinden: Die vierstufigen naturwissenschaftlichen Aufgaben verlangen die Identifikation mehrerer inhaltlicher Teilaspekte, wofür Fachwissen erforderlich ist, über das ein größeres Sprachmodell aus dem Vortraining eher verfügt. Als Größeneffekt ist es jedoch nicht zu lesen. Beide Spalten beruhen auf Einzelläufen, und Llama-3.2-1B wurde, wie in Abschnitt 5.8 dargelegt, bei einem Viertel seiner Referenzlernrate trainiert, Mistral-7B bei der seinen. Ob die Konzentration auf die vierstufigen Aufgaben bei angepasster Lernrate bestehen bliebe, ist offen.

5.11 Instabile Läufe

Drei Läufe liegen deutlich unterhalb ihrer Vergleichsbedingungen. Da für sie nur eine Zufalls-saat vorliegt, wurden die zugehörigen Vorhersagedateien gesondert ausgewertet, um zwischen einem inhaltlichen Effekt und einer Trainingsinstabilität zu unterscheiden. Als Referenzpunkte dienen die Accuracy eines trivialen Modells, das stets die häufigste Stufe vorhersagt, sowie das QWK bei zufälliger Permutation der Vorhersagen – also bei unveränderter Randverteilung, aber aufgehobener Zuordnung.

Die beiden Mistral-Läufe sind funktionsfähig, aber schwach. Beide sagen sämtliche Stufen in annähernd korrekter Randverteilung vorher – beim SciEntsBank-Lauf mit $\rho = 0,6$ stimmt die Vorhersageverteilung auf dem UD-Split mit (1733, 1001, 1828) nahezu exakt mit der Referenzverteilung (1659, 986, 1917) überein. Ihre Accuracy liegt mit 0,674 beziehungsweise 0,578 deutlich über der Mehrheitsschranke, und ihre Konfusionsmatrizen zeigen eine sinnvolle Diagonalstruktur. Es handelt sich also nicht um abgebrochene Trainingsläufe oder um Modelle,

Tabelle 5.23: Diagnose der auffälligen Läufe. $Acc_{Mehrheit}$ ist die Accuracy bei konstanter Vorhersage der häufigsten Stufe, QWK_{perm} das QWK bei einer einzelnen zufälligen Permutation der Vorhersagen.

Lauf	Split	QWK	Acc	$Acc_{Mehrheit}$	QWK_{perm}
Beetle, Mistral, $\rho = 0,2$	UA	0,587	0,674	0,401	−0,018
Beetle, Mistral, kausal	UA	0,803	0,788	0,401	0,000
Beetle, Mistral, $\rho = 0,2$	UQ	0,210	0,480	0,420	−0,001
Beetle, Mistral, kausal	UQ	0,606	0,661	0,420	−0,015
SciEntsBank, Mistral, $\rho = 0,6$	UQ	0,378	0,578	0,411	−0,004
SciEntsBank, Mistral, $\rho = 0,2$	UQ	0,627	0,690	0,411	−0,005
SciEntsBank, BERT	UA	0,052	0,431	0,431	−0,012
SciEntsBank, RoBERTa	UA	0,633	0,680	0,431	0,005

die lediglich die Klassenverteilung reproduzieren, sondern um Modelle, die die Aufgabe gelernt haben – nur erheblich schlechter. Eine Auswertung je Frage zeigt ferner, dass die Verschlechterung breit verteilt ist: Auf Beetle fällt der Lauf mit $\rho = 0,2$ bei 28 der 47 Fragen um mehr als 0,1 QWK gegenüber der kausalen Referenz ab und verbessert sich nur bei vier Fragen. Ein einzelnes problematisches Item oder ein Datenartefakt scheidet damit als Erklärung aus.

Die Abstände von bis zu 0,22 QWK auf UA und 0,40 auf UQ liegen weit über der in Abschnitt 5.2 bestimmten Rauschgrenze und auch über der größten dort beobachteten Spannweite (0,118). Zugleich spricht gegen eine inhaltliche Deutung als Maskeneffekt, dass $\rho = 0,6$ mit 19 bidirektionalen Schichten auf demselben Benchmark 0,801 erreicht – mehr als das Dreifache an bidirektionalen Schichten bei nahezu unveränderter Leistung. Beide Läufe sollten wiederholt werden; bis dahin werden sie in den Tabellen gekennzeichnet und von der Deutung ausgenommen.

Der BERT-Lauf auf SciEntsBank ist dagegen ein echter Ausfall. Seine Accuracy beträgt 0,431 und stimmt damit auf drei Nachkommastellen exakt mit der Mehrheitsschranke überein. Die Konfusionsmatrix zeigt, dass 148 von 194 Instanzen der Stufe 0 und 76 von 113 der Stufe 1 der Stufe 2 zugeordnet werden; insgesamt entfallen 77 % aller Vorhersagen auf die häufigste Stufe, gegenüber 43 % in der Referenz. Das Modell hat also nichts über die Klassenverteilung hinaus gelernt. Die in Abschnitt 5.9 berichtete Lernratenvariation zeigt zudem, dass sich dieser Ausfall durch keine der drei zusätzlich geprüften Lernraten beheben lässt. Der Lauf ist damit als Fehlschlag einzuordnen, dessen Ursache offen bleibt; inhaltlich zu deuten ist er in keinem Fall.

5.12 Robustheit gegenüber der Metrik

Sämtliche Urteile der Abschnitte 5.3 bis 5.7 beruhen auf QWK. Da Abschnitt 2.2 Macro- F_1 als zweite Metrik einführt, wurde jeder dort berichtete Paarvergleich mit drei Zufallssaaten unter Macro- F_1 wiederholt, und zwar nach derselben Auswertungsregel, angewandt auf Mittelwerte

Tabelle 5.24: Paarvergleiche mit drei Zufallssaaten, deren Urteil unter QWK und Macro- F_1 verschieden ausfällt. Δ ist die Differenz der Mittelwerte (erste minus zweite Bedingung); das Urteil folgt der Auswertungsregel aus Abschnitt 5.2, angewandt auf die jeweilige Metrik. „—“ bedeutet nicht nachweisbar. Kontext bezeichnet Frage und Musterlösung.

Abschn.	Vergleich	Split	Δ QWK	Urteil	ΔF_1	Urteil
5.3	SciEntsBank 1B: Kontext mit gegen ohne Rubrik	UQ	+0,015	—	+0,019	Hinweis
5.3	SciEntsBank 1B: Kontext mit gegen ohne Rubrik	UD	+0,006	—	+0,029	Effekt
5.3	SciEntsBank 1B: nur Kontext gegen nur Rubrik	UQ	+0,060	Effekt	-0,003	—
5.3	SciEntsBank 1B: nur Kontext gegen nur Rubrik	UD	+0,077	Effekt	+0,014	Hinweis
5.3	SciEntsBank 1B: Kontext und Rubrik gegen nur Rubrik	UQ	+0,075	Effekt	+0,017	—
5.3	Beetle 1B: Rubrik gegen weder noch	UA	+0,054	Hinweis	+0,040	Effekt
5.4	Beetle 1B: CONDIFF gegen CONCAT	UA	+0,005	—	+0,006	Hinweis
5.4	Beetle 1B: CONDIFF gegen GATE	UQ	+0,038	—	+0,041	Hinweis
5.4	Beetle 1B: CONCAT gegen GATE	UQ	+0,032	—	+0,036	Hinweis
5.4	SciEntsBank 1B: CONDIFF gegen CONCAT	UD	-0,017	—	-0,011	Hinweis
5.4	SciEntsBank 1B: CONCAT gegen GATE	UD	+0,017	—	+0,019	Hinweis
5.4	Beetle 3B: DIFF gegen GATE	UA	-0,017	Hinweis	-0,009	Effekt
5.4	SciEntsBank 3B: CONDIFF gegen GATE	UQ	-0,049	Hinweis	-0,019	—
5.4	SciEntsBank 3B: DIFF gegen GATE	UA	-0,014	—	-0,012	Hinweis
5.5	Beetle 3B: t-concat gegen CONCAT	UA	+0,026	Hinweis	+0,014	Effekt
5.7	Beetle 1B: bidirektional gegen kausal	UQ	-0,016	—	-0,041	Effekt
5.7	Beetle 3B: bidirektional gegen kausal	UA	-0,073	Effekt	-0,028	—
5.7	SciEntsBank 3B: bidirektional gegen kausal	UA	+0,032	Hinweis	+0,011	—

und Spannweiten des Macro- F_1 . Einbezogen sind 83 Vergleiche je Split: die Ablations- und Kontextvergleiche aus Abschnitt 5.3, sämtliche Paare von Fusionsvarianten je Backbone, t-concat gegenüber CONCAT, Span-Alignment gegenüber festem Kopf sowie die vollständig bidirektionale gegenüber der kausalen Bedingung.

In 65 der 83 Vergleiche fällt das Urteil unter beiden Metriken gleich aus. Tabelle 5.24 listet die 18 abweichenden.

Aufgabeninformation

Der Kernbefund ist metrikunabhängig. Der Einbruch ohne jede Aufgabeninformation besteht unter Macro- F_1 auf beiden Llama-Größen und allen drei SciEntsBank-Splits ebenso als Effekt wie unter QWK. Dasselbe gilt auf UA für den Vorsprung von Kontext allein gegenüber Rubrik

Tabelle 5.25: SciEntsBank, Llama-3.2-1B, fester Klassifikationskopf: Anteil weiter Fehler (Vorhersage und Referenz liegen zwei Stufen auseinander) und Accuracy. Mittelwerte über drei Zufallssaaten, beim Anteil weiter Fehler mit Bereich über die Saaten.

Bedingung	UQ		UD	
	weite Fehler	Acc	weite Fehler	Acc
nur Kontext	0,068 (0,064–0,074)	0,652	0,074 (0,062–0,082)	0,641
nur Rubrik	0,117 (0,109–0,128)	0,666	0,114 (0,108–0,122)	0,627
Kontext und Rubrik	0,072 (0,065–0,079)	0,675	0,078 (0,072–0,084)	0,665

allein und für den Zugewinn durch Kontext bei vorhandener Rubrik; Letzterer besteht auch auf UD.

Metrikabhängig ist dagegen das Verhältnis beider Informationsquellen auf den Transfersplits. Unter QWK übertrifft Kontext allein die Rubrik allein auch auf UQ und UD als Effekt. Unter Macro- F_1 liegen beide auf UQ gleichauf ($-0,003$), auf UD bleibt ein schwacher Hinweis ($+0,014$). Umgekehrt trägt der Rubriktext bei vorhandenem Kontext unter Macro- F_1 etwas bei: auf UD als Effekt ($+0,029$), auf UQ als Hinweis ($+0,019$), während QWK in beiden Fällen keinen Unterschied nachweist.

Die Abweichung lässt sich an der Art der Fehler festmachen. Bei drei Stufen gewichtet QWK die Verwechslung von *incorrect* und *correct* viermal so stark wie die Verwechslung benachbarter Stufen; Macro- F_1 unterscheidet beide Fehlerarten nicht. Tabelle 5.25 zeigt, dass der Aufgabenkontext genau die weiten Fehler reduziert.

Ohne Kontext liegen Vorhersage und Referenz auf beiden Transfersplits bei gut 11 % der Instanzen zwei Stufen auseinander, mit Kontext bei rund 7 %; die Bereiche über die Saaten überlappen in keinem Fall. Die Accuracy unterscheidet sich dagegen wenig, auf UQ liegt die Rubrik allein nominell sogar vorn. Der Kontext macht das Modell also nicht wesentlich häufiger richtig, sondern seltener grob falsch – ein Unterschied, den QWK belohnt und Macro- F_1 nicht erfasst. Umgekehrt erhöht der Rubriktext bei vorhandenem Kontext auf UD die Accuracy, ohne den Anteil weiter Fehler zu verändern; das erklärt, weshalb dort Macro- F_1 einen Beitrag anzeigt und QWK nicht. Die Aussage, Frage und Musterlösung seien dem Rubriktext überlegen, betrifft auf den Transfersplits somit die Schwere der Fehler, nicht ihre Häufigkeit.

Architektur

Fusion und Bezugsgröße. Unter den Fusionsvergleichen verschiebt sich das Urteil in acht Fällen, davon sieben zwischen „nicht nachweisbar“ und „Hinweis“. Die einzige Verschiebung zum Effekt betrifft GATE gegenüber DIFF auf Beetle-UA mit Llama-3.2-3B, bei einer Differenz von lediglich 0,009 Macro- F_1 und entsprechend enger Streuung. Das in Abschnitt 5.4 beschriebene Muster bleibt unter Macro- F_1 in der Tendenz erhalten: Beim kleineren Backbone liegt GATE auf Beetle-UQ mit disjunkten Spannweiten hinter CONDIFF und CONCAT, beim größeren vor DIFF; der Hinweis auf einen Vorsprung von GATE vor CONDIFF auf SciEntsBank-UQ entfällt dagegen. Der Hinweis auf einen Vorteil von t-concat auf Beetle-UA wird unter Macro- F_1

zum Effekt, mit einer Differenz von 0,014. Die Vergleiche von Span-Alignment und festem Kopf fallen unter beiden Metriken gleich aus.

Attention-Maske. Der Verlust des größeren Backbones auf Beetle-UQ und sein Gewinn auf SciEntsBank-UQ bestehen unter beiden Metriken als Effekt, ebenso der Gewinn des kleineren Backbones auf Beetle-UA. Nicht metrikrobust sind der Verlust des größeren Backbones auf Beetle-UA und der Hinweis auf SciEntsBank-UA. Hinzu kommt unter Macro- F_1 ein Verlust des kleineren Backbones auf Beetle-UQ ($-0,041$). Auf Beetle-UQ schadet die vollständige Aufhebung der Kausalität unter dieser Metrik somit beiden Backbones.

Folgerung

Robust gegenüber der Wahl der Metrik sind der Einbruch ohne Aufgabeninformation, der Vorsprung des Kontexts auf UA, das Fehlen von Fusionsunterschieden oberhalb der Rauschgrenze sowie die Maskeneffekte auf den UQ-Splits des größeren Backbones und auf Beetle-UA des kleineren. Metrikabhängig sind die Überlegenheit des Kontexts auf den Transfersplits und der fehlende Beitrag des Rubriktexts bei vorhandenem Kontext; beide Aussagen gelten nur unter QWK, das grobe Fehlbewertungen stärker gewichtet.

5.13 Zusammenfassung der Befunde

Die Ergebnisse dieses Kapitels lassen sich entlang einer einzigen Unterscheidung ordnen: Entscheidend ist, *welche Information* dem Modell zur Verfügung steht, nicht, *wie* sie architektonisch verarbeitet wird.

1. **Aufgabenbezogene Information ist der bestimmende Faktor – nicht ihre Form** (Abschnitt 5.3). Wird dem Modell jede Aufgabeninformation entzogen, kostet das zwischen 0,235 und 0,528 QWK, und auf den Transfersplits fällt die Leistung auf oder unter das Zufallsniveau; zusätzliche Modellkapazität kompensiert das nicht. Der Rubriktext ist dabei jedoch *austauschbar*: Liegen Frage und Musterlösung vor, trägt er unter QWK nichts mehr bei ($-0,012$ bis $+0,015$ QWK, sämtlich im Rauschen), und als alleinige Informationsquelle ist er ihnen um 0,060 bis 0,100 QWK unterlegen. Unter Macro- F_1 gilt beides nur eingeschränkt: Die Unterlegenheit zeigt sich dort nur auf UA, und bei vorhandenem Kontext trägt der Rubriktext auf UD nachweisbar bei (Abschnitt 5.12).
2. **Die Ausgestaltung der Fusion ist ohne nachweisbare Wirkung** (Abschnitte 5.4 bis 5.6). Weder die Wahl zwischen CONDIFF, CONCAT, DIFF und GATE noch die Bezugsgröße der Fusion noch – beim größeren Backbone – die Span-Extraktion selbst erzeugen Unterschiede oberhalb der Rauschgrenze. Ein schwacher Hinweis bleibt bestehen: Auf Llama-3.2-3B liegt GATE in vier von fünf Zellen vorn, in zweien davon mit disjunkten Spannweiten gegenüber mindestens einer anderen Variante.
3. **Die Attention-Maske wirkt, aber mit von Benchmark und Backbone abhängigem Vorzeichen** (Abschnitt 5.7). Die vollständige Aufhebung der Kausalität kostet Llama-3.2-3B auf Beetle 0,186 QWK auf UQ und bringt ihm auf SciEntsBank 0,060. Llama-3.2-1B



gewinnt auf Beetle dagegen 0,036 auf UA und zeigt auf SciEntsBank keinen nachweisbaren Unterschied – sämtliche Werte über drei Zufallssaaten. Diese Befunde bestehen auch unter Macro- F_1 ; dort verliert zusätzlich das kleinere Backbone auf Beetle-UQ.

4. **Über die Backbone-Größe lässt sich keine Aussage treffen** (Abschnitt 5.8). Der scheinbar deutliche Größeneffekt ist mit der Lernrate konfundiert: Llama-3.2-1B gewinnt bei seiner Referenzlernrate auf Beetle 0,096 QWK auf UQ und holt damit den gesamten Abstand zu Mistral-7B auf.
5. **Die Wahl des Evaluationssplits entscheidet über die Schlussfolgerung** (Abschnitt 5.9). Auf UA sind Encoder-Modelle konkurrenzfähig; auf UQ liegt Llama-3.2-1B um 0,221 QWK vor RoBERTa – und dieser Einbruch zeigt sich bei jeder geprüften Lernrate.
6. **Die Streuung zwischen Aufgaben übertrifft alle Architektureffekte** (Abschnitt 5.10) und erreicht auf ASAP-SAS eine Spanne von 0,21 bis 0,25 QWK zwischen den zehn Items desselben Benchmarks.

Die methodisch weitreichendste Konsequenz betrifft das Verhältnis von Befund und Messgenauigkeit. Mehrere Muster, die sich auf Einzelläufen klar abzeichneten – eine Umkehr der Fusionsrangfolge zwischen den Splits, ein Vorteil des Antwort-Span-Alignments, ein Verlust durch vollständige Bidirektionalität beim kleineren Backbone auf SciEntsBank –, halten der Wiederholung unter drei Zufallssaaten nicht stand. Sie wären ohne die in Abschnitt 5.2 durchgeführte Bestimmung der Rauschgrenze als Ergebnisse berichtet worden.

6 Diskussion

7 Fazit und Ausblick

Tabellenverzeichnis

3.1	ASAP-SAS: Aufgabenübersicht mit Fach, Klassenstufe, Label-Skala und Instanzzahlen (berechnet aus <code>train.csv</code> / <code>test_ua.csv</code>).	32
3.2	Beetle: Instanzzahlen und Fragenzahlen je Split (berechnet aus den bereitgestellten CSV-Dateien).	33
3.3	SciEntsBank: Instanzzahlen und Fragenzahlen je Split (berechnet aus den bereitgestellten CSV-Dateien).	35
3.4	Gegenüberstellung der drei verwendeten ASAG-Benchmarks.	37
4.1	Die vier verglichenen Fusionsmechanismen. d bezeichnet die Modelldimension, d_g die Eingabedimension des Scoring-Kopfs.	46
4.2	Aus dem Anteil ρ resultierende Zahl bidirektionaler Schichten n_{bidir} je Backbone nach Vorschrift (4.15). Die Zeilen sind nach der tatsächlichen Zahl bidirektionaler Schichten geordnet, nicht nach dem nominellen Wert von ρ . Die vollständig bidirektionale Bedingung $\rho = L$ liegt für Llama-3.2-1B und -3B vor, für Mistral-7B nicht; unter drei Zufallssaaten liegt sie für beide Llama-Größen vor.	49
4.3	Trainingskonfiguration der Decoder-Backbones. Die Werte entsprechen der in den ausgewerteten Decoder-Läufen verwendeten Konfiguration; die Abweichungen der Encoder-Baselines beschreibt der folgende Unterabschnitt.	53
5.1	Spannweite Δ des QWK über drei Zufallssaaten, aggregiert über 72 Zellen. . .	60
5.2	Einfluss der Bibliotheksversion bei identischer Konfiguration und identischer Zufallssaat (SciEntsBank, Llama-3.2-3B, fester Klassifikationskopf).	60
5.3	SciEntsBank, Llama-3.2-1B, fester Klassifikationskopf: Verfügbarkeit von Aufgabenkontext (Frage und Musterlösung) und Rubriktext im vollständigen 2×2 -Schema. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. . . .	62
5.4	SciEntsBank: Ablation der Rubriktexte ohne Aufgabenkontext, über zwei Modellgrößen. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. <i>Lücke</i> bezeichnet den Abstand zwischen dem schlechtesten Lauf mit und dem besten Lauf ohne Rubriktext.	64
5.5	Beetle, Llama-3.2-1B, fester Klassifikationskopf, ohne Aufgabenkontext: Ablation der Rubriktexte. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.	64
5.6	Beetle, Llama-3.2-1B, fester Klassifikationskopf: Verfügbarkeit von Aufgabenkontext und Rubriktext. Sämtliche Werte bei der Zufallssaat 114514, um die Zellen untereinander vergleichbar zu halten. Für die beiden unteren Zeilen liegen zusätzlich drei Saaten vor (Mittelwerte 0,641 / 0,456 beziehungsweise 0,587 / 0,457).	65

5.7	ASAP-SAS (UA): Fusionsmechanismen bei kausaler Attention. Einzelläufe, Zufallssaat 114514.	67
5.8	Beetle und SciEntsBank: Fusionsmechanismen bei kausaler Attention, QWK je Split. Mittelwerte über drei Zufallssaaten; für Mistral-7B liegt nur CONDIFF mit einer Saat vor.	67
5.9	Beetle und SciEntsBank: GATE- und DIFF-Fusion, QWK je Split. Mittelwerte über drei Zufallssaaten. DIFF liegt nur für Llama-3.2-3B vor.	67
5.10	Unterschied zwischen den Fusionsmechanismen (Spannweite der Mittelwerte von CONDIFF, CONCAT und GATE, bei Llama-3.2-3B zusätzlich DIFF) gegenüber der größten Seed-Spannweite derselben Zellen.	68
5.11	Beste Fusionsvariante je Benchmark, Split und Backbone nach dem Mittelwert über drei Zufallssaaten. Die Rangfolge ist nach Tabelle 5.10 nicht belastbar und wird hier ausschließlich zur Illustration ihrer Beliebigkeit angegeben.	68
5.12	Bezugsgröße der Fusion auf Llama-3.2-3B: Antwort-Span (t-concat) gegenüber Sequenzrepräsentation (CONCAT, CONDIFF). Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.	69
5.13	SciEntsBank: Span-Alignment (CONDIF) gegenüber festem Klassifikationskopf. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. <i>Über Spannweite</i> bedeutet, dass die Differenz die größere der beiden zellspezifischen Spannweiten übersteigt; sämtliche Differenzen dieser Tabelle liegen unterhalb des globalen Medians von 0,037 QWK.	71
5.14	ASAP-SAS (UA): Attention-Maske bei CONDIFF-Fusion, sortiert nach der tatsächlichen Zahl bidirektionaler Schichten n_{bidir} . Einzelläufe, Zufallssaat 114514. Die vollständig bidirektionale Bedingung liegt für Mistral-7B nicht vor.	72
5.15	Beetle und SciEntsBank: Attention-Maske bei CONDIFF-Fusion, QWK je Split, sortiert nach n_{bidir} . Einzelläufe, Zufallssaat 114514. Für Llama-3.2-3B und Mistral-7B liegen dieselben Bedingungen ebenfalls als Einzelläufe vor; sie werden hier nicht aufgeführt, da sie zu keiner Aussage beitragen, die nicht schon aus der vollständigen Aufhebung folgt.	72
5.16	Vollständig bidirektionale Attention ($n_{\text{bidir}} = L = 28$) gegenüber kausaler Attention, Llama-3.2-3B, CONDIFF-Fusion. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern.	73
5.17	Vollständig bidirektionale Attention ($n_{\text{bidir}} = L = 16$) gegenüber kausaler Attention, Llama-3.2-1B, CONDIFF-Fusion. Mittelwerte über drei Zufallssaaten, Spannweiten in Klammern. <i>Über Spannweite</i> wie in Tabelle 5.13.	74
5.18	Effekt der Backbone-Größe bei CONDIFF und kausaler Attention (QWK). Für Llama-3.2-1B und -3B Mittelwerte über drei Zufallssaaten, für Mistral-7B Einzelläufe.	75
5.19	Llama-3.2-1B mit CONDIFF bei der Referenzlernrate $2 \cdot 10^{-4}$ gegenüber den Läufen bei $5 \cdot 10^{-5}$ (Mittelwert und Spanne über drei Zufallssaaten). Der Lauf bei $2 \cdot 10^{-4}$ beruht auf einer einzelnen Zufallssaat.	76
5.20	Encoder-Baselines gegenüber Llama-3.2-1B (jeweils CONDIFF, kausal), QWK. Encoder-Werte sind Einzelläufe einer frühen Trainingswelle, von der nur die Vorhersagen erhalten sind. [‡] Lauf mit Ergebnis auf Zufallsniveau, siehe Text.	77

5.21 Lernratenvariation der Encoder-Baselines mit CONDIFF-Fusion, QWK. Einzelläufe, Zufallssaat 114514. Die BERT-Läufe verwenden Batchgröße 16 bei $1 \cdot 10^{-5}$ und $2 \cdot 10^{-5}$ sowie 32 bei $3 \cdot 10^{-5}$, die RoBERTa-Läufe durchgängig 16. Die letzte Spalte gibt den Hauptlauf aus Tabelle 5.20 wieder, dessen Lernrate nicht dokumentiert ist.	78
5.22 ASAP-SAS (UA): QWK je Aufgabe für CONDIFF mit kausaler Attention. Einzelläufe.	79
5.23 Diagnose der auffälligen Läufe. $Acc_{Mehrheit}$ ist die Accuracy bei konstanter Vorhersage der häufigsten Stufe, QWK_{perm} das QWK bei einer einzelnen zufälligen Permutation der Vorhersagen.	81
5.24 Paarvergleiche mit drei Zufallssaat, deren Urteil unter QWK und Macro- F_1 verschieden ausfällt. Δ ist die Differenz der Mittelwerte (erste minus zweite Bedingung); das Urteil folgt der Auswertungsregel aus Abschnitt 5.2, angewandt auf die jeweilige Metrik. „—“ bedeutet nicht nachweisbar. Kontext bezeichnet Frage und Musterlösung.	82
5.25 SciEntsBank, Llama-3.2-1B, fester Klassifikationskopf: Anteil weiter Fehler (Vorhersage und Referenz liegen zwei Stufen auseinander) und Accuracy. Mittelwerte über drei Zufallssaat, beim Anteil weiter Fehler mit Bereich über die Saaten.	83

Abbildungsverzeichnis

2.1	Klassische und rubrik-konditionierte Aufgabenformulierung	5
2.2	Generalisierungsdimensionen UA, UQ und UD	6
2.3	Gewichtsmatrizen von Accuracy und QWK	9
2.4	Rauschgrenze und Interpretierbarkeit von Unterschieden	12
2.5	Drei Architekturfamilien für rubrik-geleitetes Scoring	16
2.6	Aufbau der Transformer-Architektur	18
2.7	Berechnungsweg der skalierten Punktprodukt-Attention	19
2.8	Rotary Position Embedding	21
2.9	Kausale, bidirektionale und blocksparse Attention-Maske	23
2.10	Schichtweise Aufhebung der kausalen Maske	25
2.11	Low-Rank Adaptation	27
3.1	Umfang und Split-Struktur der drei Benchmarks	36
4.1	Gesamtarchitektur des RUSPAN-Systems	40
4.2	Serialisierung und Span-Pooling	43
4.3	Konstruktionen der Fusionsmechanismen	46
5.1	Kontext und Rubriktext im 2×2 -Schema	66
5.2	Attention-Maske über den Anteil bidirektionaler Schichten	73
5.3	QWK je Aufgabe auf ASAP-SAS	80

Literatur

- [1] Jimmy Lei Ba, Jamie Ryan Kiros und Geoffrey E. Hinton. *Layer Normalization*. arXiv:1607.06450. 2016. arXiv: 1607.06450.
- [2] Xiaoyu Bai und Manfred Stede. „A Survey of Current Machine Learning Approaches to Student Free-Text Evaluation for Intelligent Tutoring“. In: *International Journal of Artificial Intelligence in Education* 33.1 (2022), S. 1–39.
- [3] Parishad BehnamGhader u. a. *LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders*. arXiv:2404.05961. 2024. arXiv: 2404.05961.
- [4] MV Berry u. a. „Can apparent superluminal neutrino speeds be explained as a quantum weak measurement?“ In: *arXiv preprint arXiv:1110.2832* (2011).
- [5] Marie Bexte, Andrea Horbach und Torsten Zesch. „Similarity-Based Content Scoring – How to Make S-BERT Keep Up with BERT“. In: *Proceedings of the 17th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2022)*. Seattle, Washington: Association for Computational Linguistics, 2022, S. 118–123.
- [6] Gary Blake und Robert W Bly. *The elements of technical writing*. Macmillan New York, NY, 1993.
- [7] Steven Burrows, Iryna Gurevych und Benno Stein. „The Eras and Trends of Automatic Short Answer Grading“. In: *International Journal of Artificial Intelligence in Education* 25.1 (2015), S. 60–117.
- [8] Leon Camus und Anna Filighera. „Investigating Transformers for Automatic Short Answer Grading“. In: *Artificial Intelligence in Education: 21st International Conference, AIED 2020, Proceedings, Part II*. Springer, 2020, S. 43–48.
- [9] Imran Chamieh, Torsten Zesch und Bela Gipp. „LLMs in Short Answer Scoring: Limitations and Promise of Zero-Shot and Few-Shot Approaches“. In: *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*. Association for Computational Linguistics, 2024, S. 309–315.
- [10] L. H. Chang, P. M. Taiga und J. Vilén. „Automatic Short Answer Grading for Finnish with ChatGPT“. In: *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-24)*. 2024.
- [11] Qian Chen u. a. „Enhanced LSTM for Natural Language Inference“. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, 2017, S. 1657–1668.

- [12] Jacob Cohen. „A Coefficient of Agreement for Nominal Scales“. In: *Educational and Psychological Measurement* 20.1 (1960), S. 37–46.
- [13] Jacob Cohen. „Weighted Kappa: Nominal Scale Agreement Provision for Scaled Disagreement or Partial Credit“. In: *Psychological Bulletin* 70.4 (1968), S. 213–220.
- [14] Alexis Conneau u. a. „Supervised Learning of Universal Sentence Representations from Natural Language Inference Data“. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Copenhagen, Denmark: Association for Computational Linguistics, 2017, S. 670–680.
- [15] Tim Dettmers u. a. „QLoRA: Efficient Finetuning of Quantized LLMs“. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 2023.
- [16] Jacob Devlin u. a. „BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding“. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, S. 4171–4186.
- [17] Jesse Dodge u. a. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. arXiv:2002.06305. 2020. arXiv: 2002.06305.
- [18] Afrizal Doewes, Nughthoh Arfawi Kurdhi und Akrati Saxena. „Evaluating Quadratic Weighted Kappa as the Standard Performance Metric for Automated Essay Scoring“. In: *Proceedings of the 16th International Conference on Educational Data Mining (EDM 2023)*. 2023. DOI: 10.5281/zenodo.8115784.
- [19] Myroslava Dzikovska u. a. „SemEval-2013 Task 7: The Joint Student Response Analysis and 8th Recognizing Textual Entailment Challenge“. In: *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*. Atlanta, Georgia, USA: Association for Computational Linguistics, 2013, S. 263–274.
- [20] Ahmed Elshabrawy u. a. „Enabling Natural Zero-Shot Prompting on Encoder Models via Statement-Tuning“. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025, S. 8302–8321.
- [21] European Commission. *Data Protection Explained*. https://commission.europa.eu/law/law-topic/data-protection/data-protection-explained_en. Abgerufen am 05.09.2026. 2026.
- [22] Rafael Ferreira Mello u. a. „Automatic Short Answer Grading in the LLM Era: Does GPT-4 with Prompt Engineering Beat Traditional Models?“ In: *Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK)*. ACM, 2025, S. 93–103.
- [23] Zorik Gekhman u. a. *Inside-Out: Hidden Factual Knowledge in LLMs*. arXiv:2503.15299. 2025. arXiv: 2503.15299.
- [24] Sebastian Gombert u. a. „Are Rubrics All You Need? Towards Rubric-Based Automatic Short Answer Scoring via Guided Rubric-Answer Alignment“. In: *Proceedings of LAK26: 16th International Learning Analytics and Knowledge Conference*. 2026.
- [25] Aaron Grattafiori u. a. *The Llama 3 Herd of Models*. arXiv:2407.21783. 2024. arXiv: 2407.21783.



- [26] Momchil Hardalov u. a. *Few-Shot Cross-Lingual Stance Detection with Sentiment-Based Pre-Training*. arXiv:2109.06050. 2021. arXiv: 2109.06050.
- [27] Neil Houlsby u. a. „Parameter-Efficient Transfer Learning for NLP“. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, S. 2790–2799.
- [28] Edward J. Hu u. a. „LoRA: Low-Rank Adaptation of Large Language Models“. In: *The Tenth International Conference on Learning Representations (ICLR)*. 2022.
- [29] Albert Q. Jiang u. a. *Mistral 7B*. arXiv:2310.06825. 2023. arXiv: 2310.06825.
- [30] Gerd Kortemeyer. „Performance of the Pre-Trained Large Language Model GPT-4 on Automated Short Answer Grading“. In: *Discover Artificial Intelligence 4.1* (2024).
- [31] Yaman Kumar u. a. „Get IT Scored Using AutoSAS – An Automated System for Scoring Short Answers“. In: *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*. 2019, S. 9662–9669.
- [32] Chankyu Lee u. a. *NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models*. arXiv:2405.17428. 2025. arXiv: 2405.17428.
- [33] Xiang Lisa Li und Percy Liang. „Prefix-Tuning: Optimizing Continuous Prompts for Generation“. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 2021, S. 4582–4597.
- [34] Zhaohui Li u. a. „Answer-State Recurrent Relational Network (AsRRN) for Constructed Response Assessment and Feedback Grouping“. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, S. 3879–3891.
- [35] Ziyong Lin u. a. „Look Both Ways and No Sink: Converting LLMs into Text Encoders Without Training“. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, S. 22839–22853.
- [36] Chun Liu u. a. „LLMEmbed: Rethinking Lightweight LLM’s Genuine Function in Text Classification“. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, S. 7994–8004.
- [37] Yinhan Liu u. a. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692. 2019. arXiv: 1907.11692.
- [38] Kun Luo u. a. „Large Language Models as Foundations for Next-Gen Dense Retrieval: A Comprehensive Empirical Assessment“. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, 2024, S. 1354–1365.
- [39] Nitin Madnani u. a. „A Model of Two-Stage Grading for Short Answer Questions“. In: *Proceedings of the 8th Workshop on Innovative Use of NLP for Building Educational Applications*. 2013, S. 1–10.
- [40] Tom Mitchell u. a. „The Use of Computer-Based Feedback in the Teaching of Science“. In: *School Science Review* 84.307 (2002), S. 88–92.

- [41] Michael Mohler und Rada Mihalcea. „Text-to-Text Semantic Similarity for Automatic Short Answer Grading“. In: *Proceedings of the 12th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Association for Computational Linguistics, 2009, S. 567–575.
- [42] Lili Mou u. a. „Natural Language Inference by Tree-Based Convolution and Heuristic Matching“. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Berlin, Germany: Association for Computational Linguistics, 2016, S. 130–136.
- [43] Christopher Ormerod. *Short-Answer Scoring with Ensembles of Pretrained Language Models*. arXiv:2202.11558. 2022. arXiv: 2202.11558.
- [44] Ellis B. Page. „The Imminence of Grading Essays by Computer“. In: *The Phi Delta Kappan* 47.5 (1966), S. 238–243.
- [45] Dan Qiao u. a. „Decoder-Only LLMs Can Be Masked Auto-Encoders“. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, S. 713–723.
- [46] Nils Reimers und Iryna Gurevych. „Reporting Score Distributions Makes a Difference: Performance Study of LSTM-Networks for Sequence Tagging“. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Copenhagen, Denmark: Association for Computational Linguistics, 2017, S. 338–348.
- [47] Nils Reimers und Iryna Gurevych. „Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks“. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, S. 3982–3992.
- [48] Brian Riordan u. a. „Investigating Neural Architectures for Short Answer Scoring“. In: *Proceedings of the 12th Workshop on Innovative Use of NLP for Building Educational Applications*. Copenhagen, Denmark: Association for Computational Linguistics, 2017, S. 159–168.
- [49] Qian Ruan, Ilia Kuznetsov und Iryna Gurevych. *Are Large Language Models Good Classifiers? A Study on Edit Intent Classification in Scientific Document Revisions*. arXiv:2410.02028. 2024. arXiv: 2410.02028.
- [50] Timo Schick und Hinrich Schütze. *It's Not Just Size That Matters: Small Language Models Are Also Few-Shot Learners*. arXiv:2009.07118. 2021. arXiv: 2009.07118.
- [51] Noam Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202. 2020. arXiv: 2002.05202.
- [52] Shashank Sonkar u. a. *Automated Long Answer Grading with RiceChem Dataset*. arXiv:2404.14316. 2024. arXiv: 2404.14316.
- [53] Wolfgang G Stock und Mechtild Stock. *Wissensrepräsentation: Informationen auswerten und bereitstellen*. Bd. 2. Oldenbourg Verlag, 2008, S. 382. ISBN: 978-3-486-58439-4.
- [54] Jianlin Su u. a. „RoFormer: Enhanced Transformer with Rotary Position Embedding“. In: *Neurocomputing* 568 (2024), S. 127063.



- [55] Md Arafat Sultan, Cristobal Salazar und Tamara Sumner. „Fast and Easy Short Answer Grading with High Accuracy“. In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2016, S. 1070–1075.
- [56] Chul Sung u. a. „Pre-Training BERT on Domain Resources for Short Answer Grading“. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, S. 6071–6075.
- [57] The Hewlett Foundation. *The Hewlett Foundation: Short Answer Scoring*. Kaggle-Wettbewerb (ASAP-SAS), <https://www.kaggle.com/c/asap-sas>. Abgerufen am 05.09.2026. 2012.
- [58] UK Department for Education. *Generative Artificial Intelligence (AI) in Education*. Policy paper, <https://www.gov.uk/government/publications/generative-artificial-intelligence-in-education/generative-artificial-intelligence-ai-in-education>. Zuletzt aktualisiert am 12.08.2025, abgerufen am 05.09.2026. 2025.
- [59] Ashish Vaswani u. a. „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. 2017, S. 5998–6008.
- [60] Guangxuan Xiao u. a. „Efficient Streaming Language Models with Attention Sinks“. In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024.
- [61] Haike Xu u. a. „A Universal Discriminator for Zero-Shot Generalization“. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, 2023, S. 10559–10575.
- [62] Biao Zhang und Rico Sennrich. „Root Mean Square Layer Normalization“. In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. 2019.

Appendices

